

Practical DevOps AI

Build an AI Logging Agent
from Scratch

Quan Huynh



FREE EBOOK

Practical DevOps AI

Learn, Build, and Deploy Your First AI Logging Agent

Quan Huynh

This book is available at <https://leanpub.com/practical-devops-ai>

This version was published on 2026-02-26



This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2026 Quan Huynh

Contents

Preface	1
What Makes This Book Different	1
Who This Book Is For	2
What You'll Build	2
How to Read This Book	3
What This Book Isn't	3
The Code Repository	4
A Note on AI Evolution	4
My Promise to You	5
Let's Build Something Useful	5
 Chapter 1: Introduction to AI Agents for Logging	 6
What is an AI Agent?	6
The Problem with Traditional Tools	6
How AI Agents Help	7
A Real Example	9
When AI Actually Helps	12
The Honest Trade-offs	15
What You'll Build in This Series	18
 Chapter 2: AI Agents vs. Traditional Tools	 22
The Three Approaches to Log Analysis	22
Core Components of an AI Agent	24
How the Components Work Together	30
Design Patterns for AI Agents	31
Which Pattern Should You Use?	37
Putting It All Together	38
Real-World Example	38
What You've Learned	40
What's Next	40

Chapter 3: Understanding Core AI Building Blocks 42

 The AI Model: Your Agent’s Brain 42

 Data Retrieval: Getting Logs into Your Agent 44

 What Your Agent Needs to Know 48

 Putting It Together: Building Your Logging Agent 51

 Applying the ReAct Pattern to Your Logging Agent 54

 What You’ve Learned 61

 What’s Next 62

Chapter 4: Setting Up Your Development Environment 64

 What You’ll Need 64

 Installing Python 64

 Setting Up a Project Directory 65

 Installing Python Libraries 66

 Getting Your Gemini API Key 67

 Your First AI Agent Code 68

 Common Issues and Solutions 70

 Understanding Your Project Structure 71

 Best Practices for Development 72

 What You Can Do Now 73

 Creating Sample Log Files 74

 Next: Understanding Code Structure 76

 What You’ve Learned 76

 What’s Next 77

Chapter 5: Levels of AI Logging Systems 78

 Level 1: Basic Log Parser and Responder 78

 Level 2: Pattern Recognition and Routing Decisions 80

 Level 3: Multi-Source Integration 83

 Level 4: Collaborative Agent Systems 85

 Level 5: Autonomous Management and Remediation 87

 Mapping Your Journey 88

 What You’ve Learned 90

 What’s Next 90

Chapter 6: Introduction to LangChain for AI Logging Agents 92

 What LangChain Actually Is 92

 Installing LangChain 93

 Models 94

CONTENTS

Prompts	95
Chains	96
Agents	97
Memory	99
Tools	100
A Complete Example: Log Analyzer with LangChain	102
Raw API vs. LangChain	109
What's Next	110
Chapter 7: Building Your First Components	111
What We're Building	111
The Architecture	111
Layer 1: Configuration (config.py)	113
Layer 2: Model Wrapper (models/gemini.py)	115
Layer 3: Tools (tools/log_reader.py)	116
Layer 4: Agent Orchestration (agents/log_analyzer.py)	120
Layer 5: Response Utilities (utils/response.py)	125
Layer 6: Main Entry Point (main.py)	126
Putting It All Together	129
Testing Your Agent	130
What You've Learned	131
Chapter 8: Building a Web Interface with Streamlit	133
Why We Need a Web Interface	133
What We're Building	134
Understanding Streamlit	134
The Architecture	135
Project Structure	135
The Streamlit Application	136
Modifying the Agent	142
Running the Application	143
What Makes This Better Than the CLI	143
Session State vs Database	144
Testing Your Interface	144
Deployment Options	145
What You've Learned	146
Chapter 9: Adding Decision-Making and Actions	147
The Problem with Analysis-Only Agents	147

What We're Adding	148
Real-World Integration Patterns	148
Understanding the Workflow	150
Code Structure	152
The System Prompt	157
Testing the Agent	157
What Makes This Different	159
Extending the System	160
What's Next	160
Chapter 10: Building a Complex Agent with Actions	162
The Scenario	162
Project Structure	164
The Log Files	165
Building the Tools	166
The Model Factory	170
The Agent	172
The System Prompt	178
Configuration	180
The Web Interface	182
Response Parsing	185
Running the Agent	186
Key Design Decisions	187
What You've Learned	188
What's Next	189

Preface

I've been in DevOps for over a decade. I've seen the industry evolve from manual server provisioning to Infrastructure as Code, from monoliths to microservices, from on-premise to cloud-native. Each shift changed how we work, but they were gradual. We had time to adapt.

AI is different. This isn't a gradual shift—it's a fundamental change in what's possible.

When I first experimented with ChatGPT for log analysis in early 2023, I expected marginal improvements. Maybe it could write better regex patterns or summarize log files faster than grep. What I discovered was something far more significant: AI could understand context in ways that traditional tools never could.

This realization kept me up at night—not from stress, but from excitement and urgency. I knew what this meant for the future of DevOps, and I knew most engineers weren't prepared for it.

What Makes This Book Different

I'm not an AI researcher. I'm not a data scientist. I'm a DevOps engineer who recognized that AI would fundamentally change our field and spent two years figuring out how to use it effectively.

This book doesn't teach you machine learning theory. It teaches you how to build AI agents that solve real DevOps problems—the ones you face every day.

Practical, not theoretical. Every chapter builds working code. Every example comes from real production challenges. Every pattern has been tested in actual infrastructure.

Progressive complexity. You start simple—reading logs and getting AI analysis. Then you add decision-making, memory, multi-source integration. By the end, you have a production-ready system. No overwhelming jumps in complexity.

Future-focused. The specific APIs and models will evolve, but the patterns won't. You're learning how to think about AI in DevOps, not just how to call an API endpoint.

Honest about limitations. AI isn't magic. It makes mistakes. It needs guardrails. I'll show you exactly where it fails and how to handle those failures gracefully.

Who This Book Is For

I wrote this book for the person I was three years ago—a DevOps engineer who sees the potential of AI but doesn't know where to start.

You don't need to be an AI expert. I'm not an AI researcher. I'm a DevOps engineer who figured out how to use AI to solve real problems. This book teaches you exactly what you need to know—nothing more, nothing less.

You don't need a PhD in machine learning. We're not training models from scratch. We're using existing LLMs through APIs to build practical tools that make your job easier.

You don't need a huge budget. The examples in this book start with free APIs and local log files. You can build and test everything before spending a dollar on infrastructure.

You do need Python basics. You should be comfortable reading Python code and installing packages. If you can write a simple Flask app or automation script, you're ready.

You do need DevOps experience. This book assumes you know what logs are, why they matter, and what problems you're trying to solve. If you've ever debugged a production issue, you'll understand the motivation behind every chapter.

What You'll Build

This isn't a theory book. By the time you finish, you'll have built a working AI logging agent—not a toy example, but a real tool you can use in your day-to-day work.

Here's the journey:

Chapters 1-4 teach you the fundamentals. What AI agents are, how they differ from traditional tools, and how to set up your development environment. You'll understand the building blocks before you start building.

Chapters 5-7 get your hands dirty. You'll build a Level 1 agent that can read logs and provide intelligent analysis. It's simple but useful—something you could deploy in production for quick log summaries.

Chapters 8-9 add decision-making and memory. Your agent learns to categorize issues, route alerts to the right teams, and remember past incidents. This is where it becomes genuinely helpful for on-call engineers.

Chapters 10-11 scale up to multiple log sources. You'll connect to Elasticsearch, Kubernetes, and AWS CloudWatch. Your agent learns to correlate events across services and explain complex failures in context.

Chapter 12 makes it production-ready. Proper error handling, monitoring, security, and deployment strategies. Everything you need to run this agent reliably.

By the end, you'll have a Level 3 AI logging agent—something that provides real value to your team and saves hours of investigation time.

How to Read This Book

I wrote this book to be read sequentially, but I know DevOps engineers don't always have time to read cover-to-cover.

If you're completely new to AI: Start from Chapter 1. Don't skip the fundamentals. You need to understand what AI agents are before you build them.

If you know the basics: You might skim Chapters 1-3 and dive deep starting at Chapter 4. But read Chapter 5—understanding the five levels of AI systems is crucial for knowing what you're building.

If you just want working code: I get it. Each chapter has complete, runnable examples. You can grab the code and study it. But I recommend reading the explanations—they'll save you hours when things don't work as expected.

If you're already experienced with LLMs: You might be tempted to skip straight to the advanced chapters. Don't. The patterns and practices for DevOps-specific AI agents are different from general AI application development. At least skim the early chapters.

What This Book Isn't

Let me be clear about what you won't find here:

This isn't a machine learning textbook. We're not training neural networks or fine-tuning models. If you want to understand transformer architectures or backpropagation, look elsewhere.

This isn't a comprehensive LLM guide. We use LLMs as tools through APIs. I'll teach you prompt engineering for log analysis, but this isn't a deep dive into LLM theory.

This isn't enterprise architecture guidance. This book shows you how to build agents. Deploying them at scale, integrating them with existing enterprise systems, and managing them in complex environments—that's a different book.

This isn't a replacement for proper logging infrastructure. AI agents enhance your existing logging setup; they don't replace it. You still need tools like Elasticsearch, proper log aggregation, and monitoring systems.

The Code Repository

All the code examples in this book are available in the accompanying GitHub repository. You'll find:

- Complete working examples for each chapter
- Sample log files for testing
- Configuration templates
- Deployment scripts
- Additional resources and updates

The code is MIT licensed—use it however you want. Fork it, modify it, deploy it in production. If you find bugs or improvements, pull requests are welcome.

A Note on AI Evolution

AI technology moves fast. By the time you read this, there might be newer models, better frameworks, or different best practices. That's okay.

The fundamental principles in this book—how to structure agents, how to prompt for log analysis, how to correlate events across services—these don't change. The specific API calls might evolve, but the patterns remain valuable.

I'll update the code repository as tools evolve, but the book focuses on timeless concepts that will serve you regardless of which LLM you're using.

My Promise to You

I promise to be honest about limitations. AI agents aren't perfect. They make mistakes. They sometimes hallucinate details that aren't there. I'll show you how to catch these issues and build guardrails.

I promise to keep it practical. Every technique in this book comes from real production experience. I'm not teaching theory—I'm teaching what actually works.

I promise to respect your time. No fluff, no filler. Every chapter teaches you something specific and useful. If I'm explaining something, it's because you need to know it.

Let's Build Something Useful

You're about to learn how to build AI agents that solve real DevOps problems. Not demos, not prototypes—actual tools that will save you time and make your on-call shifts less stressful.

I wish I had this book three years ago. Maybe I wouldn't have spent so many late nights staring at logs, manually connecting dots that an AI agent could have connected in seconds.

But now you have it. Let's get started.

– *Your fellow DevOps engineer who got tired of manual log analysis*

Chapter 1: Introduction to AI Agents for Logging

What is an AI Agent?

An AI agent is a piece of software that can think, learn, and act on its own. In the world of DevOps, an AI agent for logging reads your log files, understands what's happening in your systems, and helps you find and fix problems faster.

Think of it like having a smart assistant who watches your application logs 24/7. This assistant doesn't just search for keywords—it actually understands what your logs mean. When something goes wrong, it can connect the dots across different services and tell you what's really happening.

Here's what makes an AI agent different from regular scripts:

- It reads logs like a human would, understanding the meaning behind error messages
- It spots patterns and connections you might miss
- It learns what “normal” looks like in your system
- It can explain its findings in plain language
- It remembers past problems and uses that knowledge to solve new ones

The Problem with Traditional Tools

Let's be honest: tools like Kibana, Datadog, and Splunk are excellent. They collect your logs, make them searchable, and show you nice dashboards. So why would you need anything else?

The answer is simple. These tools show you data, but they don't understand it.

What You Still Have to Do Manually

When something breaks in production, here's what typically happens:

1. You get an alert that something is wrong
2. You open your logging tool
3. You search through logs trying to figure out what happened
4. You check different services one by one
5. You try to connect events across multiple systems
6. Eventually, after 30 minutes to a few hours, you find the root cause

The problem isn't the tool—it's that you need to know what to look for. You need to:

- Write the right search queries
- Know which services to check
- Understand how different errors relate to each other
- Remember similar problems from the past
- Manually connect events across your microservices

This takes time and experience. A junior engineer might struggle for hours on something a senior engineer would spot in minutes.

How AI Agents Help

An AI agent sits on top of your existing logging infrastructure. It connects to tools like Elasticsearch or AWS CloudWatch and analyzes the logs that are already there. You're not replacing your logging system—you're adding intelligence to it.

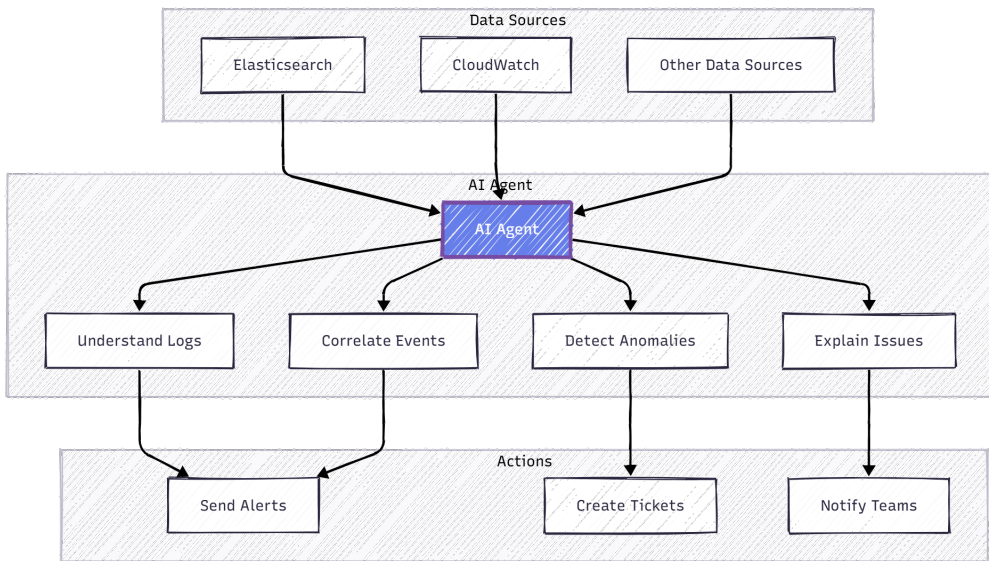


Figure 1.1: AI Agent Architecture - The agent connects to existing log sources and adds an intelligence layer on top.

What AI Does Better

Here are the specific ways AI improves log analysis:

Understanding Natural Language

Developers write log messages in plain English. “Connection timeout after 30 seconds” and “Failed to connect within timeout period” mean the same thing, but traditional tools treat them as different strings.

An AI agent understands both messages mean the same problem. It groups related errors together automatically, even when they’re worded differently.

Connecting Events Across Services

Modern applications have many moving parts. A single user request might touch 10 different services. When something fails, the root cause might be three services away from where you see the error.

Traditional tools can search across all your logs, but you have to manually figure out which events are related. An AI agent does this automatically. It tracks how your services talk to each other and connects related problems.

Finding Unknown Problems

With traditional tools, you can only alert on problems you know about. You write a rule: “Alert me if error rate goes above 5%.” But what about problems you haven’t seen before?

An AI agent learns what normal behavior looks like for your system. When something unusual happens—even if you never wrote a rule for it—the agent notices and tells you.

Remembering Past Incidents

When you solve a problem, that knowledge usually lives in a postmortem document or someone’s memory. If the same problem happens again months later, you might not remember the solution.

An AI agent remembers past incidents. When it sees similar patterns, it can tell you: “This looks like the database connection issue from March that was caused by a configuration change.”

A Real Example

Let me show you a concrete scenario where AI makes a real difference.

The Scenario

You run an online store. The customers are reporting that checkout is slow. You have logs flowing into Elasticsearch from:

- Frontend (React app)
- Backend API (Node.js)
- Payment service (Java)
- Database (PostgreSQL)
- Message queue (RabbitMQ)

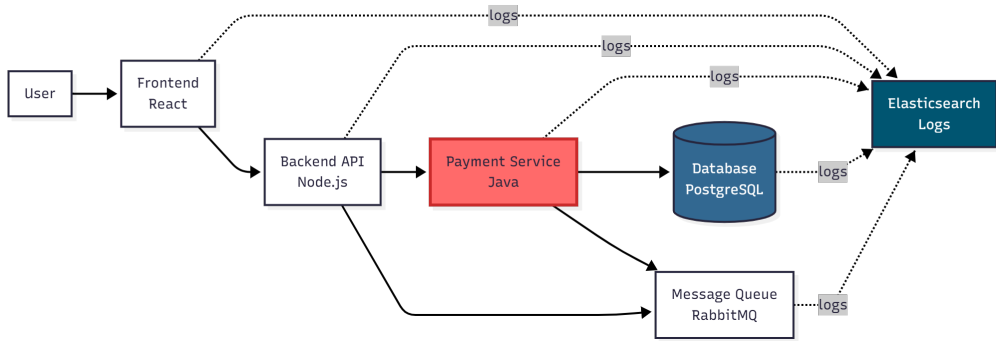


Figure 1.2: Microservices Architecture - The payment service (highlighted) is the source of the problem, but symptoms appear across all services.

Without AI: Manual Investigation

You wake up to the alert and open Kibana.

First, you search for “checkout” errors. No obvious errors—everything returns 200 OK.

You check your dashboard. Response times have been climbing for the past 2 hours. But that doesn’t tell you why.

You search the payment service logs. Transactions are completing successfully. Nothing looks wrong here.

You check the database logs. The connection pool is at 80%. That’s high, but not at the limit yet. Is this the problem? You’re not sure.

You search for “timeout” across all services. Now you find scattered timeout errors in different places. Are they related? You start piecing it together.

After 30 minutes, you figure it out:

- The payment service has a slow memory leak
- This causes occasional database connection timeouts
- The payment service retries failed transactions
- These retries create more messages in the queue
- The queue builds up
- The backend waits on the payment service
- The frontend times out waiting on the backend

Root cause: memory leak in the payment service. You restart it and everything recovers.

With AI: Automatic Analysis

Same scenario, but now you have an AI agent connected to your Elasticsearch.

The agent has been watching your logs continuously. It notices:

- Payment service response times slowly increasing
- Database connection pool usage rising at the same rate
- Retry patterns appearing in queue messages
- Memory usage in payment service containers growing

The agent recognizes this pattern. It's seen similar resource leak signatures before. Instead of waiting for everything to break, it alerts you 10 minutes after symptoms start:

```
1 Alert: Resource Leak Detected
2 Service: payment-service
3 Confidence: 85%
4
5 What's happening:
6 Payment service is not releasing database connections properly.
7 This is causing cascading delays across dependent services.
8
9 Evidence:
10 - Payment service p95 latency up 40% over 2 hours
11 - DB connection pool usage rising in sync with latency
12 - Retry patterns in queue indicate connection failures
13 - Memory usage trending upward in payment-service pods
14 - Similar pattern seen in incident from September 15
15
16 Affected services:
17 - payment-service (primary)
18 - api-backend (secondary)
19 - frontend (tertiary)
20
21 What to do:
22 1. Restart payment-service pods immediately
23 2. Check recent code changes in DB connection handling
24 3. Review connection pool configuration
```

You restart the payment service based on the recommendation. Problem solved.

The Difference

The AI didn't do anything magical. You would have figured out the problem eventually. The difference is speed and clarity.

The AI agent:

- Connected logs across multiple services automatically
- Recognized the pattern from past incidents
- Explained the problem in plain language
- Suggested specific actions to take

This is where AI adds real value—not by replacing your tools, but by doing the analysis work faster than you can manually.

When AI Actually Helps

AI for logging isn't right for everyone. Let's be specific about when it makes sense.

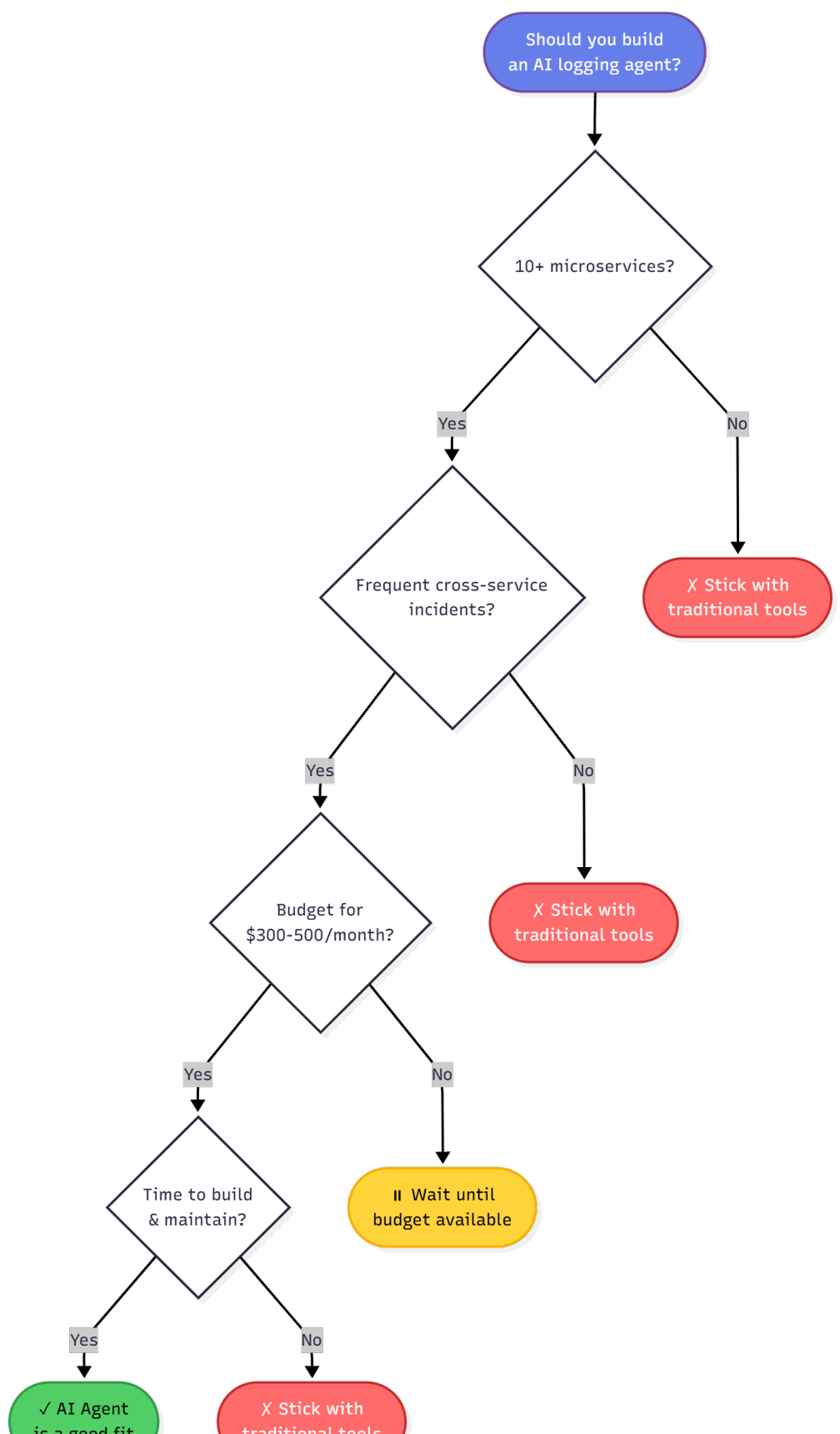


Figure 1.3: Decision Tree – Use this flowchart to determine if an AI logging agent makes sense for your situation.

Good Use Cases

You Have Many Microservices

If you have 10+ services that talk to each other, tracking problems across service boundaries gets hard. The more services you have, the more valuable automatic correlation becomes.

Incidents Involve Multiple Services

When problems span multiple services, finding the root cause takes time. If this happens weekly or more, AI can significantly reduce investigation time.

You Get Unexpected Problems

If you frequently encounter issues you haven't seen before, AI's ability to detect unusual patterns helps. Traditional alerts only catch known problems.

Your Team is Growing

New team members don't have the experience to quickly diagnose issues. An AI agent helps level the playing field by providing context and guidance.

You Want Faster Resolution

If reducing mean time to resolution (MTTR) is important for your business, AI can cut investigation time by 60-80%.

When to Stick with Traditional Tools

Your System is Simple

If you have 5 or fewer services with straightforward interactions, manual correlation is fast enough. The extra complexity isn't worth it.

Your Alerts Work Well

If your current alerting catches problems quickly and false positives are rare, you might not need AI. Don't fix what isn't broken.

Budget is Tight

AI costs money (more on this below). If budget is limited and incidents are rare, spend your money elsewhere.

You Don't Have Time to Build

Building an AI agent properly takes weeks. If you can't commit that time, don't start.

The Honest Trade-offs

Let's talk about the downsides, because there are real costs and limitations.

Cost

Running an AI agent costs money in several ways:

LLM API Costs

Using services like OpenAI's GPT-4 or Anthropic's Claude costs money per request. Depending on your log volume, expect \$200-500 per month for a medium-sized system.

Hosting

The agent needs to run somewhere. This adds \$50-100 per month for compute resources.

Your Time

Building and tuning the agent takes several weeks of engineering time upfront, plus ongoing maintenance.

Total Cost Example

Here's what it might look like for a typical setup:

- Existing logging (Elasticsearch/Kibana): \$500-1000/month (you already pay this)
- AI agent LLM costs: \$300/month (new)
- AI agent hosting: \$75/month (new)
- Engineering time: 4-6 weeks initial + 5 hours/month maintenance

Is this worth it? Do the math on your team's time. If you're spending 10+ hours per week investigating incidents, the ROI is clear. If incidents are rare, it's harder to justify.

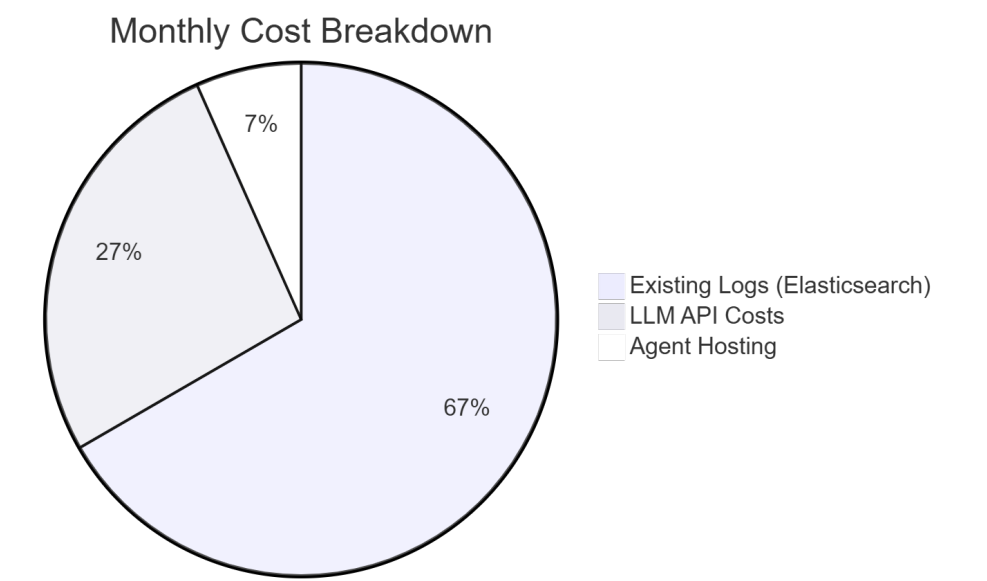


Figure 1.4: Cost Distribution - Most cost is your existing logging infrastructure. AI adds about 33% to monthly costs.

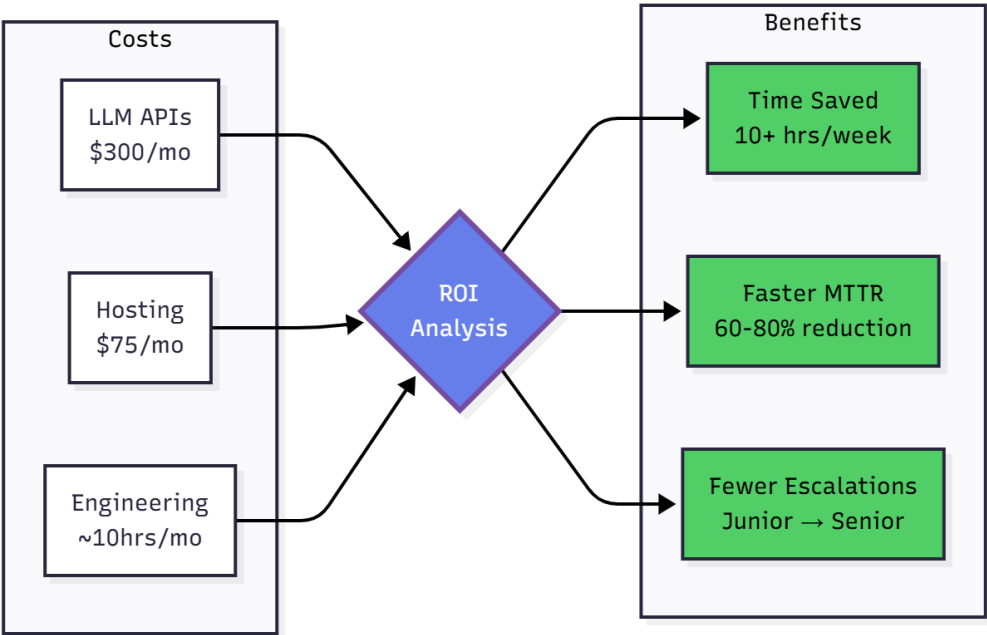


Figure 1.5: ROI Analysis - Weigh costs against time savings and faster incident resolution.

Complexity

An AI agent is another system to maintain. It can break. It needs monitoring. It requires updates. And here's the important part—you can't remove your existing monitoring. The AI is supplementary, not a replacement.

This means more infrastructure, not less.

Not Perfect

AI agents make mistakes. You'll see:

False Positives

The agent might flag things as problems when they're actually fine. Especially in the first few weeks while it's learning your system.

False Negatives

Sometimes the agent misses real problems. It's not foolproof.

Wrong Explanations

The agent might correlate things that aren't actually related. It can sound very confident about incorrect conclusions.

Unusual Patterns

When something truly novel happens, the agent might struggle to make sense of it.

You need human oversight. Think of the AI as a junior engineer—helpful and fast, but you should verify its conclusions before taking major actions.

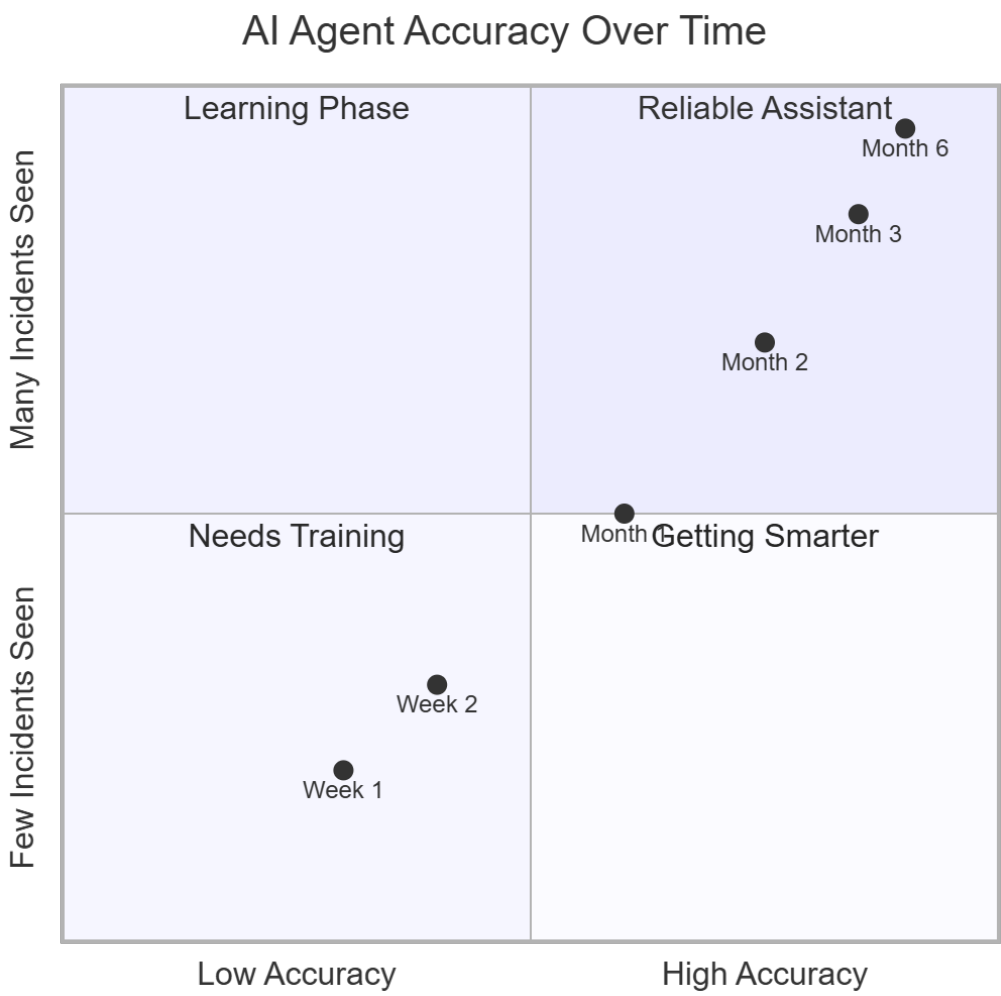


Figure 1.6: AI Learning Curve - The agent gets more accurate as it learns your system’s normal patterns.

When Not to Use AI

Don’t build an AI agent if:

- Your logging needs are simple and well-handled
- You rarely have production incidents
- Your team is too small to maintain another system
- You’re just chasing the AI hype
- You expect it to solve all problems automatically

What You'll Build in This Series

By the end of this series, you'll have a working AI logging agent. Not a toy—something you can run against real production logs.

How It Works

The agent connects to your existing log infrastructure:

```
1  # Connect to your existing logs
2  agent = LoggingAgent()
3
4  agent.add_datasource(
5      type="elasticsearch",
6      host="logs.company.com",
7      indices=["app-*", "service-*"]
8  )
9
10 agent.add_datasource(
11     type="aws_cloudwatch",
12     region="us-east-1",
13     log_groups=["/aws/lambda/*"]
14 )
15
16 # Start analyzing
17 agent.start()
```

We're not building a log collector. You already have Elasticsearch or CloudWatch doing that job. We're building the intelligence layer on top.

What the Agent Does

Pulls Logs

The agent queries your existing log systems at regular intervals, just like you would manually.

Analyzes Patterns

It uses AI to understand what the logs mean and identify patterns.

Correlates Events

It connects related events across different services.

Detects Anomalies

It flags unusual behavior, even without explicit rules.

Explains Findings

Instead of just alerting, it explains what's happening and why.

Takes Action

It can create tickets, send notifications, or trigger other workflows.

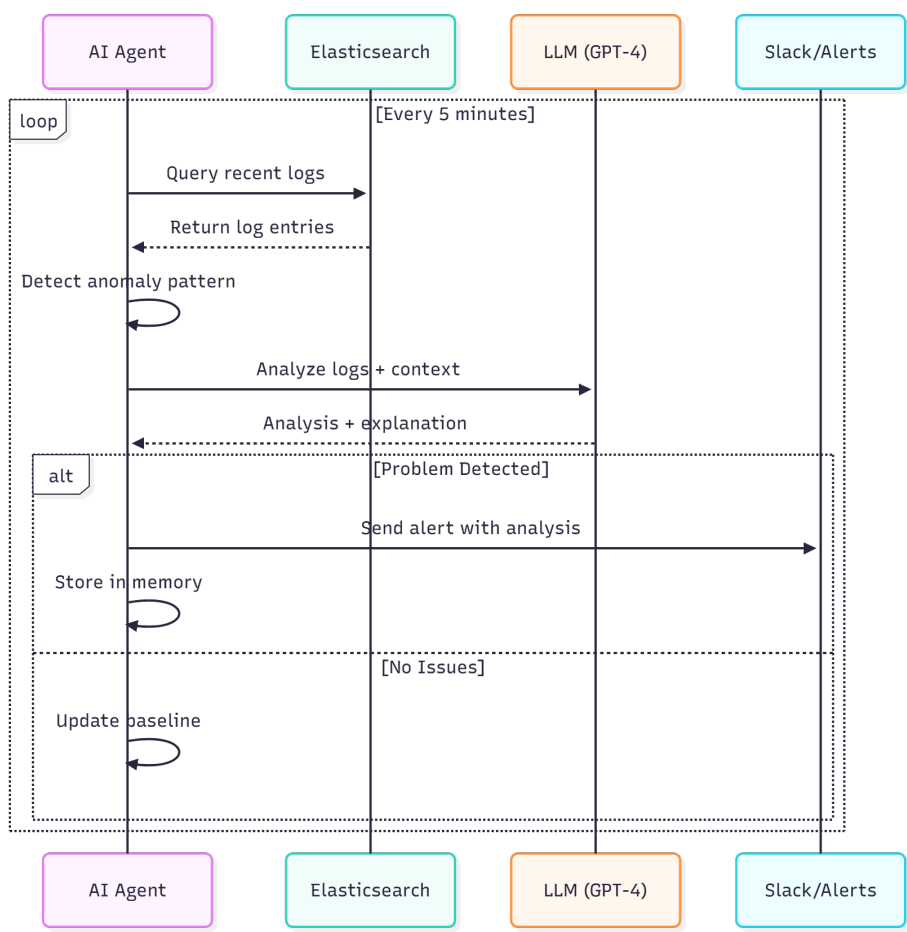


Figure 1.7: Agent Operation Flow - The agent continuously monitors logs, analyzes patterns, and alerts when issues are detected.

What You'll Learn

Building this agent teaches you:

AI Fundamentals

How large language models work, how to use them effectively, and how to manage costs.

Agent Design

How to build systems that can reason and act autonomously.

Data Integration

How to connect to Elasticsearch, CloudWatch, and other log sources properly.

Pattern Recognition

How to teach AI to identify what matters in your logs.

Production Skills

Error handling, cost optimization, monitoring, and reliability.

Let's get started.

Chapter 2: AI Agents vs. Traditional Tools

Before you build an AI logging agent, you need to know where it fits in your existing setup. You already have scripts, you probably use Elasticsearch or Splunk, and now you're looking at adding AI to the mix.

This chapter shows you the real differences between these approaches. More importantly, you'll learn the specific pieces that make an AI agent work and how they fit together.

The Three Approaches to Log Analysis

You have three basic options when it comes to handling logs. Let me show you what each one does and where it falls short.

Basic Scripts

Most teams start here. You write a Python or Bash script that searches log files for specific keywords, counts how many errors you're seeing, and sends an email when certain patterns show up.

Example:

```
1 # Simple log monitoring script
2 def check_logs():
3     with open('/var/log/app.log', 'r') as f:
4         for line in f:
5             if 'ERROR' in line and 'database' in line:
6                 send_alert("Database error found!")
```

Scripts are simple to write, they don't need external dependencies, and you have complete control over the logic. But they only find what you explicitly program them to find. They don't learn or adapt. When your log format changes, they break. And they can't handle complex patterns that span multiple services.

Traditional Logging Tools (ELK, Splunk, Datadog)

These are the enterprise-grade platforms that collect logs from all your services, index and store them efficiently, provide search and visualization, and support alerting rules.

Example workflow:

- 1 1. **Logs** → Filebeat → Elasticsearch
- 2 2. Create dashboard in Kibana
- 3 3. Set alert: **if** error_count > 100, **notify** team
- 4 4. When alert fires, manually investigate

These tools are battle-tested and reliable. They handle massive log volumes, provide great search and visualization, and do real-time monitoring. But here's the catch: you still define all the rules. They don't automatically correlate issues across services. They can't understand context or meaning. They're limited to patterns you already know about.

AI Agents

An AI agent is different. It connects to your existing log infrastructure, uses AI models to actually understand what the logs are saying, automatically correlates events across services, learns what normal patterns look like, and explains what it finds in plain language.

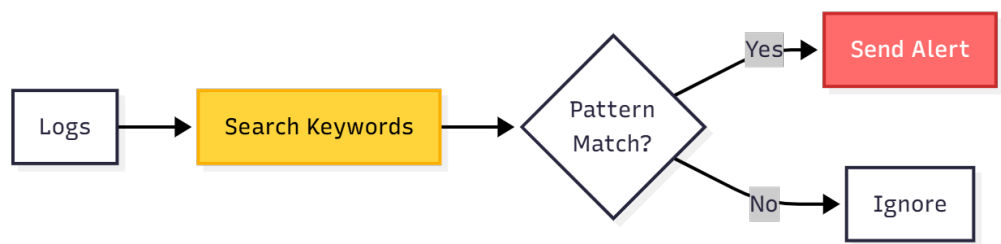
Example workflow:

- 1 1. Agent pulls **logs** from Elasticsearch
- 2 2. AI analyzes: "**Payment service errors correlate with**
- 3 **database connection pool exhaustion**"
- 4 3. Agent **explains**: "**Memory leak in payment service causing**
- 5 **cascading failures. Similar to incident on Sept 15.**"
- 6 4. Recommends: "**Restart payment service, review recent changes**"

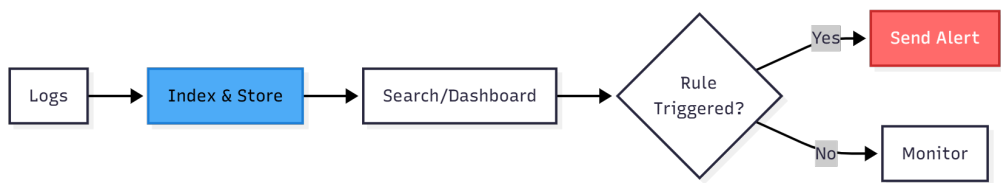
This is where things get interesting. AI agents find issues you didn't anticipate. They understand context and relationships between services. They learn from past incidents. Investigation time drops significantly—what used to take 45 minutes might take 10.

But there are trade-offs. They're more complex to build and maintain. They cost money for the LLM API calls. They're not 100% accurate. And you still need human oversight to catch when they get things wrong.

Basic Scripts:



Traditional Tools (ELK/Splunk):



AI Agents:



Figure 2.1: Three Approaches - Each approach has different capabilities and complexity.

Core Components of an AI Agent

An AI agent isn't magic. It's built from six specific pieces that you can understand and implement yourself. Let me break down each one.

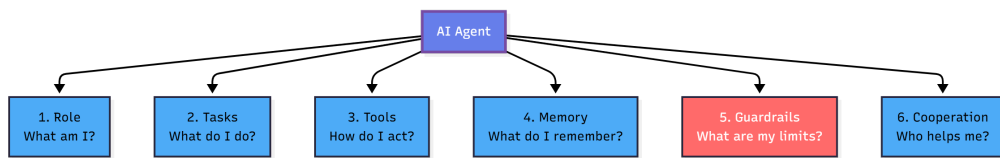


Figure 2.2: Six Core Components - Every AI agent is built from these building blocks.

1. Role

Think of the role as the agent's job description. It defines what your agent is and what it's responsible for. It sets the context for everything the agent does.

Example role definition:

```

1 role = """
2 You are a DevOps log analyst. Your job is to monitor application
3 and system logs, identify problems, and explain root causes.
4
5 You have access to logs from multiple services and past incident data.
6 You understand log formats, common error patterns, and how services
7 interact with each other.
8 """

```

That's it. Not complicated, but it matters. The role gives the agent context about what it is and what it should focus on.

For our logging agent, the role covers monitoring logs from multiple sources, understanding DevOps terminology and patterns, identifying anomalies and correlations, and explaining findings clearly.

2. Focus and Tasks

The role says what the agent is. Tasks say what it actually does.

Example tasks:

```

1 tasks = [
2     "Check logs every 5 minutes for error patterns",
3     "Correlate errors across frontend, backend, and database",
4     "Compare current behavior to historical patterns",
5     "When anomaly detected, identify root cause",
6     "Generate clear explanation with evidence",
7     "Suggest remediation actions"
8 ]

```

Be specific here. “Monitor logs” is too vague. “Check logs every 5 minutes for error patterns” is clear. The more specific your tasks, the better your agent performs.

3. Tools

Tools are the functions your agent can call to actually do things—query databases, check metrics, send alerts. The agent decides which tools to use and when. This is where the AI adds value, because it figures out which tool makes sense for each situation.

Example tools for a logging agent:

```

1 # Tool 1: Query Elasticsearch
2 def query_elasticsearch(index, query, time_range):
3     """Search logs in Elasticsearch"""
4     return elasticsearch_client.search(
5         index=index,
6         body=query,
7         time_range=time_range
8     )
9
10 # Tool 2: Get service metrics
11 def get_service_metrics(service_name, metric_type):
12     """Retrieve metrics like CPU, memory, error rate"""
13     return metrics_api.get(service_name, metric_type)
14
15 # Tool 3: Check past incidents
16 def search_incidents(keywords):
17     """Find similar past incidents"""
18     return incident_db.search(keywords)
19
20 # Tool 4: Send alert
21 def send_alert(severity, message, evidence):
22     """Send alert to Slack or PagerDuty"""
23     return alert_system.send(severity, message, evidence)

```

You give the agent a toolbox. When it encounters a problem, it picks the right tool for the job.

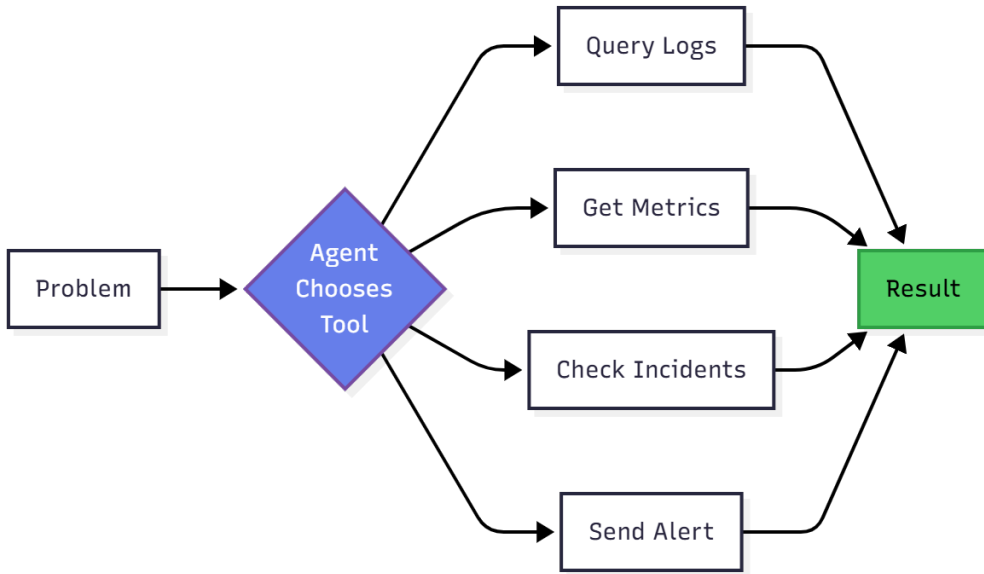


Figure 2.3: Tool Selection - Agent decides which tools to use based on the situation.

4. Memory

Memory lets the agent remember things—both from the current investigation and from the past. Without memory, the agent starts from scratch every time. With it, the agent learns.

Two types of memory:

Short-term memory is what's happening right now—the current conversation or analysis session. Long-term memory is historical context—past incidents, learned patterns, and what normal behavior looks like.

Short-term memory: Current conversation or analysis session

```

1 # Example short-term memory
2 current_session = {
3     "logs_analyzed": [...],
4     "patterns_found": [...],
5     "hypotheses": [...],
6     "tools_used": [...]
7 }

```

Long-term memory: Past incidents, learned patterns, historical context

```

1 # Example long-term memory
2 knowledge_base = {
3     "past_incidents": [
4         {
5             "date": "2024-09-15",
6             "issue": "Database connection leak",
7             "symptoms": ["timeouts", "pool exhaustion"],
8             "solution": "Restart service, fix connection handling"
9         }
10    ],
11    "normal_patterns": {
12        "payment_service": {
13            "avg_response_time": 150,
14            "error_rate": 0.1
15        }
16    }
17 }

```

This is what lets the agent say things like: “This looks like the incident from September 15.” It’s connecting past knowledge to present problems.

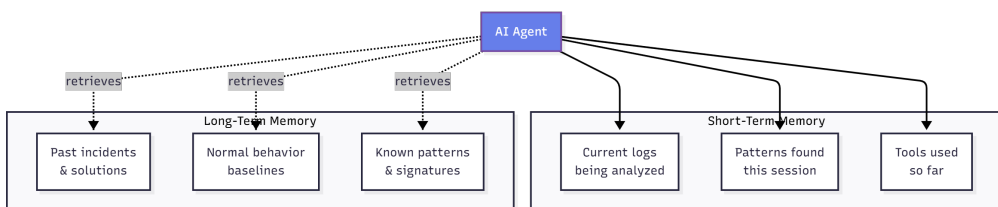


Figure 2.4: Memory Types - Short-term for current session, long-term for historical knowledge.

5. Guardrails

Guardrails stop the agent from doing harmful or expensive things. An AI agent without proper guardrails can spam your team with alerts, rack up a huge API bill, give you wrong information with complete confidence, or try to take actions you didn't intend.

You need guardrails.

Example guardrails:

```

1  guardrails = {
2      # Don't restart production services automatically
3      "no_auto_restart": True,
4
5      # Always require human approval for destructive actions
6      "require_approval_for": ["restart", "deploy", "delete"],
7
8      # Limit how many API calls the agent can make per minute
9      "rate_limit": {
10         "elasticsearch": 10,
11         "metrics_api": 20
12     },
13
14     # Don't send more than 5 alerts in 10 minutes
15     "alert_throttle": {
16         "max_alerts": 5,
17         "time_window": 600 # seconds
18     },
19
20     # Validate all outputs before sending
21     "output_validation": True
22 }

```

These checks run before every action. Rate limits prevent API spam. Alert throttles prevent notification fatigue. Approval requirements prevent destructive actions. Validation catches bad outputs before they go anywhere.

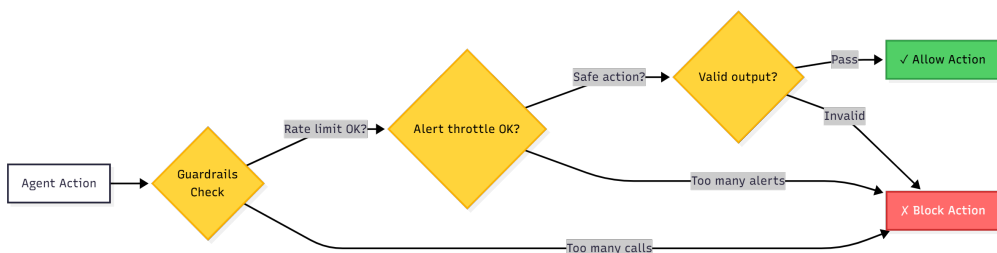


Figure 2.5: Guardrails in Action - Multiple safety checks before allowing actions.

6. Cooperation (Optional)

In more advanced setups, you can have multiple agents work together, each with a specialized role. One agent might parse logs, another finds patterns, a third determines root cause, and a fourth communicates the results.

Example multi-agent setup:

```
1  # Agent 1: Log Parser
2  log_parser_agent = {
3      "role": "Parse and categorize logs",
4      "focus": ["Extract structured data", "Identify log types"]
5  }
6
7  # Agent 2: Pattern Analyzer
8  pattern_agent = {
9      "role": "Identify patterns and correlations",
10     "focus": ["Cross-service correlation", "Trend detection"]
11 }
12
13 # Agent 3: Root Cause Analyzer
14 root_cause_agent = {
15     "role": "Determine root cause of issues",
16     "focus": ["Hypothesis generation", "Evidence gathering"]
17 }
18
19 # Agent 4: Communicator
20 comm_agent = {
21     "role": "Explain findings to humans",
22     "focus": ["Clear explanations", "Actionable recommendations"]
23 }
```

For this series, we're building a single agent. That's simpler and works fine for most teams. Once you have that working, you can always split it into multiple specialized agents if you need to scale.

How the Components Work Together

Now that you know the six pieces, let me show you how they work together when something actually goes wrong.

Scenario: Error rate spike in your API service

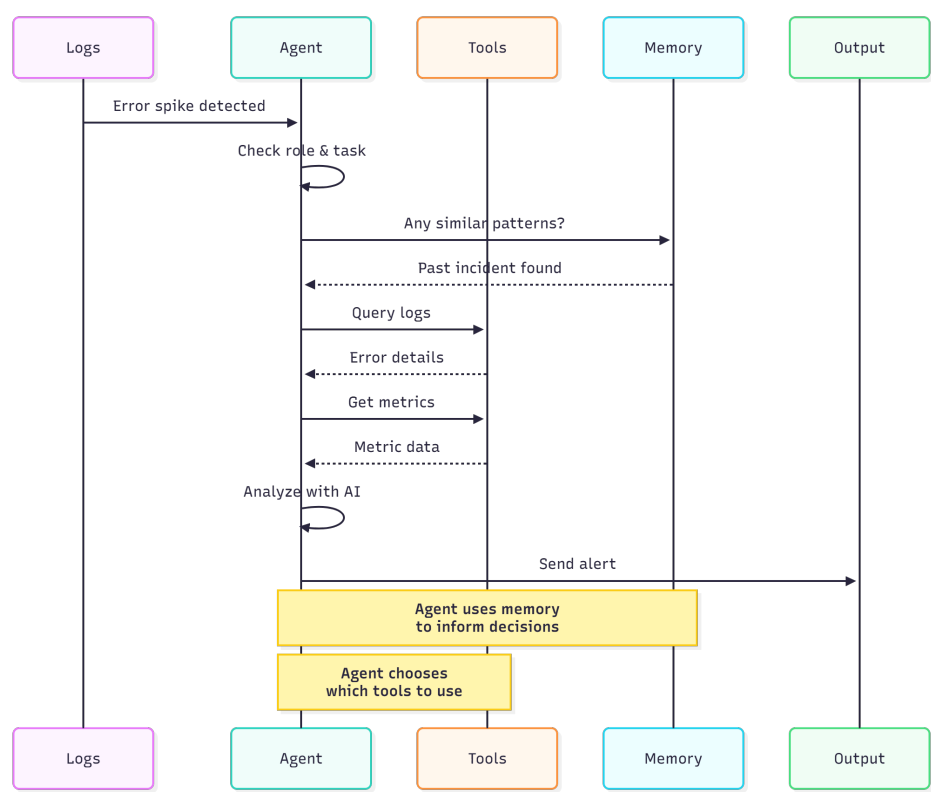


Figure 2.6: Component Flow - How components interact during a real incident.

Here’s what happens step by step. Error logs appear—that’s the trigger. The agent checks its role and confirms it’s responsible for this. It follows its defined tasks. It queries memory to see if it’s seen similar patterns before. It finds a past incident that looks related.

Now it needs more information. It uses tools to query Elasticsearch for log details, get metrics to see system health, and search past incidents for solutions. The AI model processes all this data and identifies the pattern. Guardrails validate that the actions are safe. The agent updates its memory with what it learned. Finally, it sends an alert with a clear explanation.

That’s the flow. All the components working together, not in isolation.

Design Patterns for AI Agents

A design pattern is a proven way to solve a common problem. For AI agents, patterns describe how the agent makes decisions and takes actions. Here are the five main patterns you'll see.

Pattern Overview

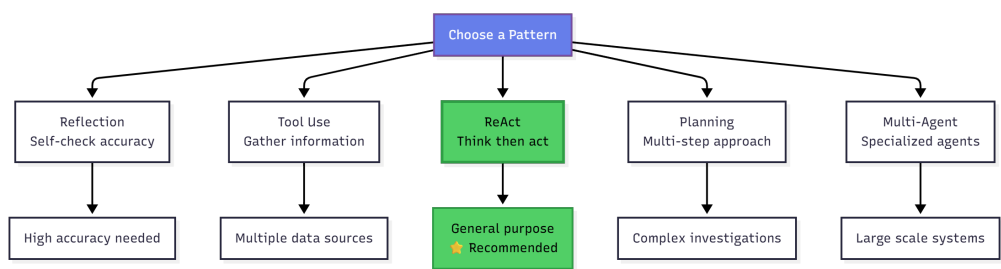


Figure 2.7: Design Patterns Overview - ReAct is recommended for most use cases.

1. Reflection Pattern

With the reflection pattern, the agent checks its own work before committing to an answer. It analyzes the logs, forms a hypothesis, then stops and asks itself: “Wait, is this actually correct? What evidence contradicts this?”

Maybe it realizes the timeline doesn’t match, or the symptoms don’t fully align. So it revises its answer to something more accurate. You use this pattern when accuracy matters more than speed.

How it works:

- 1 1. Agent analyzes logs and forms a hypothesis
- 2 2. Agent asks itself: "Is this conclusion correct?"
- 3 3. Agent checks for contradictory evidence
- 4 4. Agent revises hypothesis if needed
- 5 5. Agent provides final answer with confidence level

Example:

```
1 # First analysis
2 hypothesis = "Database connection pool exhausted"
3
4 # Reflection
5 reflection = ""
6 Wait, let me verify this hypothesis:
7 - Connection pool shows 80% usage, not 100%
8 - Errors started before pool was stressed
9 - Timeline doesn't match
10
11 Let me reconsider... The connection pool issue is a
12 symptom, not the cause. The real issue is the memory
13 leak causing timeouts, which then stresses the pool.
14 ""
15
16 # Revised conclusion
17 conclusion = "Memory leak in payment service causing
18             connection timeouts, leading to pool stress"
```

The agent doesn't just give you its first answer. It double-checks itself and gives you the revised, more accurate conclusion.

When to use: When accuracy is more important than speed.

2. Tool Use Pattern

The tool use pattern is about the agent deciding which tools to use based on what it's trying to figure out. You're investigating an error spike. The agent has four tools: query logs, get metrics, check deployments, search past incidents. It picks which ones to use and in what order based on what it learns at each step.

The key is you don't hard-code the sequence. The AI figures it out dynamically.

How it works:

- 1 1. Agent receives a problem
- 2 2. Agent considers available **tools**
- 3 3. Agent chooses the right **tool(s)**
- 4 4. Agent uses **tools** **to** gather information
- 5 5. Agent synthesizes results

Example:

```
1 available_tools = [  
2     "query_elasticsearch",  
3     "get_metrics",  
4     "search_incidents",  
5     "check_deployments"  
6 ]  
7  
8 # Agent reasoning  
9 """  
10 I need to investigate an error spike. Let me:  
11 1. query_elasticsearch - get recent error logs  
12 2. get_metrics - check if metrics show anomaly  
13 3. check_deployments - see if recent deploy related  
14 4. search_incidents - find similar past issues  
15 """
```

The agent looks at the problem and picks the right tools to gather the information it needs.

When to use: When the agent needs to gather information from multiple sources.

3. ReAct Pattern (Reasoning + Acting)

ReAct is the pattern I recommend you start with. It's straightforward: the agent alternates between thinking and doing.

The agent thinks about what it needs to know, acts by using a tool, observes what came back, thinks about what that means, acts again with another tool, and keeps going until it solves the problem. This loop of think → act → observe continues until the agent reaches a conclusion.

How it works:

```
1 1. Thought: What do I need to find out?  
2 2. Action: Use a tool to get information  
3 3. Observation: Analyze the result  
4 4. Thought: What does this tell me? What next?  
5 5. Action: Use another tool  
6 ... (repeat until solved)  
7 6. Conclusion: Here's what I found
```

Example:

```
1 # ReAct loop  
2 Thought: "Error rate is high. I need to find out why."  
3 Action: query_elasticsearch("error logs last 10 minutes")  
4 Observation: "Mostly 'InvalidToken' errors in auth service"  
5  
6 Thought: "Token errors. Was there a recent change?"  
7 Action: check_deployments("auth service")  
8 Observation: "Deploy 30 minutes ago changed token expiry"  
9  
10 Thought: "That matches the timeline. Let me confirm."  
11 Action: get_metrics("auth service", "deployment impact")  
12 Observation: "Error rate spiked exactly after deploy"  
13  
14 Conclusion: "Deploy changed token expiry, breaking validation.  
15             Recommend rollback or config fix."
```

It's simple to implement and works well for most situations. This is your default choice.

When to use: Most general-purpose use case. Good default choice.

4. Planning Pattern

The planning pattern is for complex investigations that need structure. Instead of just jumping in, the agent first creates a plan. It breaks the problem into smaller pieces, figures out the order to tackle them, then executes each step. If something unexpected happens, it can adjust the plan before continuing.

You use this when the investigation is complicated and you want a methodical approach. It's more overhead than ReAct, so only use it when you need that structure.

How it works:

- 1 1. Agent receives a complex problem
- 2 2. Agent breaks it **into** sub-problems
- 3 3. Agent creates a **step-by-step** plan
- 4 4. Agent executes each **step**
- 5 5. Agent adjusts plan **if** needed

Example:

```
1  # Planning for complex investigation
2  plan = [
3      "Step 1: Identify which service is the primary source",
4      "Step 2: Get timeline of when problem started",
5      "Step 3: Check for correlating events (deploys, config changes)",
6      "Step 4: Analyze log patterns for root cause",
7      "Step 5: Verify hypothesis with metrics",
8      "Step 6: Search for similar past incidents",
9      "Step 7: Generate report with recommendations"
10 ]
11
12 # Execute plan step by step
13 for step in plan:
14     result = execute_step(step)
15     if result.requires_plan_change:
16         adjust_plan(result)
```

The agent makes a plan, follows it, and adapts if needed.

When to use: For complex, multi-step investigations.

5. Multi-Agent Pattern

The multi-agent pattern uses multiple specialized agents working together. You have a main agent that coordinates, and specialist agents for specific tasks. One parses logs, another finds patterns, a third determines root cause. The main agent brings everything together.

This is the most complex pattern. You only need it for very large systems or when you want different teams handling different parts. For most cases, including what we're building here, a single agent is enough.

How it works:

- 1 1. Main agent receives problem
- 2 2. Main agent delegates **to** specialized agents
- 3 3. Each specialist does its job
- 4 4. Main agent coordinates **and** synthesizes results

Example:

```
1  # Main agent delegates
2  main_agent.delegate("log_parser", task="parse and categorize logs")
3  main_agent.delegate("pattern_finder", task="find correlations")
4  main_agent.delegate("root_cause", task="determine cause")
5
6  # Specialized agents work
7  log_parser_result = log_parser_agent.execute()
8  pattern_result = pattern_agent.execute(log_parser_result)
9  root_cause_result = root_cause_agent.execute(pattern_result)
10
11 # Main agent synthesizes
12 final_report = main_agent.synthesize([
13     log_parser_result,
14     pattern_result,
15     root_cause_result
16 ])
```

Each specialist does its job, and the main agent synthesizes everything into a final answer.

When to use: For very complex systems or when you want specialists for different tasks.

Which Pattern Should You Use?

For our logging agent, we're going with ReAct. Here's why.

ReAct is simple enough that you can understand what's happening at each step. When something goes wrong, you can debug it because you can see the agent's reasoning. It's flexible enough to handle most situations you'll encounter in log analysis. And it's well-supported by AI frameworks, so you're not building everything from scratch.

We're skipping Reflection because it adds complexity you don't need initially. We're skipping Planning because it's overkill for log analysis. And we're skipping Multi-Agent because it's too much for a first project.

You can add any of these later once your basic agent works. Start simple, then expand.

Putting It All Together

Let me show you how all these pieces connect in a real logging agent.

You have logs coming from Elasticsearch and CloudWatch. The agent has a role—it's a log analyst. It has tasks—monitor and analyze those logs. It uses the ReAct pattern to structure its work. When it needs to act, it has tools for querying logs, getting metrics, searching history, and sending alerts. It has memory to track what it's doing now and what happened in the past. And it has guardrails to make sure it doesn't do anything harmful.

All these components work together. The role and tasks give direction. The ReAct pattern provides structure. The tools enable action. Memory adds context. Guardrails keep it safe.

That's the architecture. Once you see how the pieces fit, it's not that complicated.

Real-World Example

Let's walk through a complete scenario so you can see everything working together.

Your API service latency is increasing. Users are complaining. Here's how the agent handles it.

Scenario: API service latency increasing

Agent components:


```
1  # Role
2  role = "DevOps Log Analyst monitoring system health"
3
4  # Tasks
5  current_task = "Investigate API latency increase"
6
7  # Available tools
8  tools = {
9      "query_logs": elasticsearch_query,
10     "get_metrics": prometheus_query,
11     "check_deployments": deployment_api,
12     "search_history": incident_database
13 }
14
15 # Memory
16 memory = {
17     "short_term": {
18         "logs_checked": [],
19         "metrics_retrieved": [],
20         "hypotheses": []
21     },
22     "long_term": {
23         "similar_incidents": [
24             "2024-09-15: DB connection leak",
25             "2024-08-03: External API timeout"
26         ]
27     }
28 }
29
30 # Guardrails
31 guardrails = {
32     "max_api_calls": 50,
33     "alert_cooldown": 300, # seconds
34     "confidence_threshold": 0.7
35 }
```

The agent has everything it needs. Now it starts working through the problem using the ReAct pattern.

ReAct execution:

```
1 Thought 1: "API latency is increasing. Let me check recent logs."
2 Action 1: query_logs(service="api", time="last 30 min")
3 Observation 1: "Seeing increased wait times for database queries"
4
5 Thought 2: "Database might be the bottleneck. Check DB metrics."
6 Action 2: get_metrics(service="database", metric="query_time")
7 Observation 2: "Average query time up 300% in last hour"
8
9 Thought 3: "Queries are slow. Any recent changes?"
10 Action 3: check_deployments(service="database")
11 Observation 3: "No recent database changes"
12
13 Thought 4: "Not a deploy. Check if query patterns changed."
14 Action 4: query_logs(service="api", search="SELECT queries")
15 Observation 4: "New query without index causing full table scans"
16
17 Thought 5: "Found it. Check if we've seen this before."
18 Action 5: search_history(keywords="query performance index")
19 Observation 5: "Similar incident on Aug 3: missing index"
20
21 Conclusion: "API latency due to unindexed query. Same as Aug 3 incident."
```

The whole investigation took about 2 minutes. A human doing the same thing manually would take 20-30 minutes.

What You've Learned

You now understand where AI agents fit compared to scripts and traditional logging tools. Scripts only find what you program them to find. Traditional tools need you to do the analysis. AI agents add intelligence on top of your existing infrastructure.

You know the six components every agent needs. Role defines what it is. Tasks define what it does. Tools let it act. Memory gives it context. Guardrails keep it safe. And cooperation lets multiple agents work together if you need that later.

You learned five design patterns. ReAct is the recommended starting point because it's simple, flexible, and well-supported. You can add the others later if you need them.

And you saw a complete example of how an agent would investigate a real production issue—from detecting high latency to identifying an unindexed query as the root cause in about 2 minutes.

What's Next

In Chapter 3, we get into the technical details. How large language models work. How to write effective prompts. How to pull logs from Elasticsearch and CloudWatch. Which AI frameworks to use. And how much this all costs.

Then in Chapter 4, we'll set up your development environment and start building.

You understand the concepts. Time to get technical.

Chapter 3: Understanding Core AI Building Blocks

In the last chapter, we talked about what makes AI agents different from scripts and traditional tools. Now it's time to understand the actual pieces you'll work with when building one.

Think of this like learning to cook. Before you make a meal, you need to know what ingredients are available and what each one does. You wouldn't throw flour, sugar, and chicken into a pot and hope for the best. Similarly, before we start coding, you need to understand the building blocks of AI agents.

The good news? There are only a handful of core concepts you need to grasp. Once you understand these, building an AI agent becomes straightforward.

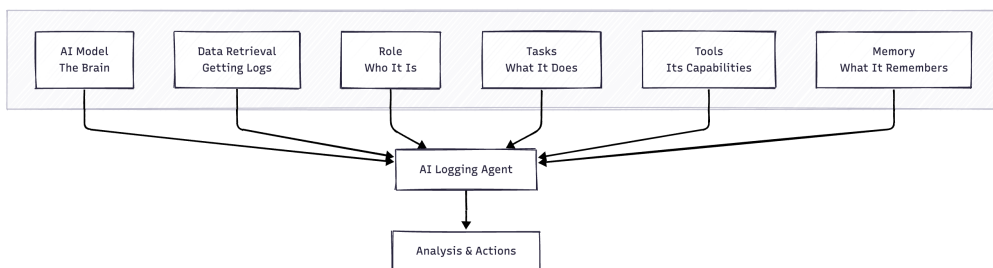


Figure 3.1: Core Building Blocks - Seven key components that make up an AI logging agent.

The AI Model: Your Agent's Brain

At the heart of every AI agent is a language model. This is the part that reads text, understands it, and generates responses.

You've probably used ChatGPT or similar tools. Those interfaces use the same kind of models you'll use in your agent. The difference is that instead of chatting with humans, your model will be analyzing logs and making decisions about your infrastructure.

How Models Work (The Simple Version)

You don't need to understand neural networks or training algorithms to build an AI agent. What you do need to know is this: a language model is a piece of software that predicts what text should come next based on what it's seen before.

When you give it a log file and ask "What's wrong here?", the model:

1. Reads the log entries
2. Recognizes patterns it learned during training
3. Generates a response based on what makes sense

The basic interaction pattern is straightforward: you send the model a system message (defining its role), a user message (with the logs), and it responds with its analysis. Most AI APIs follow this same conversation pattern, whether you're using OpenAI, Anthropic, or other providers.

The model sees patterns like repeated connection failures and retries, then explains what's happening in plain language. That's all there is to it at the basic level.

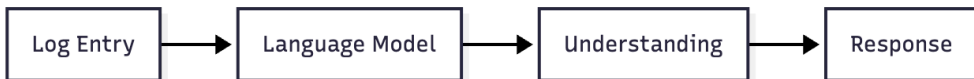


Figure 3.2: Basic Model Processing - Input goes in, understanding happens, output comes out.

Choosing Your Model

You have two main options when building an AI agent:

Cloud APIs (OpenAI, Anthropic, Google)

These services host the models for you. You send your data over the internet, and they send back the results. This is the easiest way to start because you don't need powerful hardware.

The tradeoff is cost and data privacy. Every API call costs money, and your logs leave your infrastructure. For many companies, especially startups and

small teams, this is totally fine. For others with strict security requirements, it's a non-starter.

Local Models (Llama, Mistral, others)

You can also run models on your own servers. This keeps your data private and eliminates per-request costs. The downside is you need GPUs or powerful CPUs, and the setup is more complex.

For this series, we'll use Gemini's API because it's the simplest way to learn. Once you understand the concepts, switching to a local model is straightforward—the code structure stays the same.

Data Retrieval: Getting Logs into Your Agent

Your AI model can't analyze logs it doesn't have. This sounds obvious, but data retrieval is where many projects get stuck. You need a reliable way to pull logs from wherever they live and feed them to your agent.

Where Your Logs Live

In a typical DevOps environment, logs come from multiple sources:

- Application logs in Elasticsearch
- Container logs from Kubernetes
- System logs in CloudWatch
- Database logs in RDS
- Load balancer logs in S3

Your AI agent needs to connect to these sources, retrieve the relevant logs, and organize them in a way the model can process.

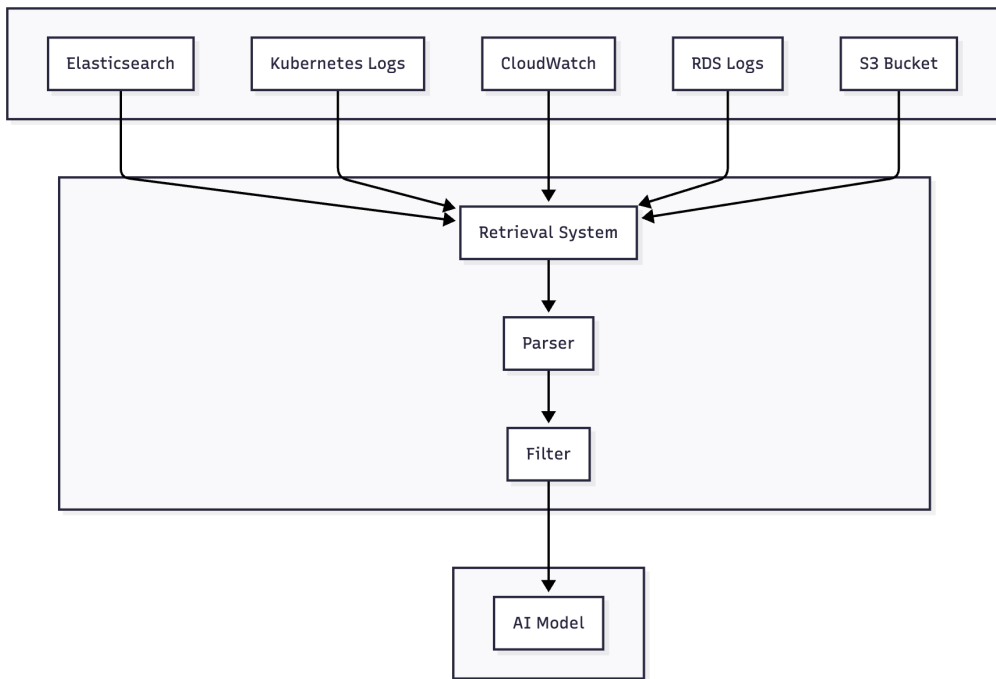


Figure 3.3: Data Retrieval Architecture - Logs flow from multiple sources through a retrieval layer to the AI model.

Building Your Retrieval Layer

The pattern for retrieving logs is similar across all data sources:

1. **Connect** to your log source (Elasticsearch, CloudWatch, etc.)
2. **Define a time range** (last hour, last 5 minutes, etc.)
3. **Filter by criteria** (error level, service name, keywords)
4. **Limit the results** to a manageable number
5. **Extract and format** the log messages

For Elasticsearch, you'd use their Python client to search for logs matching your criteria. For CloudWatch, you'd use boto3 to query log groups. For Kubernetes, you'd use the Kubernetes Python client to fetch pod logs.

The key is building a consistent interface that your agent can use regardless of where the logs come from. You want your agent to call something like `retrieve_logs(source, time_range, filters)` without caring about the underlying implementation.

Dealing with Volume

One challenge with log retrieval is volume. Your production systems might generate thousands of logs per minute. You can't send all of them to an AI model—it would be slow and expensive.

Instead, you need to filter intelligently:

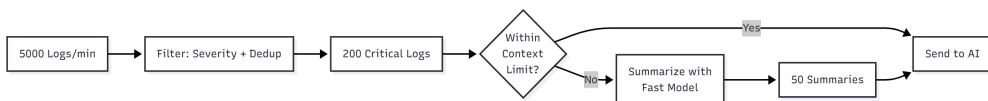
- Only retrieve logs above a certain severity level (ERROR, WARN)
- Focus on specific time windows (like the last 5 minutes)
- Pull logs from services that are having problems
- Sample logs when volume is high
- Aggregate similar logs together

Your retrieval function should accept parameters for time window and maximum number of logs. Start narrow (5 minutes, 50 logs) and widen the search if you need more context. This keeps your API costs down and your response times fast.

Handling Volume Spikes

When incidents happen, log volume can spike from hundreds to thousands of entries per minute. Your agent needs a strategy to handle this without breaking the bank or hitting context limits.

The key is a two-layer approach: **filter first, summarize if needed**.



Layer 1: Aggressive Pre-filtering

Before any logs reach the AI model, apply intelligent filtering. Think of this as a bouncer at a club—only the most important entries get through.

Priority-based selection means critical errors (FATAL, ERROR) always get analyzed first. Warnings and info messages are included only if there's room.

This ensures you never miss the serious problems even when logs are flooding in.

Deduplication is where you catch identical or nearly identical log entries. If you have the same “Connection timeout” error repeated 100 times, you don’t need to send all 100. Instead, send it once with metadata:

- How many times it occurred
- First and last occurrence timestamps
- Which services were affected

This gives the AI the pattern without overwhelming the context window. The model can understand “this happened 127 times over 5 minutes” just as well as seeing 127 individual entries—actually better, because it gets the pattern immediately.

Service-aware filtering means you prioritize logs from services that are actually experiencing issues. If your payment service has a 50% error rate but your auth service is fine, focus on payment logs first. You determine which services need attention based on error rates, then pull their logs preferentially.

Layer 2: Progressive Summarization

When filtering alone isn’t enough, use a two-pass approach with an interesting twist: **use a cheaper, faster model for the first pass.**

Pass 1 - Quick Summarization takes chunks of filtered logs and creates one-line summaries. The key insight is that you don’t need your most expensive model for this. A fast, cheap model can create summaries like:

```
1 [ERROR] DB timeout (47 times, 10:15-10:18) - payment-db unreachable
2 [WARN] High memory (3 times, 10:16-10:17) - api-gateway at 87% usage
3 [ERROR] API 500s (23 times, 10:17-10:19) - downstream service failures
```

Each summary captures what happened, how often, when, and a brief description. This reduces 200 log lines to perhaps 20 summaries.

Pass 2 - Detailed Analysis then sends these summaries (not the raw logs) to your main, more capable model for root cause analysis. The expensive model now sees clean, structured information instead of raw log dumps.

This two-pass approach keeps you within context limits while preserving the critical information. And because the first pass uses a cheaper model, your costs don't explode during incidents.

The goal is to maintain analysis quality regardless of log volume, while keeping costs predictable and response times fast. Your agent should be just as effective during a major outage as during normal operations—it just uses different strategies to get there.

What Your Agent Needs to Know

Now we get to the pieces that make an agent actually useful. A language model by itself is just a tool—it can analyze whatever you give it. An agent, on the other hand, has a specific purpose and capabilities.

This is where you define what your agent does and how it does it.

Role: Who Is Your Agent?

The role defines your agent's identity and expertise. It's like giving someone a job title and job description.

For a logging agent, you might define roles like:

- “You are a DevOps engineer specializing in application error analysis”
- “You are a database reliability expert who investigates performance issues”
- “You are a security analyst monitoring for suspicious activity in logs”

The role matters because it influences how the model interprets information. A security-focused agent will look for different patterns than a performance-focused agent, even when analyzing the same logs.

When you initialize your agent, you'll set this role as the system message in your conversation with the AI model. The role becomes the lens through which all log analysis happens.

You might create different agents for different purposes:

- An error analysis agent that focuses on application failures

- A performance agent that looks for slow queries and bottlenecks
- A security agent that monitors for suspicious patterns
- A cost optimization agent that identifies wasteful resource usage

Same underlying model, but the role makes each one specialized for its task.

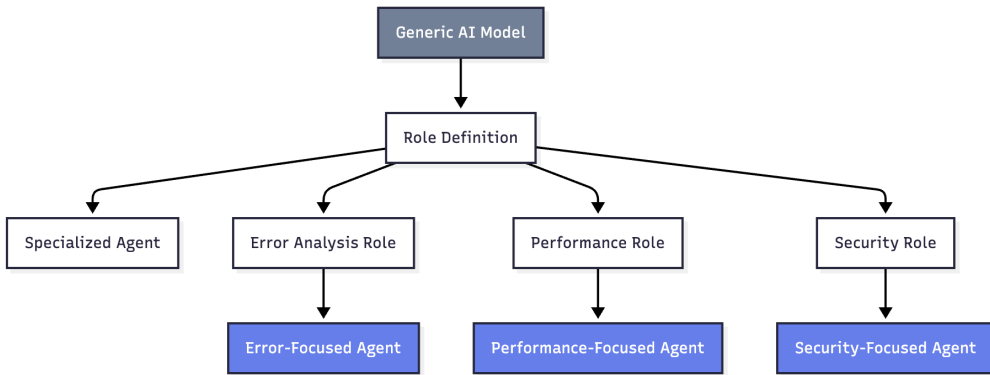


Figure 3.4: Role Specialization - A generic model becomes a specialized agent through role definition.

Tasks: What Should Your Agent Do?

Tasks are the specific actions your agent performs. While the role defines who the agent is, tasks define what it does.

For a logging agent, common tasks include:

1. Analyze recent error logs
2. Identify patterns across multiple services
3. Correlate errors with deployment events
4. Suggest likely root causes
5. Recommend remediation steps

You can think of tasks as a checklist the agent works through. Each task focuses the agent's attention on a specific aspect of the logs.

When you build your agent, you'll define each task with:

- A name (for identification)

- A description (what the task should accomplish)
- A priority (which order to execute them)

The agent executes tasks in sequence, with each task building on the results of previous ones. The output from “error detection” feeds into “pattern analysis,” which feeds into “correlation,” and so on.



Figure 3.5: Task Pipeline – Tasks execute in sequence, each building on the previous results.

Tools: How Your Agent Takes Action

Tools are the capabilities you give your agent. They’re functions the agent can call to do things beyond just analyzing text.

For a logging agent, useful tools might include:

- Query Elasticsearch for specific log patterns
- Check service health endpoints
- Look up recent deployments
- Create Jira tickets
- Send Slack notifications
- Restart services (with appropriate safeguards)

Each tool is a function your agent can call. You define what the tool does and what parameters it needs. The AI model then decides when to use each tool based on what it discovers in the logs.

For example, if the agent detects database connection errors, it might:

1. Use the `search_logs` tool to find more database-related logs
2. Use the `check_service_health` tool to verify the database is actually down
3. Use the `notify_team` tool to alert the on-call engineer

The tools give your agent hands and feet—they let it do more than just think and analyze.

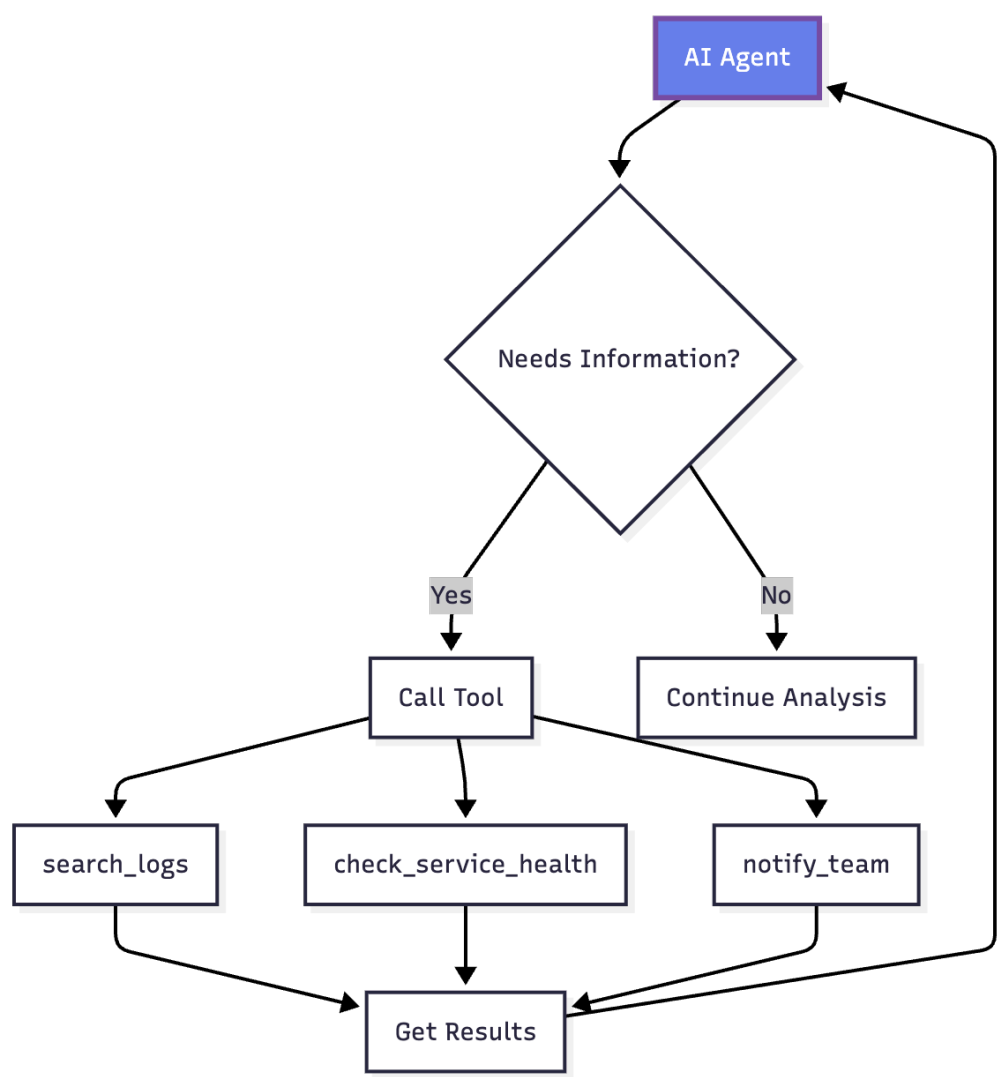


Figure 3.6: Tool Usage Pattern - The agent calls tools when it needs to take action or gather more information.

Putting It Together: Building Your Logging Agent

Now you understand the core building blocks. Let's see how they work together in a complete agent.

The Agent Structure

A logging agent typically has this structure:

1. **Initialization:** Set up the role, tasks, and tools
2. **Log Retrieval:** Pull the relevant logs from your data sources
3. **Context Building:** Create the conversation with the AI model, starting with the role
4. **Task Execution:** Run through each task in sequence
5. **Result Compilation:** Gather all findings and recommendations

Each task sees the results of previous tasks, allowing the agent to build a complete picture. The first task might identify errors, the second task finds patterns in those errors, the third task correlates them with other events, and so on.

Your agent class will maintain:

- The role (who it is)
- The tasks (what it does)
- The tools (its capabilities)
- The conversation history (context from the current analysis)
- The AI client (connection to the model)

When you call the agent's main analysis method, it orchestrates all these pieces to produce a comprehensive log analysis.

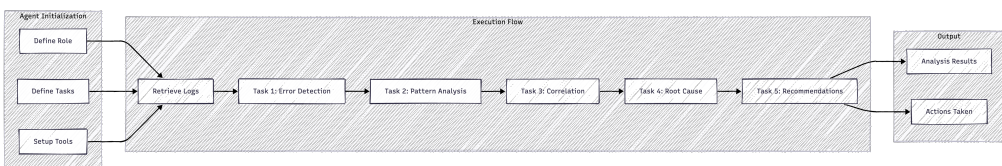


Figure 3.7: Complete Agent Flow - From initialization through execution to output.

Memory: Remembering Past Analysis

One feature that makes AI agents powerful is memory. Your agent can remember past analyses and use that knowledge to improve future ones.

For a logging agent, memory helps with:

- Recognizing recurring issues
- Tracking whether problems were fixed
- Learning which solutions worked
- Building a knowledge base of your infrastructure

There are different types of memory you can implement:

Short-term Memory

This is the conversation history within a single analysis session. Each task sees the results of previous tasks, allowing the agent to build on its own analysis. This is typically just a list of messages that you maintain during the conversation with the AI model.

Long-term Memory

This persists across analysis sessions. The agent stores important findings in a database or file and can recall them later. You might store:

- Past incidents and their resolutions
- Recurring error patterns
- Service-specific quirks or known issues
- Solutions that worked in the past

The implementation is straightforward: save key findings to a JSON file or database with timestamps and metadata. When analyzing new logs, check if similar issues exist in memory first.

With memory, your agent gets smarter over time. When it sees an error pattern, it can check if the same pattern appeared before and how it was resolved.

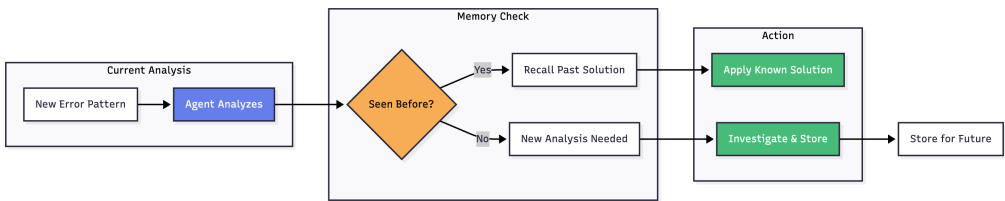


Figure 3.8: Memory Usage - The agent checks memory before analyzing new issues.

Applying the ReAct Pattern to Your Logging Agent

In Chapter 2, we explored five design patterns and chose ReAct as the recommended approach for our logging agent. Now it's time to understand how to actually implement it.

ReAct (Reasoning + Acting) is a loop where your agent alternates between thinking and doing. It thinks about what it needs to know, acts by using a tool, observes what came back, thinks about what that means, and continues until it solves the problem.

This pattern works particularly well for log analysis because debugging is inherently iterative. You don't usually know the root cause immediately—you form hypotheses, gather evidence, and refine your understanding.

The ReAct Loop Structure

The ReAct pattern has three phases that repeat:

1. **Thought:** The agent reasons about the current situation
2. **Action:** The agent uses a tool to gather more information
3. **Observation:** The agent processes the results

This continues until the agent reaches a conclusion or hits a maximum iteration limit.

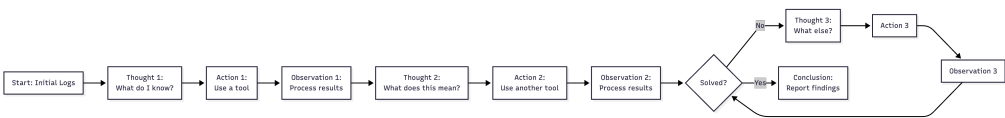


Figure 3.9: ReAct Loop - Continuous cycle of thinking, acting, and observing.

Building the Thought Phase

The thought phase is where your agent analyzes what it knows and decides what to do next. You construct a prompt that includes:

- The current situation (logs, errors, metrics seen so far)
- Previous thoughts and actions from earlier iterations
- Available tools the agent can use
- A request for the agent to think step-by-step

The key is structuring the prompt so the agent explicitly states its reasoning. You want to see: “Based on X, I believe Y. To confirm, I should Z.”

Your prompt template might look like this:

```
1 Current Situation:
2 [Log entries or data from previous observations]
3
4 Previous Actions:
5 [What the agent has done so far]
6
7 Available Tools:
8 - query_logs(service, time_range, level)
9 - get_metrics(service, metric_name, time_range)
10 - check_deployments(service, hours_back)
11 - search_incidents(keywords)
12
13 Think step-by-step:
14 1. What do we know for certain?
15 2. What are possible explanations?
16 3. What information would help confirm or rule out each explanation?
17 4. Which tool should we use next?
18
19 Provide your reasoning, then specify your next action.
```

The model responds with its thought process and indicates which action to take.

Implementing the Action Phase

Once the agent decides on an action, you need to execute it. The action is typically a tool call—a function that gathers more information.

Your implementation needs to:

1. Parse the agent's response to extract the requested action
2. Validate that the action is allowed (using guardrails)
3. Execute the corresponding tool function
4. Capture the results

For example, if the agent says “I need to check recent deployments for the payment service,” you would:

1. Parse this into: `check_deployments(service="payment", hours_back=24)`
2. Validate the service name exists and the time range is reasonable
3. Call your deployment API to get the data
4. Format the results for the next observation phase

The key is making tools deterministic and reliable. If a tool call fails, handle it gracefully and pass that information back to the agent so it can try something else.

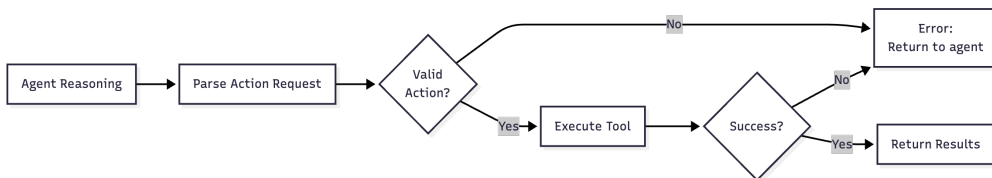


Figure 3.10: Action Execution Flow - Parse, validate, execute, and return results.

Processing the Observation Phase

The observation phase takes the results from the action and adds them to the context for the next thought phase. This is where you maintain the conversation history.

You append three things to your conversation:

1. The agent's thought and chosen action
2. A system message indicating the action was executed
3. The actual results from the tool

This gives the agent full visibility into what it's learned so far. Each iteration builds on previous ones, allowing the agent to form increasingly accurate hypotheses.

The format might look like:

```
1 Thought: "I need to check if there was a recent deployment"
2 Action: check_deployments(service="payment", hours_back=24)
3 Observation:
4   - Deploy at 14:23 UTC changed configuration
5   - Deploy increased timeout from 30s to 60s
6   - Errors started at 14:25 UTC
```

Now the agent has new information to reason about in the next thought phase.

Setting Loop Boundaries

The ReAct loop needs boundaries to prevent infinite running:

Maximum Iterations: Set a limit (typically 5-10 iterations) to prevent the agent from looping forever. If it reaches this limit without a conclusion, it should return its best guess with a note that more investigation is needed.

Conclusion Detection: Watch for the agent to explicitly state it has reached a conclusion. You might look for keywords like "CONCLUSION:" or "FINAL ANSWER:" in the response.

Cost Limits: Since each iteration calls the AI model, set a budget. If you're spending too much on a single analysis, stop and escalate to a human.

Time Limits: Set a wall-clock time limit (e.g., 2 minutes). Even with iteration limits, if the agent is making very slow API calls, you want to timeout.

Handling Edge Cases

Real-world log analysis hits edge cases. Your ReAct implementation needs to handle them:

No Data Available: What if a tool returns empty results? The agent needs to recognize this and try a different approach rather than getting stuck.

Contradictory Information: Sometimes different data sources contradict each other. The agent should recognize this and explicitly state the contradiction.

Tool Failures: If a tool call fails (API timeout, permission denied, etc.), pass that information to the agent. It can try a different tool or work with incomplete information.

Circular Reasoning: Monitor for the agent repeatedly trying the same action. If it uses the same tool with the same parameters twice in a row, intervene and suggest a different approach.

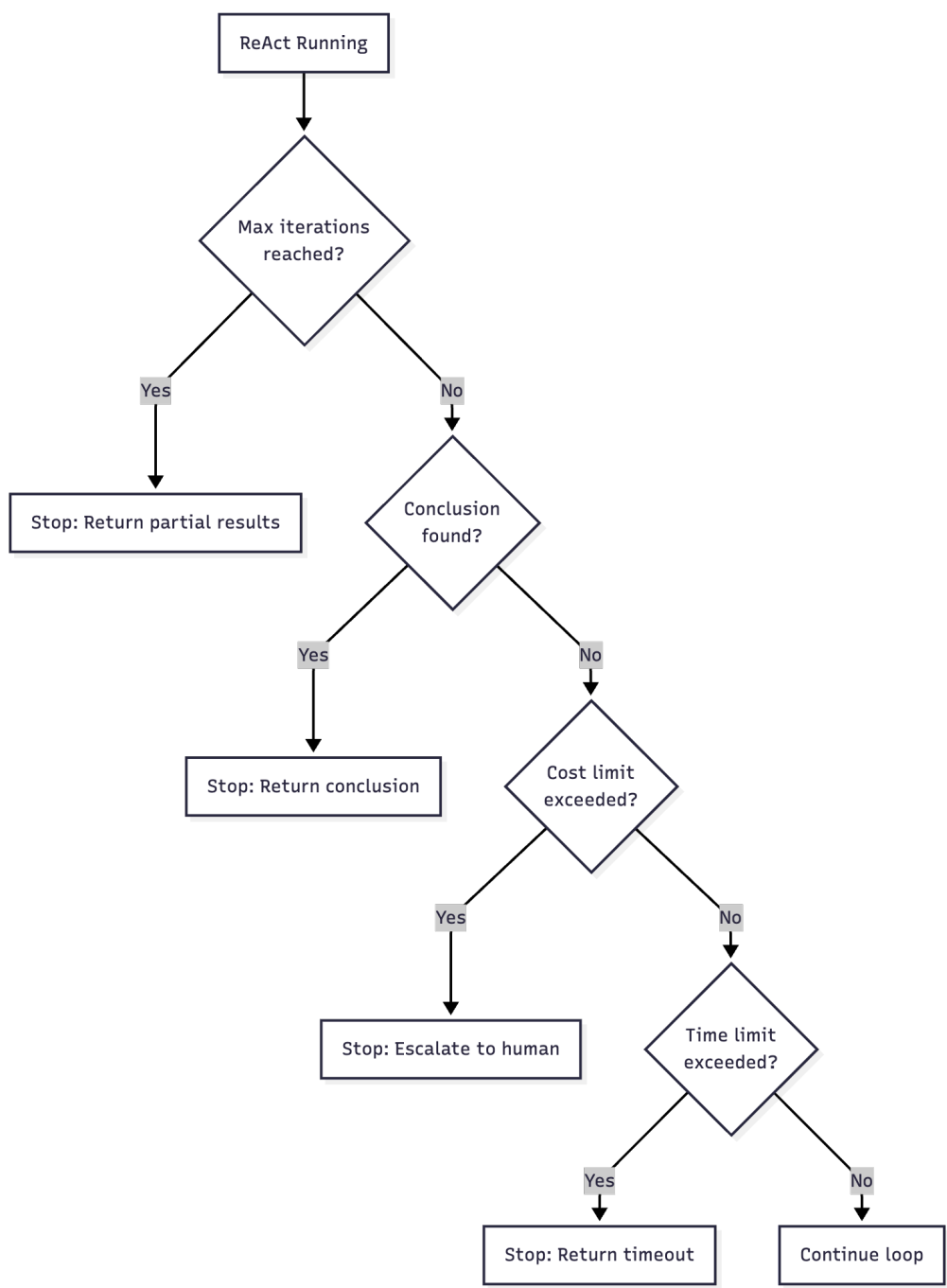


Figure 3.11: Loop Boundaries - Multiple exit conditions ensure the agent stops appropriately.

Example: Complete ReAct Flow

Let's walk through a concrete example to see how all the pieces work together.

Scenario: Error rate spike in API service

Initial Input:

```
1 Recent logs show:
2 - 200 errors in the last 5 minutes
3 - Error type: "Connection timeout"
4 - Affected service: api-gateway
```

Iteration 1:

Thought: "Connection timeout errors in api-gateway. This could be network issues, backend service problems, or resource exhaustion. I should first check if backend services are healthy."

Action: `check_service_health(service="payment-service")`

Observation: "Payment service health check failing. Response time: 30000ms (timeout)."

Iteration 2:

Thought: "Payment service is timing out. This could explain the api-gateway errors. I need to check if this is a recent change or a resource problem."

Action: `get_metrics(service="payment-service", metric="memory_usage", time_range="1h")`

Observation: "Memory usage: Started at 40%, now at 98%. Steady increase over the last 45 minutes."

Iteration 3:

Thought: "Memory leak pattern. Rising memory usage correlating with service timeouts. I should check if there was a recent deployment that might have introduced this."

Action: `check_deployments(service="payment-service", hours_back=4)`

Observation: "Deploy 50 minutes ago: version 2.4.1. Changes include new caching layer implementation."

Iteration 4:

Thought: “Deploy timing matches the memory leak start. New caching layer likely has a memory leak. Let me verify if there are similar past incidents.”

Action: `search_incidents(keywords="payment service memory leak cache")`

Observation: “No similar incidents found in the past 90 days.”

Conclusion:

“Root cause: Memory leak in payment-service version 2.4.1 introduced 50 minutes ago. The new caching layer is not releasing memory properly, causing the service to slow down and timeout requests. This cascades to api-gateway connection timeout errors.

Recommendation: Rollback payment-service to version 2.4.0 immediately. After rollback, investigate caching implementation for memory leak before redeploying.“

Why ReAct Works for Log Analysis

The ReAct pattern is particularly well-suited for log analysis because:

It matches how humans debug: We don’t know the answer upfront. We gather evidence, form hypotheses, and test them iteratively.

It’s transparent: You can see exactly what the agent is thinking and why it chose each action. This builds trust and makes debugging easier.

It’s flexible: The agent isn’t locked into a rigid plan. If it discovers something unexpected, it can change direction.

It handles uncertainty: When information is incomplete or contradictory, the agent can seek additional data rather than making assumptions.

It’s cost-effective: Compared to multi-agent systems, ReAct uses a single agent with a simple loop, keeping complexity and costs down.

As you build your logging agent in the next chapters, you’ll implement this ReAct pattern step by step. You’ll see how to structure the prompts, execute the tools, maintain the conversation history, and handle edge cases. The pattern becomes second nature once you build it a few times.

What You've Learned

This chapter covered the fundamental building blocks of AI agents. You now understand that an AI agent is built from several key components working together.

The AI model is the brain that processes information and generates responses. You learned that you can use cloud APIs like OpenAI or run models locally, depending on your needs. The model itself is just a tool—what matters is how you structure the work around it.

Data retrieval is how your agent gets the logs it needs to analyze. You saw practical examples of connecting to Elasticsearch and pulling relevant logs while managing volume. The key is filtering intelligently so you're only analyzing what matters.

The essential elements give your agent structure and purpose. The role defines who your agent is, tasks define what it does, and tools give it the ability to take action. You learned how to define each of these in code and how they work together.

Memory allows your agent to learn from past analyses. Short-term memory maintains context within a session, while long-term memory lets the agent remember patterns and solutions across sessions. This makes your agent smarter over time.

Finally, you learned how to apply the ReAct pattern to your logging agent. You understand the three-phase loop (Thought, Action, Observation), how to structure prompts for the thought phase, how to execute and validate actions, how to process observations, and how to set appropriate loop boundaries. You also saw a complete example of ReAct in action, debugging an error spike through four iterations to reach a conclusion with actionable recommendations.

What's Next

You have the concepts. Now it's time to set up your development environment and start building. In the next chapter, you'll install the necessary tools, set up your API keys, and write your first working agent code. You'll see all these concepts come together in actual running software.

By the end of Chapter 4, you'll have a development environment ready to go and you'll have run your first AI agent analysis on real logs. That's when things get interesting.

Chapter 4: Setting Up Your Development Environment

You've learned what AI agents are, how they're different from traditional tools, and what components you need. Now it's time to actually build something.

Before you can write your first AI agent, you need a development environment. This means installing Python, setting up the right libraries, getting API keys, and making sure everything works together.

Don't worry if you're not a Python expert. I'll walk you through every step. By the end of this chapter, you'll have everything installed and you'll run your first piece of code that talks to an AI model.

What You'll Need

Here's the complete list of what we're setting up:

1. **Python 3.9 or higher** - The programming language we'll use
2. **pip** - Python's package installer (comes with Python)
3. **A code editor** - VS Code is a good choice
4. **Git** - For version control (optional but recommended)
5. **Google AI Studio account** - To get your Gemini API key
6. **Python libraries** - We'll install these with pip

Let's go through each one.

Installing Python

First, you need Python installed on your machine. Most modern operating systems come with Python, but it might be an older version. You want Python 3.9 or newer.

Check If You Have Python

Open your terminal and type:

```
1 python3 --version
```

If you see something like “Python 3.11.5” or any version 3.9 or higher, you’re good. Skip to the next section.

If you get an error or see a version older than 3.9, you need to install Python.

Installing Python on Mac

The easiest way is using Homebrew. If you don’t have Homebrew, install it first:

```
1 /bin/bash -c "$(curl -fsSL
  ↪ https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Then install Python:

```
1 brew install python3
```

Installing Python on Linux

On Ubuntu or Debian:

```
1 sudo apt update
2 sudo apt install python3 python3-pip
```

On Fedora or CentOS:

```
1 sudo dnf install python3 python3-pip
```

Setting Up a Project Directory

Create a folder for your AI logging agent project. This keeps everything organized.

```
1 # Create project directory
2 mkdir ai-logging-agent
3 cd ai-logging-agent
4
5 # Create a directory for your code
6 mkdir src
7
8 # Create a directory for logs (we'll use this later)
9 mkdir logs
```

Your structure should look like this:

```
1 ai-logging-agent/
2 |— src/
3 |— logs/
```

Installing Python Libraries

Now you need to install the libraries your agent will use. We'll use a virtual environment to keep things clean.

Create a Virtual Environment

A virtual environment is like a sandbox for your Python project. It keeps your project's libraries separate from your system's Python installation.

```
1 ` `` bash
2 python3 -m venv venv
3
4 # Activate it
5 source venv/bin/activate
```

When activated, your terminal prompt will show `(venv)` at the beginning. This means you're in the virtual environment.

Install Required Libraries

Create a file called `requirements.txt` in your project root:

```
1 touch requirements.txt
```

Add these libraries to the file:

```
1 google-generativeai>=0.8.5
2 python-dotenv>=1.1.1
3 requests>=2.32.5
```

Now install them:

```
1 pip install -r requirements.txt
```

Here's what each library does:

- **google-generativeai**: Official Gemini API client
- **python-dotenv**: Loads environment variables from a `.env` file
- **requests**: Makes HTTP requests (we'll use this to fetch logs)

Getting Your Gemini API Key

Your AI agent needs to talk to Gemini's API. For that, you need an API key.

Step 1: Go to Google AI Studio

Open your browser and go to <https://aistudio.google.com/app/api-keys>

Sign in with your Google account.

Step 2: Create an API Key

Click "Get API Key" or "Create API Key in new project".

Google will generate a key that looks like this:

```
1 AIzaSyA1B2C3D4E5F6G7H8I9J0K1L2M3N4O5P6Q
```

Important: Keep this key secret. Don't share it or commit it to Git.

Step 3: Store Your API Key Safely

Create a `.env` file in your project root:

```
1 touch .env
```

Add your API key to this file:

```
1 GEMINI_API_KEY=your-actual-api-key-here
```

Replace `your-actual-api-key-here` with the key you just got.

Step 4: Add `.env` to `.gitignore`

If you're using Git, make sure your `.env` file doesn't get committed.

Create a `.gitignore` file if you don't have one:

Mac/Linux:

```
1 cat > .gitignore << EOF
2 .env
3 .venv/
4 __pycache__/
5 *.pyc
6 EOF
```

This prevents accidentally sharing your API key.

Your First AI Agent Code

Let's write a simple script to test that everything works. This will be your "Hello World" for AI agents.

Create a file called `src/test_setup.py`:

```

1  import os
2  from dotenv import load_dotenv
3  import google.generativeai as genai
4
5  # Load environment variables
6  load_dotenv()
7
8  # Configure Gemini
9  api_key = os.getenv('GEMINI_API_KEY')
10 if not api_key:
11     print("Error: GEMINI_API_KEY not found in .env file")
12     exit(1)
13
14 genai.configure(api_key=api_key)
15
16 # Create a model instance
17 model = genai.GenerativeModel('gemini-2.5-flash-lite')
18
19 # Test with a simple log analysis
20 sample_log = """
21 2024-10-21 14:23:45 ERROR Database connection failed
22 2024-10-21 14:23:46 WARN Retry attempt 1 of 3
23 2024-10-21 14:23:48 ERROR Database connection failed
24 2024-10-21 14:23:49 WARN Retry attempt 2 of 3
25 2024-10-21 14:23:51 ERROR Database connection failed
26 2024-10-21 14:23:52 ERROR Maximum retries reached
27 """
28
29 # Ask the AI to analyze it
30 prompt = f"""You are a DevOps engineer analyzing application logs.
31 Analyze this log and explain what's happening:
32
33 {sample_log}
34
35 Provide a brief analysis and suggest what might be wrong."""
36
37 print("Analyzing logs with AI...")
38 print("-" * 50)
39
40 response = model.generate_content(prompt)
41 print(response.text)
42
43 print("-" * 50)
44 print("✓ Setup successful! Your environment is ready.")

```

This script does four things:

1. Loads your API key from the .env file
2. Connects to Gemini

3. Sends a sample log for analysis
4. Prints the AI's response

Run Your Test Script

Make sure you're in your project root directory and your virtual environment is activated, then run:

```
1 python src/test_setup.py
```

If everything is set up correctly, you'll see output like:

```
1 Analyzing logs with AI...
2 -----
3 The logs show a database connection failure with
4 multiple retry attempts. The application tried to
5 connect three times but failed each time, eventually
6 reaching the maximum retry limit.
7
8 Possible causes:
9 1. Database server is down or unreachable
10 2. Network issues between app and database
11 3. Invalid connection credentials
12 4. Connection pool exhausted
13
14 Recommended actions:
15 - Check if the database service is running
16 - Verify network connectivity
17 - Review database logs for additional clues
18 - Check connection string configuration
19 -----
20 ✓ Setup successful! Your environment is ready.
```

The exact response will vary because AI models generate slightly different text each time. But you should see a coherent analysis of the log file.

Common Issues and Solutions

Here are the problems you're most likely to run into, and how to fix them.

“No module named ‘google.generativeai’ ”

Problem: The library isn’t installed.

Solution: Make sure your virtual environment is activated and run:

```
1 pip install -r requirements.txt
```

“GEMINI_API_KEY not found in .env file”

Problem: Your .env file is missing or the API key isn’t set correctly.

Solution:

1. Check that the .env file exists in your project root
2. Open it and verify it has: GEMINI_API_KEY=your-key-here
3. Make sure there are no spaces around the =
4. Make sure you replaced your-key-here with your actual API key

“API key not valid”

Problem: Your API key is incorrect or expired.

Solution:

1. Go back to Google AI Studio
2. Generate a new API key
3. Update your .env file with the new key

“Rate limit exceeded”

Problem: You’re making too many API calls.

Solution:

- Wait a few minutes and try again
- Free tier has limits on requests per minute
- For production use, consider upgrading to a paid plan

Understanding Your Project Structure

Let's look at what you have now:

```
1 ai-logging-agent/
2 |— .env                # Your API key (never commit this)
3 |— .gitignore          # Tells Git what to ignore
4 |— requirements.txt    # Python dependencies
5 |— venv/               # Virtual environment (never commit this)
6 |— logs/               # Sample logs will go here
7 |— src/
8   |— test_setup.py     # Your first test script
```

This structure will grow as we add more components in the next chapters. You'll add:

- A main agent file
- Tool implementations
- Memory management
- Log retrieval functions
- Configuration files

But for now, this is a good starting point.

Best Practices for Development

Now that you have everything set up, here are some tips to keep your development smooth:

Use Virtual Environments Always

Every time you start working on this project, activate your virtual environment:

```
1 source venv/bin/activate
```

When you're done, deactivate it:

```
1 deactivate
```

This keeps your project's dependencies isolated from other Python projects.

Keep Your Dependencies Updated

Periodically update your libraries:

```
1 pip install --upgrade -r requirements.txt
```

But test after updating to make sure nothing breaks.

Use Version Control

If you're not using Git yet, now is a good time to start:

```
1 git init
2 git add .
3 git commit -m "Initial setup"
```

Commit your changes regularly. This gives you a history to roll back to if something breaks.

Keep API Keys Secret

Never, ever commit your `.env` file or API keys to Git. If you accidentally do:

1. Immediately revoke the API key in Google AI Studio
2. Generate a new one
3. Update your `.env` file
4. Remove the key from Git history

Monitor Your API Usage

Gemini's free tier has limits. Keep an eye on your usage in Google AI Studio so you don't get surprised by rate limits or costs.

What You Can Do Now

With your environment set up, you can start experimenting. Try modifying the test script:

Change the log content:

```
1 sample_log = """
2 2024-10-21 15:30:12 INFO User login successful: user_id=12345
3 2024-10-21 15:30:15 WARN High memory usage: 85%
4 2024-10-21 15:30:20 ERROR Out of memory exception
5 2024-10-21 15:30:21 ERROR Application crashed
6 """
```

Change the prompt:

```
1 prompt = f"""You are a security analyst.
2 Check these logs for any security concerns:
3
4 {sample_log}
5
6 Report any suspicious activity."""
```

Try different models:

```
1 # Try other Gemini models:
2 model = genai.GenerativeModel('gemini-2.5-pro') # More capable, faster
```

Play around. Break things. That's how you learn what works and what doesn't.

Creating Sample Log Files

Let's create some sample log files you can use for testing. Create a file called `logs/sample_app.log`:

```

1  2024-10-21 10:15:23 INFO Application started successfully
2  2024-10-21 10:15:24 INFO Database connection pool initialized: size=10
3  2024-10-21 10:15:25 INFO HTTP server listening on port 8080
4  2024-10-21 10:17:42 INFO Processing request: GET /api/users
5  2024-10-21 10:17:43 INFO Request completed: 200 OK (125ms)
6  2024-10-21 10:18:15 WARN Slow query detected: SELECT * FROM orders (850ms)
7  2024-10-21 10:19:03 ERROR Database connection timeout
8  2024-10-21 10:19:04 WARN Retry attempt 1 of 3
9  2024-10-21 10:19:07 ERROR Database connection timeout
10 2024-10-21 10:19:08 WARN Retry attempt 2 of 3
11 2024-10-21 10:19:11 ERROR Database connection timeout
12 2024-10-21 10:19:12 ERROR Maximum retries exceeded
13 2024-10-21 10:19:12 ERROR Request failed: 500 Internal Server Error
14 2024-10-21 10:19:45 WARN High memory usage: 87%
15 2024-10-21 10:20:12 ERROR Out of memory exception
16 2024-10-21 10:20:13 FATAL Application shutting down

```

Now create a script to analyze this file. Create `src/analyze_log_file.py`:

```

1  import os
2  from dotenv import load_dotenv
3  import google.generativeai as genai
4
5  # Load environment variables
6  load_dotenv()
7
8  # Configure Gemini
9  genai.configure(api_key=os.getenv('GEMINI_API_KEY'))
10 model = genai.GenerativeModel('gemini-2.5-flash-lite')
11
12 # Read the log file
13 with open('logs/sample_app.log', 'r') as f:
14     logs = f.read()
15
16 # Analyze with AI
17 prompt = f"""You are an expert DevOps engineer analyzing application logs.
18
19 Analyze these logs and provide:
20 1. A summary of what happened
21 2. The root cause of any issues
22 3. Recommended actions to fix the problems
23
24 Logs:
25 {logs}
26
27 Provide a clear, structured analysis."""
28
29 print("Analyzing log file...")
30 print("=" * 60)

```

```
31
32 response = model.generate_content(prompt)
33 print(response.text)
34
35 print("=" * 60)
```

Run it:

```
1 python src/analyze_log_file.py
```

You should see a detailed analysis of the log file, identifying the database connection issues and memory problems.

Next: Understanding Code Structure

Before we move to the next chapter, let's understand the basic code pattern you'll be using:

```
1 # 1. Setup
2 import os
3 from dotenv import load_dotenv
4 import google.generativeai as genai
5
6 # 2. Configuration
7 load_dotenv()
8 genai.configure(api_key=os.getenv('GEMINI_API_KEY'))
9 model = genai.GenerativeModel('gemini-2.5-flash-lite')
10
11 # 3. Prepare data
12 data = "your log content here"
13
14 # 4. Create prompt
15 prompt = f"""Your instructions here
16 Data: {data}"""
17
18 # 5. Get AI response
19 response = model.generate_content(prompt)
20
21 # 6. Use the result
22 print(response.text)
```

This is the core pattern you'll see over and over. You'll add more sophistication (tools, memory, loops), but this is the foundation.

What You've Learned

You now have a complete development environment for building AI agents. You installed Python, set up a virtual environment, installed the necessary libraries, and got your Gemini API key configured.

You wrote and ran your first AI-powered script that analyzes logs. You understand the basic code structure and how to communicate with the Gemini API. You know how to keep your API keys secure and how to organize your project files.

You also learned common pitfalls and how to fix them, and you have sample log files you can use for testing.

Most importantly, you verified that everything works end-to-end. Your environment is ready for real development.

What's Next

Now that your environment is set up, it's time to understand the different levels of AI logging systems. In Chapter 5, you'll learn how AI agents can start simple and grow more sophisticated over time.

We'll map out a progression from basic log parsing to autonomous incident response. This will help you understand where to start and how to incrementally improve your agent.

Then in Chapter 6, we'll start building the actual components. You'll write code for roles, tasks, and tools. You'll implement memory so your agent learns from past incidents. That's when your AI logging agent starts to come alive.

Chapter 5: Levels of AI Logging Systems

You’ve got your development environment set up. You understand the building blocks. Now you’re probably wondering: “What exactly am I building? How advanced does this need to be?”

Here’s the thing about AI agents: you can build them at different levels of capability. A Level 1 agent that parses logs and answers basic questions is useful. A Level 5 agent that autonomously manages your entire logging infrastructure and fixes issues without human intervention is powerful—but also complex and potentially risky.

The key is knowing where you’re headed and why. This chapter breaks down five levels of AI logging systems, from simple to advanced. By the end, you’ll understand what each level can do, what it requires, and where you should focus your efforts.

Think of this like a roadmap. You wouldn’t try to drive from New York to Los Angeles without understanding the route. Similarly, you shouldn’t build an AI agent without understanding the progression from basic to advanced capabilities.



Figure 5.1: The Five Levels - Each level builds on the previous one, adding capability and complexity.

Level 1: Basic Log Parser and Responder

This is where everyone should start. A Level 1 agent does one thing: it reads logs and tells you what it sees.

What It Does

At this level, your agent takes a log file or log stream as input and produces analysis as output. That's it. No fancy integrations, no autonomous actions, no multi-agent coordination. Just log in, analysis out.

Here's what a Level 1 agent can do:

Read and understand log formats. It knows that [ERROR] means something went wrong. It recognizes timestamps, severity levels, and message patterns. When you show it a stack trace, it can explain what the error means in plain language.

Answer questions about logs. You can ask it "What errors happened in the last hour?" or "Why did the application crash?" and it gives you answers based on what it sees in the logs. It's like having a junior engineer who can read logs but doesn't yet know how to take action.

Summarize log patterns. Instead of scrolling through thousands of log lines, you get a summary: "Database connection failed 45 times between 2 AM and 3 AM, retried successfully after timeout increased."

What It Doesn't Do

A Level 1 agent is deliberately limited. It doesn't connect to multiple log sources. It doesn't make routing decisions. It doesn't take actions. It doesn't remember what happened yesterday. It's a stateless analyzer that processes whatever you give it.

This might sound too simple to be useful, but here's the reality: a well-built Level 1 agent solves real problems. When you're debugging at 3 AM and drowning in logs, having something that can instantly summarize what's happening is incredibly valuable.

Building Level 1

In code terms, Level 1 means:

You have a function that reads logs from a file or string. You send those logs to an AI model with a prompt like "You are a DevOps engineer analyzing application logs. Identify errors, patterns, and potential issues." The model responds with analysis. You display the results.

That's your entire system. Maybe 100-150 lines of Python code. No database, no API integrations, no complex orchestration.



Figure 5.2: Level 1 Architecture - A simple pipeline from logs to analysis.

Why Start Here

You might be tempted to jump straight to Level 3 or 4 because they sound more impressive. Don't. Here's why Level 1 matters:

You learn the fundamentals. How do you structure prompts for log analysis? What makes a good response versus a vague one? How do you handle different log formats? These questions are easier to answer when you're not also dealing with multiple data sources and complex workflows.

You build working software fast. You can have a functioning Level 1 agent in an afternoon. That early win builds momentum and confidence. Plus, you'll actually use it while building the more advanced versions.

You discover what you really need. Once you have Level 1 running, you'll quickly see where it falls short for your specific use case. Maybe you need it to handle multiple log sources. Maybe you need memory of past issues. Maybe you need it to take actions. These insights guide your next steps.

By the end of this book series, we'll build up to Level 3. But we'll start at Level 1, and you'll appreciate why.

Level 2: Pattern Recognition and Routing Decisions

Once your Level 1 agent is working, the natural next question is: "Can it do more than just analyze? Can it make decisions?"

Level 2 is where your agent starts to act, not just observe.

What It Does

A Level 2 agent recognizes patterns in logs and makes routing decisions based on what it finds. This means it can:

Categorize issues by type and severity. Instead of just saying “there are errors,” it distinguishes between “database timeout that self-recovered” versus “authentication service completely down.” It assigns severity levels—P1 for critical incidents, P2 for degraded service, P3 for warnings.

Route alerts to the right people. When it detects a database issue, it alerts the database team. When it sees authentication failures, it alerts the security team. When it finds memory leaks, it alerts the application developers. The routing is intelligent, not just keyword-based.

Trigger automated responses. For known patterns, it can take action. If it sees disk space warnings, it can trigger cleanup scripts. If it detects a service crash, it can attempt a restart. If it finds security anomalies, it can send alerts to your security monitoring system.

Track patterns over time. Level 2 introduces basic memory. The agent remembers what errors it’s seen recently. If the same error happens 10 times in an hour, it escalates instead of sending 10 separate alerts. This requires maintaining state between runs.

The Key Difference

The jump from Level 1 to Level 2 is about decision-making. Level 1 is passive—it tells you what’s happening. Level 2 is active—it decides what to do about it.

This is where the ReAct pattern (Reason and Act) becomes important. The agent reasons about what it sees in the logs, then acts based on that reasoning. It’s not just pattern matching with if-statements. It’s using AI to understand context and make judgment calls.

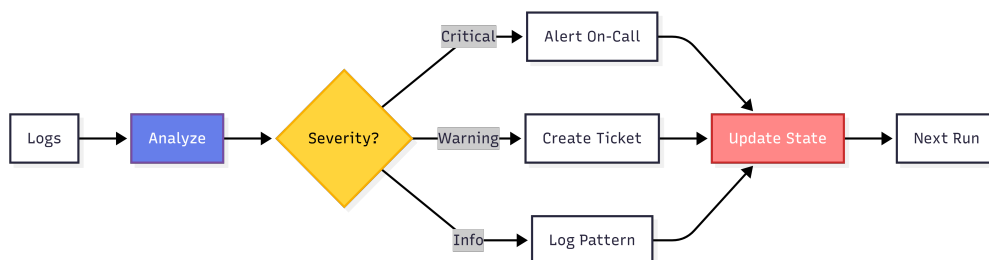


Figure 5.3: Level 2 Decision Flow - The agent analyzes, decides severity, takes action, and maintains state.

Building Level 2

To build Level 2, you add three things to your Level 1 foundation:

Decision logic. Your prompts now include instructions like “If you see critical errors, set severity to P1. If you see repeated warnings, set severity to P2.” The model outputs structured data—not just text analysis, but JSON with severity levels, affected systems, and recommended actions.

Integration points. You need code that takes the model’s decisions and executes them. If it says “alert database team,” you need an integration with PagerDuty, Slack, or email. If it says “restart service,” you need code that can actually restart that service.

State management. You need to remember what you’ve seen. This can be as simple as a JSON file that tracks recent errors and alert timestamps. When a new log batch comes in, you check: “Have we already alerted about this? Is this a repeat of something we saw 10 minutes ago?”

The complexity goes up, but it’s manageable. You’re still talking about a few hundred lines of Python code, maybe some database or file-based storage, and a few API integrations.

When You Need Level 2

You need Level 2 when passively viewing log analysis isn’t enough. If you find yourself reading your Level 1 agent’s output and then manually deciding who to alert or what action to take, that’s your signal to move to Level 2.

However, there's a catch: Level 2 introduces risk. When your agent starts taking actions automatically, it can make mistakes. A bug in your routing logic might page the wrong team at 3 AM. An overly aggressive restart policy might cause more problems than it solves.

This is why you test thoroughly. Start with read-only actions like creating tickets and sending informational alerts. Once you trust the agent's judgment, expand to actions like service restarts or config changes. And always have human oversight in the loop for critical decisions.

Level 3: Multi-Source Integration

Level 3 is where your AI agent becomes truly useful for real-world DevOps scenarios. This is also where we'll stop in this book series—it's the sweet spot of capability and complexity.

What It Does

A Level 3 agent doesn't just analyze one log file or one log stream. It integrates multiple data sources and correlates information across them:

Connects to your actual logging infrastructure. Instead of reading from files, it queries Elasticsearch for application logs, pulls container logs from Kubernetes, fetches system metrics from CloudWatch, and grabs database logs from RDS. All of this happens in real-time or near-real-time.

Correlates events across systems. This is the killer feature of Level 3. When your application throws an error, a Level 3 agent doesn't just report the error. It checks if there's a corresponding database timeout in RDS logs, if there's a network issue in CloudWatch metrics, if there's a pod restart in Kubernetes. It pieces together the full story.

Provides intelligent context. Instead of saying "service crashed," it says "service crashed after database connection pool exhausted, which happened because the RDS instance hit max connections, which spiked due to a deployment that changed the connection timeout from 30s to 5s." That's actionable intelligence.

Handles complexity at scale. A Level 3 agent deals with the reality of modern infrastructure: multiple AWS accounts, different log formats, various

services and microservices, ephemeral containers, distributed traces. It knows how to query each system efficiently and combine the results meaningfully.

The Architecture Changes

Building Level 3 requires more sophisticated architecture:

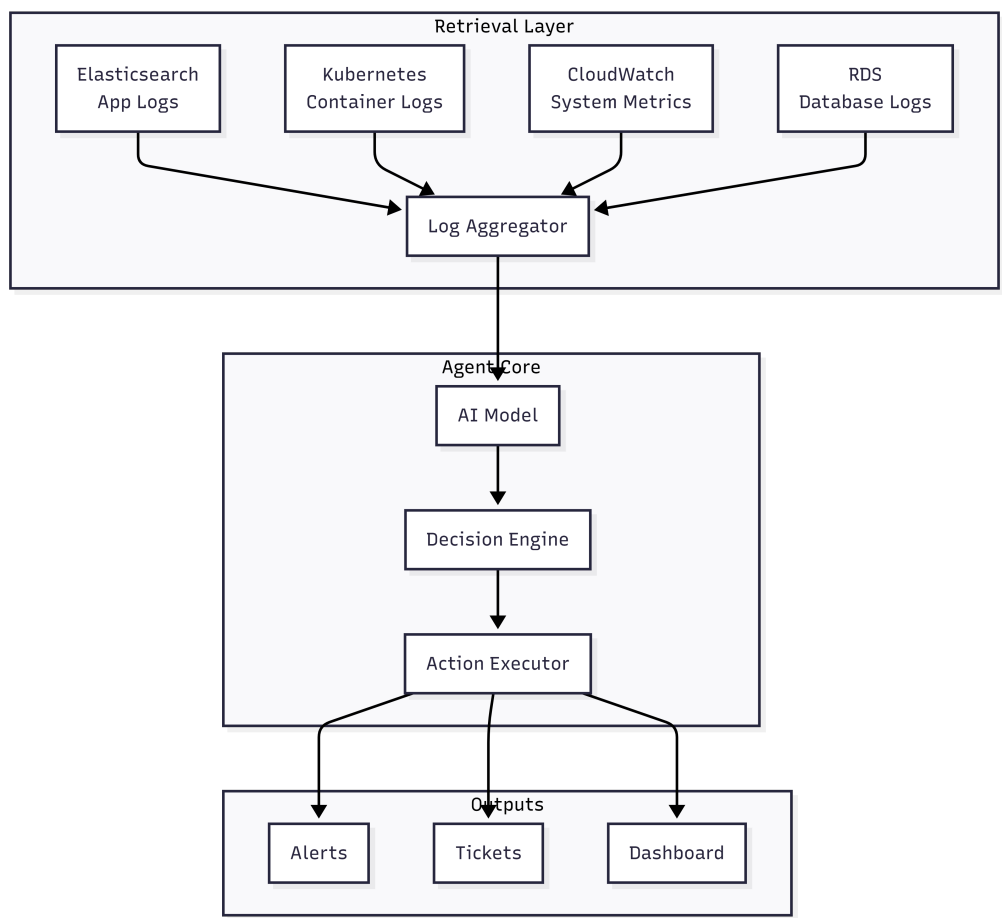


Figure 5.4: Level 3 Architecture - Multiple sources feed into a retrieval layer, which aggregates logs for the AI model.

You need API clients for each log source. You need an aggregation layer that normalizes different log formats into something consistent. You need query optimization because you can't pull every log from every system every time—that would be too slow and expensive.

You also need error handling for when APIs are down, rate limiting when you hit API quotas, and caching to avoid redundant queries. The infrastructure around the AI model becomes just as important as the model itself.

Building Level 3

This is where the book series culminates. By Chapter 12, you'll have built:

Connectors to multiple systems. We'll build Python clients that talk to Elasticsearch, Kubernetes, and AWS services. You'll learn how to authenticate, query efficiently, and handle errors gracefully.

A log aggregation pipeline. We'll create code that pulls logs from multiple sources, normalizes their formats, and combines them into a unified view that the AI model can analyze.

Cross-system correlation. We'll teach the AI agent how to link events across systems. You'll write prompts that instruct the model to look for related events in different logs and build a coherent narrative.

Production-ready infrastructure. We'll add monitoring for the agent itself, error logging, configuration management, and deployment scripts. You'll have something you can actually run in production.

The code base grows to a few thousand lines—not massive, but substantial. You'll use proper project structure with modules, tests, and documentation. This isn't a toy project anymore. It's a real tool you can use and maintain.

Why Stop at Level 3?

You might wonder why we don't go further in this series. The answer is that Level 3 solves most practical problems for most teams. It gives you automated log analysis, intelligent alerting, and multi-system correlation. That's enough to provide real value.

Levels 4 and 5 add complexity that many teams don't need. But let's look at what they entail, so you understand the full landscape.

Level 4: Collaborative Agent Systems

Level 4 is where you move from a single agent to multiple agents working together. This is advanced stuff that you probably don't need unless you're

at significant scale.

What It Does

Instead of one agent that does everything, you have specialized agents that collaborate:

Source-specific agents. One agent focuses on application logs, another on infrastructure logs, another on security logs. Each becomes an expert in its domain.

Role-based agents. You might have an agent that detects issues, another that investigates root causes, and another that proposes remediation steps. They work in sequence, each building on the previous one’s findings.

Parallel processing. Multiple agents can analyze different log sources simultaneously, then come together to share findings. This is faster than having one agent process everything sequentially.

The key pattern here is that agents communicate with each other. When the application log agent finds an error, it asks the infrastructure agent if there were any system issues around the same time. When the security agent sees suspicious activity, it asks the application agent if it correlates with failed authentication attempts.

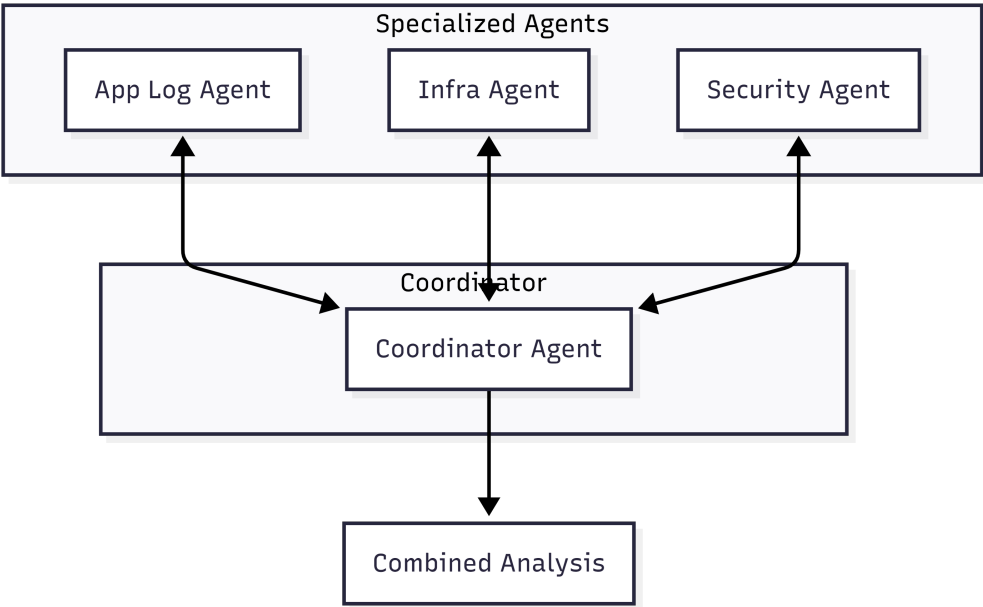


Figure 5.5: Level 4 Multi-Agent System - Specialized agents collaborate through a coordinator.

The Complexity Cost

Level 4 introduces significant complexity. You need to manage multiple AI model instances (which can be expensive). You need coordination logic to determine which agents should run when and how they should share information. You need to handle cases where agents disagree or provide conflicting information.

There's also a debugging challenge. When something goes wrong in a multi-agent system, figuring out which agent made which decision and why becomes much harder than debugging a single agent.

Most teams should stick with Level 3 and only consider Level 4 if they have specific scaling requirements or very specialized domains that benefit from expertise separation.

Level 5: Autonomous Management and Remediation

Level 5 is the “self-driving car” of logging systems. The agent doesn't just analyze and alert—it takes full autonomous action to maintain your infrastructure.

What It Does (Theoretically)

A Level 5 agent would:

Automatically remediate common issues. Service crashed? The agent restarts it. Disk filling up? The agent cleans old logs. Database connections maxed out? The agent increases the connection pool limit.

Self-improve through learning. It tracks which remediations worked and which didn't. Over time, it gets better at predicting what actions will solve problems. It adapts to your specific infrastructure patterns.

Handle novel situations. When it encounters an error it hasn't seen before, it reasons through potential causes, tests hypotheses, and tries solutions. It's not just running playbooks—it's problem-solving.

Manage its own configuration. It determines what logs to monitor, what thresholds to set for alerts, and how to optimize its own performance. It's fully autonomous.

Why This Is Risky

Level 5 sounds amazing, but it's also dangerous. An autonomous agent that can restart services, modify configurations, and take infrastructure actions can cause serious damage if it makes mistakes.

Imagine an agent that misinterprets load spikes as a problem and starts terminating instances to “save resources.” Or an agent that detects errors and keeps restarting a service in a crash loop, making the problem worse. Or an agent that sees database timeouts and decides to increase connection limits to levels that overwhelm your database.

These aren't theoretical risks. They're real failure modes that happen when you give AI systems too much autonomy without sufficient guardrails.

Most companies aren't at Level 5 yet, and many may never be. The risk-reward tradeoff doesn't make sense for most use cases. You're better off with a Level 3 agent that provides intelligence and recommendations, with humans making the final decisions on critical actions.

The Human-in-the-Loop Principle

Even if you build toward Level 5 capabilities, you should maintain human oversight for high-impact decisions. An agent can recommend restarting a critical service, but a human should approve it. An agent can suggest configuration changes, but they should go through review before deployment.

This “human-in-the-loop” approach gives you the benefits of AI speed and intelligence while keeping the safety net of human judgment for consequential actions.

Mapping Your Journey

Now that you understand all five levels, let's map out the practical path for this book series.

Chapters 1-7: Foundation (Level 1 Capability)

In the first part of this series, you're building the foundation. By the end of Chapter 6, you'll have a working Level 1 agent that can read logs and provide intelligent analysis. You'll understand the concepts, have the tools installed, and know how to structure prompts effectively.

Chapters 8-9: Integration (Level 2 Capability)

In the middle section, we'll add decision-making and basic actions. You'll learn how to structure outputs as JSON so you can route decisions to code. You'll integrate with alerting systems. You'll add memory so your agent tracks patterns over time.

By the end of Chapter 9, you'll have a Level 2 agent that doesn't just analyze—it acts.

Chapters 10-12: Production Scale (Level 3 Capability)

The final section brings everything together. You'll build connectors to real log sources like Elasticsearch and CloudWatch. You'll create an aggregation pipeline. You'll add cross-system correlation. You'll make the whole system production-ready with proper error handling, monitoring, and deployment.

By the end of Chapter 12, you'll have a Level 3 agent that you can actually deploy and use in your DevOps workflows.

Beyond the Book

If you want to go further, you'll have the foundation to explore Level 4 and even Level 5. But honestly, Level 3 is where most of the practical value lives. Get there first, use it for a while, and then decide if you need more.

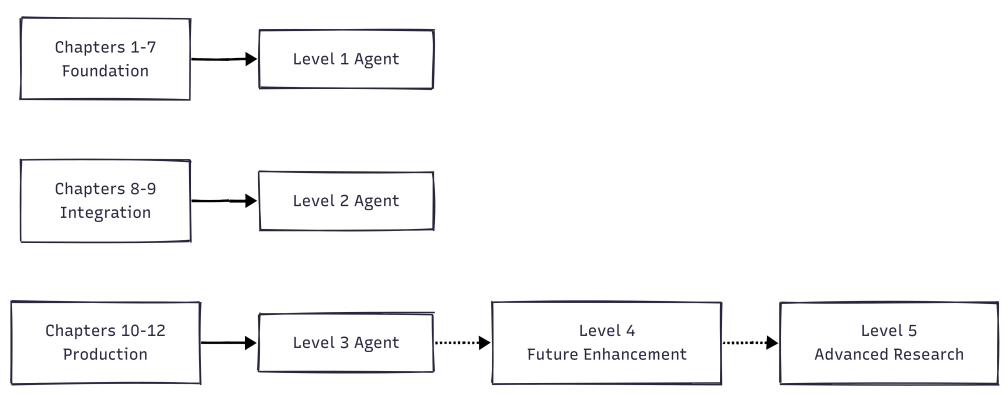


Figure 5.6: Your Learning Path - We'll progress from Level 1 to Level 3 through this book series.

What You've Learned

Let's recap what you now understand about AI logging system levels:

Level 1 (Basic Parser) reads logs and provides analysis. It's simple, fast to build, and genuinely useful. This is where you start and where you'll spend the first half of the book.

Level 2 (Pattern Recognition) adds decision-making and actions. The agent can route alerts, trigger responses, and track patterns over time. This is where you move from passive observation to active response.

Level 3 (Multi-Source Integration) brings in multiple log sources and correlates events across systems. This is the target for this book series—a production-ready agent that provides real value in DevOps workflows.

Level 4 (Collaborative Agents) uses multiple specialized agents working together. It's complex and mainly useful at scale. Most teams don't need this.

Level 5 (Autonomous Management) handles full autonomous remediation with self-improvement. It's powerful but risky. The technology exists, but most organizations shouldn't use it yet.

What's Next

Now that you understand the levels and where you're headed, the next chapter introduces LangChain—a framework that makes building these agents much

easier.

You'll learn why LangChain exists, what problems it solves, and how it simplifies the code you'll write for your AI logging agent. We'll look at core concepts like chains, memory, and tools, and you'll see how they map to the building blocks you learned in Chapter 3.

By the end of Chapter 6, you'll have both the conceptual understanding (this chapter) and the practical framework (next chapter) to start building your Level 1 agent in Chapter 7.

Let's keep moving forward.

Chapter 6: Introduction to LangChain for AI Logging Agents

In the chapter 4, you've got Python installed. You have the Gemini API working. You can send a prompt to an AI model and get a response back. At this point, you might be wondering: "Why do I need LangChain? Can't I just write the code myself?"

The short answer is yes, you can write everything from scratch. The longer answer is that you'll end up rebuilding what LangChain already provides, but with more bugs and less flexibility.

Here's what happened when I built my first AI logging agent without a framework. I wrote a function that sent logs to the Gemini API. It worked fine. Then I needed to add memory so the agent could remember previous errors. I wrote some code to save conversation history in a JSON file. That worked too. Then I needed to give the agent tools—the ability to query Elasticsearch or restart services. I added more code to handle tool calling. Then I needed to chain operations together, where the output of one step became the input of another. More code. Pretty soon I had 800 lines of tangled logic that was hard to test and harder to extend.

When I rewrote it using LangChain, the entire thing shrank to about 150 lines. More importantly, it became easier to understand and modify. That's the value of a framework—it handles the boring, repetitive parts so you can focus on what makes your agent unique.

What LangChain Actually Is

LangChain is a Python library for building applications with language models. It provides abstractions for common patterns you'll encounter when working with AI.

Think of it like Flask or Django for web applications. You could build a web server from scratch using Python's socket library, but why would you? Flask

handles HTTP requests, routing, and sessions for you. Similarly, you could handle AI model calls, conversation history, and tool integration yourself, but LangChain already solved these problems.

The framework is built around a few core concepts: Models, Prompts, Chains, Agents, Memory, and Tools. Each one abstracts away complexity while giving you the flexibility to customize when needed.

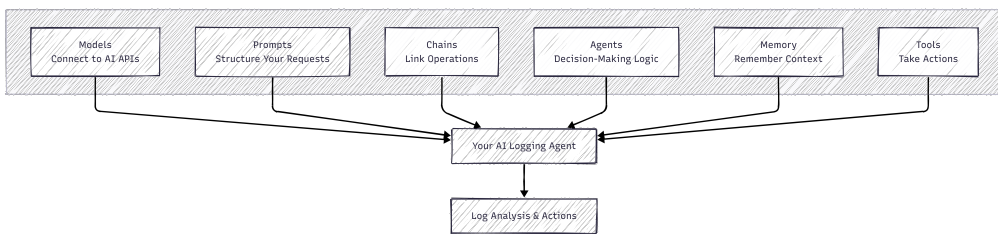


Figure 6.1: LangChain Architecture - Six core concepts that work together to build your agent.

Installing LangChain

Before we dive into concepts, let's get LangChain installed. Assuming you're in your project directory with your virtual environment activated:

```
1 pip install langchain langchain-google-genai
```

The first package is the core LangChain library. The second one provides Gemini integration. LangChain supports many different AI providers through these integration packages—OpenAI, Anthropic, local models, and more. The nice thing is that once you learn LangChain with one provider, switching to another is usually just changing a few import statements.

Important: When using LangChain with Gemini, you only need `langchain-google-genai`. Don't install the standalone `google-generativeai` package as they have conflicting dependencies.

Update your `requirements.txt` to include these:

```
1 python-dotenv>=1.2.1
2 requests>=2.32.5
3 langchain>=1.0.5
4 langchain-google-genai>=3.0.3
```

Models

Connecting to AI

In LangChain, a Model is an abstraction over an AI API. Instead of writing code that makes HTTP requests to Gemini's API and parses JSON responses, you instantiate a Model object and call methods on it.

Here's what calling the Gemini API looks like without LangChain:

```
1 import os
2 import google.generativeai as genai
3
4 genai.configure(api_key=os.environ['GEMINI_API_KEY'])
5 model = genai.GenerativeModel('gemini-2.5-flash-lite')
6 response = model.generate_content('What is the capital of France?')
7 print(response.text)
```

And here's the equivalent with LangChain:

```
1 import os
2 from langchain_google_genai import ChatGoogleGenerativeAI
3
4 llm = ChatGoogleGenerativeAI(
5     model='gemini-2.5-flash-lite',
6     google_api_key=os.environ['GEMINI_API_KEY']
7 )
8 response = llm.invoke('What is the capital of France?')
9 print(response.content)
```

At first glance, these look about the same. The benefit becomes clear when you start building more complex applications. The LangChain Model object has a consistent interface that works across different providers. If you decide to switch from Gemini to OpenAI, you change the import and model initialization, but the rest of your code stays the same. The `invoke` method, the way you handle responses, the error handling—all of it remains consistent.

For a logging agent, this means you can experiment with different models to find the one that works best for your use case without rewriting your entire application.

Prompts

Structuring Your Requests

When you talk to an AI model, you're usually sending two types of messages: system messages that define the agent's role and behavior, and user messages that contain the actual task or data.

A Prompt Template in LangChain is a way to structure these messages with placeholders. Instead of hardcoding strings and using f-strings everywhere, you define templates that LangChain fills in at runtime.

Here's a simple example for log analysis:

```
1 from langchain_core.prompts import ChatPromptTemplate
2
3 prompt = ChatPromptTemplate.from_messages([
4     ("system", "You are a DevOps expert analyzing application logs. Identify
5     ↪ errors, patterns, and potential root causes."),
6     ("user", "Analyze these logs:\n\n{logs}")
7 ])
```

Now when you want to analyze logs, you format the prompt with actual log data:

```
1 formatted = prompt.format_messages(logs=log_content)
```

This might seem like overkill for a simple use case, but it becomes valuable as your prompts get more complex. Imagine you're building an agent that handles multiple types of queries—error analysis, performance investigation, security audits. Each query type needs a different system message and structure. With Prompt Templates, you define each one cleanly and switch between them based on the request.

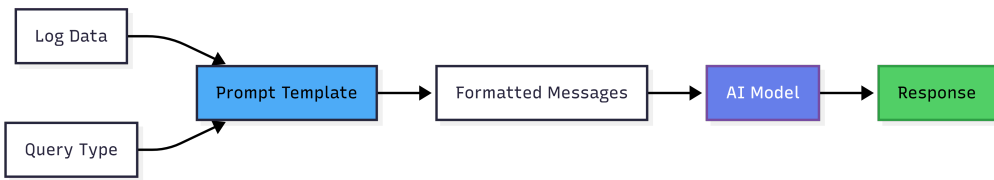


Figure 6.2: Prompt Template Flow - Data and query type combine with template to create messages for the model.

Chains

Linking Operations Together

A Chain is a sequence of operations where the output of one step becomes the input of the next. This is one of the most useful patterns in LangChain.

Let's say you're building a log analyzer that does three things in sequence:

1. Takes raw log lines and summarizes them
2. Takes the summary and identifies the most critical issues
3. Takes the critical issues and suggests remediation steps

Without a Chain, you'd write code like this:

```

1  # Step 1: Summarize
2  summary_prompt = f"Summarize these logs: {logs}"
3  summary = llm.invoke(summary_prompt)
4
5  # Step 2: Identify critical issues
6  issues_prompt = f"From this summary, identify critical issues:
   ↳ {summary.content}"
7  issues = llm.invoke(issues_prompt)
8
9  # Step 3: Suggest remediation
10 remediation_prompt = f"For these issues, suggest fixes: {issues.content}"
11 remediation = llm.invoke(remediation_prompt)
12
13 print(remediation.content)

```

With a Chain, you define the sequence once and then run it using modern LangChain syntax with the pipe operator:

```

1  from langchain_core.prompts import PromptTemplate
2
3  # Define prompts for each step
4  summary_prompt = PromptTemplate(
5      input_variables=["logs"],
6      template="Summarize these logs:\n\n{logs}"
7  )
8
9  issues_prompt = PromptTemplate(
10     input_variables=["summary"],
11     template="From this summary, identify critical issues:\n\n{summary}"
12 )
13
14 remediation_prompt = PromptTemplate(
15     input_variables=["issues"],
16     template="For these issues, suggest fixes:\n\n{issues}"
17 )
18
19 # Create chains using the pipe operator
20 summary_chain = summary_prompt | llm
21 issues_chain = issues_prompt | llm
22 remediation_chain = remediation_prompt | llm
23
24 # Run them in sequence
25 summary = summary_chain.invoke({"logs": log_content})
26 issues = issues_chain.invoke({"summary": summary.content})
27 remediation = remediation_chain.invoke({"issues": issues.content})
28
29 print(remediation.content)

```

This is more maintainable. Each chain is self-contained and testable. You can swap out steps, add new ones, or change the order without rewriting everything. The pipe operator (`|`) is the modern way to connect prompts and models in LangChain.



Figure 6.3: Chain Sequence - Each step processes output from the previous step.

Agents

Adding Decision-Making Logic

This is where things get interesting. An Agent in LangChain is a system that can decide what to do based on user input. Instead of following a fixed sequence, it reasons about the task and chooses which tools to use.

Remember when we talked about ReAct (Reason and Act) in Chapter 3? LangChain Agents implement that pattern. You give the agent a goal and a set of tools, and it figures out how to achieve the goal by using those tools.

For a logging agent, this means you can ask it “Why did the application crash?” and it will:

1. Reason about what information it needs
2. Use a tool to query Elasticsearch for error logs
3. Analyze what it finds
4. If it needs more information, use another tool to check metrics
5. Generate a final answer based on everything it learned

You don't have to write if-statements for every possible scenario. The AI model itself decides what to do next based on the situation.

Here's a basic example of an agent that can search logs:

```

1  from langchain.agents import initialize_agent, Tool
2  from langchain.agents import AgentType
3
4  # Define a tool the agent can use
5  def search_logs(query):
6      # This would actually query Elasticsearch or read files
7      # For now, let's just return a mock result
8      return f"Found 3 errors matching '{query}' in the last hour"
9
10 tools = [
11     Tool(
12         name="SearchLogs",
13         func=search_logs,
14         description="Search application logs. Input should be a search query."
15     )
16 ]
17
18 # Create an agent that can use these tools
19 agent = initialize_agent(
20     tools=tools,
21     llm=llm,
22     agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
23     verbose=True

```

```

24 )
25
26 # Now you can ask questions
27 response = agent.run("What errors happened in the last hour?")
28 print(response)

```

When you run this, you'll see the agent's thought process. It reads your question, decides it needs to search logs, calls the SearchLogs tool with an appropriate query, gets the results, and formulates an answer. That's the ReAct pattern in action.

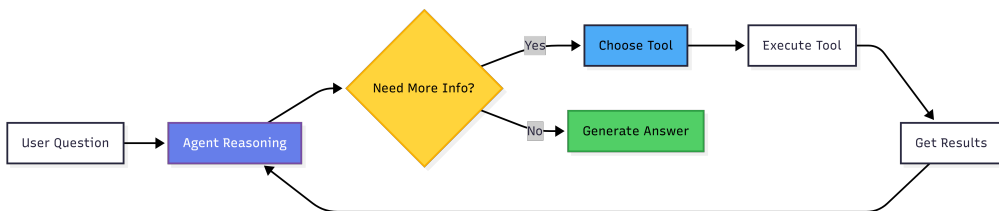


Figure 6.4: Agent Decision Loop - The agent reasons, uses tools, and iterates until it has an answer.

Memory

Keeping Context Between Runs

AI models are stateless. Every time you call the API, it starts fresh with no memory of previous conversations. If you want your agent to remember what happened earlier, you need to manage that yourself.

LangChain provides Memory components that handle this automatically. In modern LangChain, use `InMemoryChatMessageHistory` for storing conversation history:

```
1 from langchain_core.chat_history import InMemoryChatMessageHistory
2
3 # Create chat memory
4 chat_history = InMemoryChatMessageHistory()
5
6 # First interaction
7 chat_history.add_user_message("What errors are in the logs?")
8 chat_history.add_ai_message("Found 3 database connection errors")
9
10 # Second interaction - memory includes previous context
11 chat_history.add_user_message("When did they start?")
12 chat_history.add_ai_message("They started at 2:00 AM")
13
14 # The memory now contains all exchanges
15 print(chat_history.messages)
```

For a logging agent, memory is crucial. When a user asks “Show me more details about that error,” your agent needs to remember which error the user is referring to. When it sees the same error pattern multiple times, it should recognize that it’s a recurring issue, not a new one.

You can also use `RunnableWithMessageHistory` to automatically manage memory in your chains, which we’ll see in the complete example below.

Tools

Giving Your Agent Capabilities

Tools are functions that your agent can call to take actions or gather information. We saw a simple example with the `SearchLogs` tool earlier. In a real logging agent, you’d have tools for:

- Querying Elasticsearch for logs
- Checking CloudWatch metrics
- Restarting services via Kubernetes API
- Creating alerts in PagerDuty
- Running database queries to check application state

In modern LangChain, you define tools using the `@tool` decorator. It’s simpler and cleaner than the old approach:

```

1  from langchain_core.tools import tool
2
3  @tool
4  def query_elasticsearch(query_string: str) -> dict:
5      """Query Elasticsearch for log entries. Input should be an Elasticsearch
        ↳ query string."""
6      # Real implementation would connect to ES and run a query
7      # For now, mock it
8      return {"hits": 5, "sample": "Error: Connection timeout"}
9
10 @tool
11 def restart_service(service_name: str) -> str:
12     """Restart a Kubernetes service. Input should be the service name."""
13     # Real implementation would use kubectl or an API
14     return f"Service {service_name} restarted successfully"
15
16 # Just add the functions to a list
17 tools = [query_elasticsearch, restart_service]

```

The `@tool` decorator does several things for you:

1. Extracts the function name as the tool name
2. Uses the docstring as the tool description (the AI reads this!)
3. Uses type hints to validate inputs
4. Automatically handles errors and return values

The beauty of this approach is that you can add tools without changing your agent code. Want to give your agent the ability to check AWS costs? Write a function that calls the AWS API, add the `@tool` decorator, and add it to the list. The agent will automatically know it can use that capability.

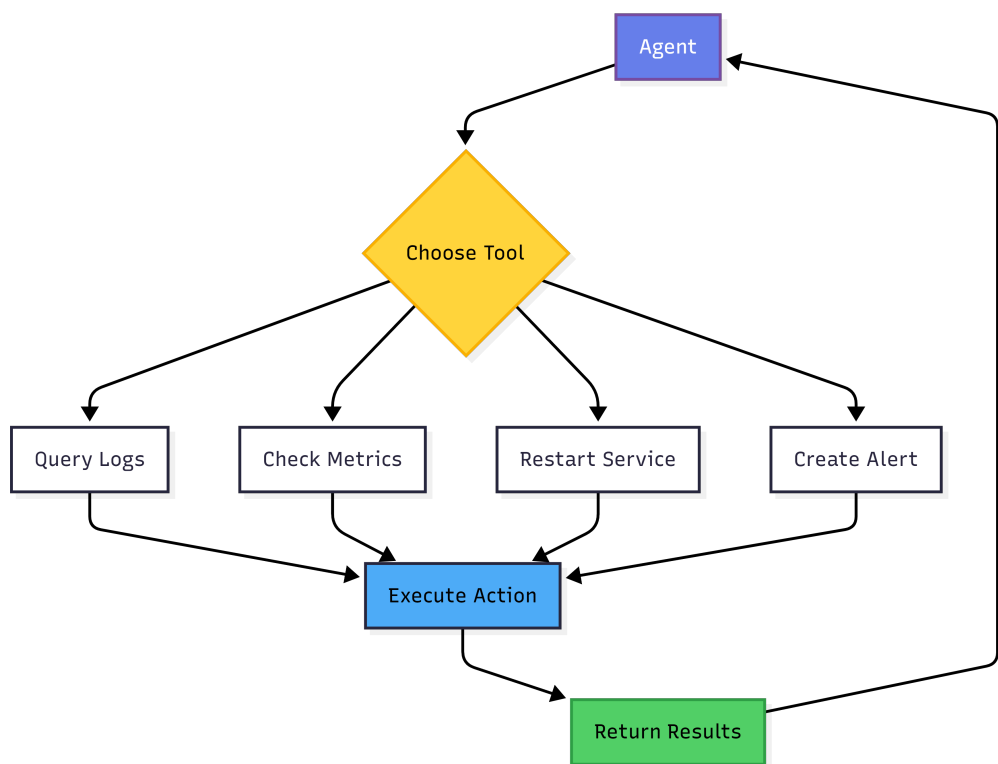


Figure 6.5: Tool Selection - The agent chooses and executes tools based on what it needs to accomplish.

A Complete Example: Log Analyzer with LangChain

Let’s put everything together into a working example. This agent will analyze logs, remember previous conversations, and use tools to gather information. I’ll walk you through each part so you understand exactly what’s happening.

Setting Up the Project Structure

First, create the necessary directories:


```

1 # Create project directory
2 mkdir -p ai-logging-agent/src
3 mkdir -p ai-logging-agent/logs
4 cd ai-logging-agent

```

Create a `.env` file with your API key:

```

1 echo "GEMINI_API_KEY=your_api_key_here" > .env

```

Understanding the Complete Code

Create `src/langchain_log_analyzer.py` with this code. I'll explain each section in detail:

```

1 import os
2 from dotenv import load_dotenv
3 from langchain_google_genai import ChatGoogleGenerativeAI
4 from langchain_core.tools import tool
5 from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
6 from langchain_core.chat_history import InMemoryChatMessageHistory
7 from langchain_core.runnables.history import RunnableWithMessageHistory
8
9 # Load environment variables
10 load_dotenv()

```

We import LangChain's core components. Notice we're using `langchain_core` for tools, prompts, and memory - these are the modern, stable APIs. The `@tool` decorator is simpler than the old `Tool` class.

```

1 # Define tools using the @tool decorator
2 @tool
3 def read_log_file(filename: str) -> str:
4     """Read contents of a log file. Input should be the filename."""
5     try:
6         with open(f"logs/{filename}", 'r') as f:
7             return f.read()
8     except FileNotFoundError:
9         return f"Log file {filename} not found"
10
11 @tool
12 def count_errors(log_content: str) -> str:
13     """Count how many error lines are in log content. Input should be the log
    ↪ content as a string."""

```

```

14     lines = log_content.split('\n')
15     errors = [line for line in lines if 'ERROR' in line or 'error' in line]
16     return f"Found {len(errors)} error lines"
17
18 # Create tools list
19 tools = [read_log_file, count_errors]

```

The `@tool` decorator automatically converts your Python functions into tools that the AI can use. The docstring is crucial - the AI reads it to understand when to use each tool. Type hints (`filename: str`) help LangChain validate inputs.

```

1 # Initialize the model
2 llm = ChatGoogleGenerativeAI(
3     model='gemini-2.5-flash',
4     google_api_key=os.environ['GEMINI_API_KEY'],
5     temperature=0.1
6 )
7
8 # Bind tools to the model
9 llm_with_tools = llm.bind_tools(tools)

```

We create a model instance and bind our tools to it. The `bind_tools()` method tells Gemini “these are the functions you can call.” The low temperature (0.1) makes responses more consistent and less creative - good for log analysis.

```

1 # Create chat memory
2 chat_history = InMemoryChatMessageHistory()
3
4 # Create a prompt that includes message history
5 prompt = ChatPromptTemplate.from_messages([
6     ("system", "You are a DevOps expert analyzing application logs. Use the
7     ↪ available tools to read log files and analyze them."),
8     MessagesPlaceholder(variable_name="chat_history"),
9     ("user", "{input}"),
10 ])
11
12 # Create a chain with the prompt and model
13 chain = prompt | llm_with_tools
14
15 # Wrap the chain with message history
16 chain_with_history = RunnableWithMessageHistory(
17     chain,
18     lambda session_id: chat_history,

```

```

18     input_messages_key="input",
19     history_messages_key="chat_history",
20 )

```

This is where LangChain's power shows. We create:

1. **Memory** - InMemoryChatMessageHistory stores the conversation
2. **Prompt** - Defines the system role and includes a placeholder for chat history
3. **Chain** - Uses the pipe operator (`|`) to connect prompt → model (modern LangChain syntax)
4. **History wrapper** - Automatically injects conversation history into each request

The MessagesPlaceholder is like a slot where LangChain inserts previous messages. You don't manage this manually.

```

1  # Helper function to extract text from response
2  def extract_response_text(response) -> str:
3      """Extract text content from various response formats."""
4      if hasattr(response, 'content'):
5          if isinstance(response.content, str):
6              return response.content
7          elif isinstance(response.content, list):
8              # Handle structured content (list of content blocks)
9              text_parts = []
10             for block in response.content:
11                 if isinstance(block, dict) and 'text' in block:
12                     text_parts.append(block['text'])
13                 elif isinstance(block, str):
14                     text_parts.append(block)
15             return ''.join(text_parts)
16         else:
17             return str(response.content)
18     return str(response)

```

AI models can return responses in different formats - sometimes as plain strings, sometimes as structured objects with metadata. This helper function handles all formats and extracts just the text we want to display. It's reusable and keeps our main code clean.

```

1  # Main interaction loop
2  def main():
3      print("AI Log Analyzer ready. Type 'quit' to exit.")
4      print("Try asking: 'Read the app.log file' or 'What errors are in the
      ↪ logs?'\n")
5
6      session_id = "default_session"
7
8      while True:
9          user_input = input("You: ")
10         if user_input.lower() in ['quit', 'exit']:
11             break
12
13         try:
14             # Get response from chain with history
15             response = chain_with_history.invoke(
16                 {"input": user_input},
17                 config={"configurable": {"session_id": session_id}}
18             )

```

We invoke the chain with history. The `session_id` lets you maintain separate conversations if needed. LangChain automatically:

- Loads previous messages from `chat_history`
- Adds them to the prompt
- Sends everything to the model
- Returns the response

```

1      # Check if model wants to use tools
2      if hasattr(response, 'tool_calls') and response.tool_calls:
3          # Execute tool calls
4          for tool_call in response.tool_calls:
5              tool_name = tool_call['name']
6              tool_args = tool_call['args']
7
8          # Find and execute the tool
9          for tool in tools:
10             if tool.name == tool_name:
11                 result = tool.invoke(tool_args)
12                 print(f"\n[Tool: {tool_name}]\n{result}\n")

```

When the model decides it needs information, it returns `tool_calls` - a list of tools to execute with their arguments. We loop through each call, find the matching tool, and execute it. The `[Tool: name]` prefix shows users what's happening behind the scenes.

```

1         # Get final response based on tool results
2         final_response = llm.invoke([
3             ("system", "You are a DevOps expert analyzing
4                 ↪ logs. The user asked a question and you
5                 ↪ used a tool to get information. Now provide
6                 ↪ a clear, concise answer to the user's
7                 ↪ question based on the tool results."),
8             ("user", f"User asked: {user_input}\n\nTool
9                 ↪ result:\n{result}\n\nPlease analyze these
10                ↪ actual logs and answer the user's
11                ↪ question."),
12         ])
13
14         # Extract and display response
15         response_text = extract_response_text(final_response)
16         print(f"Agent: {response_text}\n")
17
18         # Add to chat history
19         chat_history.add_user_message(user_input)
20         chat_history.add_ai_message(response_text)
21         break

```

After executing the tool, we need the model to analyze the results. We create a new prompt that includes:

- The original question
- The tool results
- Clear instructions to analyze

Then we extract the text, display it, and save both the question and answer to memory. This is crucial - without saving to history, the agent won't remember this exchange.

```

1     else:
2         # Direct response without tools
3         response_text = extract_response_text(response)
4         print(f"\nAgent: {response_text}\n")
5
6         # Add to chat history
7         chat_history.add_user_message(user_input)
8         chat_history.add_ai_message(response_text)
9
10    except Exception as e:
11        print(f"Error: {e}\n")
12        import traceback
13        traceback.print_exc()

```

```
14
15 if __name__ == "__main__":
16     main()
```

If the model doesn't need tools (like when asking "What is a log?"), it responds directly. We extract the text, display it, and save to history. Error handling with traceback helps you debug issues during development.

The complete code in 03-ai-agent-for-devops/code/06.

Creating Test Data

Create a sample log file to test with:

```
1 cat > logs/app.log << 'EOF'
2 [2025-01-15 10:23:45] INFO Application started
3 [2025-01-15 10:23:46] INFO Connected to database
4 [2025-01-15 10:24:12] ERROR Database connection timeout
5 [2025-01-15 10:24:13] ERROR Failed to execute query
6 [2025-01-15 10:24:14] INFO Retrying connection
7 [2025-01-15 10:24:15] INFO Connection successful
8 EOF
```

Running the Agent

```
1 python src/langchain_log_analyzer.py
```

Example Conversation

Here's what a real interaction looks like:

```
1 AI Log Analyzer ready. Type 'quit' to exit.
2 Try asking: 'Read the app.log file' or 'What errors are in the logs?'
3
4 You: What errors are in the logs?
5 Agent: I need to know which log file to check. Could you please provide the
   ↪ filename?
6
7 You: app.log
8
9 [Tool: read_log_file]
10 [2025-01-15 10:23:45] INFO Application started
11 [2025-01-15 10:23:46] INFO Connected to database
12 [2025-01-15 10:24:12] ERROR Database connection timeout
13 [2025-01-15 10:24:13] ERROR Failed to execute query
14 [2025-01-15 10:24:14] INFO Retrying connection
15 [2025-01-15 10:24:15] INFO Connection successful
16
17 Agent: The app.log file contains 2 errors:
18 1. At 10:24:12 - Database connection timeout
19 2. At 10:24:13 - Failed to execute query
20
21 The application attempted to retry and successfully reconnected at 10:24:15.
```

Notice: On the second question, the agent remembered the log contents from the first interaction. It didn't need to read the file again because the conversation history included that information. This is memory in action.

The code is about 140 lines but handles complex interactions. Without LangChain, you'd need 400+ lines to manage conversation history, tool calling, and error handling yourself.

Raw API vs. LangChain

Let's be honest about what LangChain adds and what it doesn't.

If you're building a simple script that sends logs to an AI model once and prints the result, LangChain is probably overkill. Writing the raw API call is faster and easier to understand.

But if you're building something that:

- Needs to chain multiple operations together
- Should remember context between runs
- Must use different tools based on the situation

- Will grow in complexity over time
- Needs to work with multiple AI providers

Then LangChain saves you time and headaches. The framework handles conversation management, tool calling, memory persistence, and error handling. You focus on defining what your agent should do, not how to manage state and API calls.

Think of it like building a house. For a small shed, you can probably wing it with some wood and nails. For an actual house, you want a blueprint, proper tools, and building materials that work well together. LangChain is that structure for AI applications.

What's Next

You now understand the building blocks: Models for AI interactions, Prompts for structuring requests, Chains for linking operations, Agents for decision-making, Memory for context, and Tools for capabilities.

In the next chapter, we'll use everything you've learned to build a real Level 1 logging agent. You'll write code that reads log files, analyzes them with AI, and provides useful insights. No more examples—we're building something you can actually use.

After that, we'll progressively add features: decision-making and routing in Level 2, memory and state management, and finally multi-source integration in Level 3. By the end, you'll have a production-capable AI logging agent that handles real DevOps work.

But first, we need to get Level 1 working. Let's build something.

Chapter 7: Building Your First Components

In the previous chapter, we explored LangChain and understood how it simplifies building AI applications. Now it's time to put that knowledge into practice. We're going to build our first AI logging agent, piece by piece, and understand why each component matters.

This is where theory meets reality. You'll see how all those concepts we discussed actually fit together in working code.

What We're Building

Before we dive into the code, let's be clear about what this agent can and cannot do.

Our agent will:

- Read log files from a directory
- Analyze logs using Google Gemini AI
- Answer questions about errors, patterns, and issues
- Remember the conversation context

What it won't do yet:

- Make routing decisions (we'll add that in Chapter 8)
- Take automated actions like restarting services
- Pull logs from multiple sources (that's Chapter 10)
- Store persistent memory across restarts (Chapter 9 covers this)

This is intentionally simple. We're building a foundation, not a skyscraper. Every complex system starts with simple, working components.

The Architecture

When I first started building AI agents, I made everything in one file. It worked, but the moment I wanted to add a feature or fix a bug, I had to dig through hundreds of lines of code. I learned the hard way that good architecture saves you time.

Here's how we're organizing our agent:

```

1  07/
2  └─ src/
3      └─ __init__.py      # Package marker
4      └─ __main__.py      # Package entry point (python -m src)
5      └─ config.py        # Configuration and validation
6      └─ main.py          # Main application logic
7      └─ models/          # AI model wrappers
8          └─ gemini.py
9      └─ tools/            # LangChain tools
10         └─ log_reader.py
11     └─ agents/           # Agent orchestration
12         └─ log_analyzer.py
13     └─ utils/            # Helper functions
14         └─ response.py
15 └─ logs/                 # Sample log files
16 └─ tests/                # Basic tests

```

Here's how the components interact:

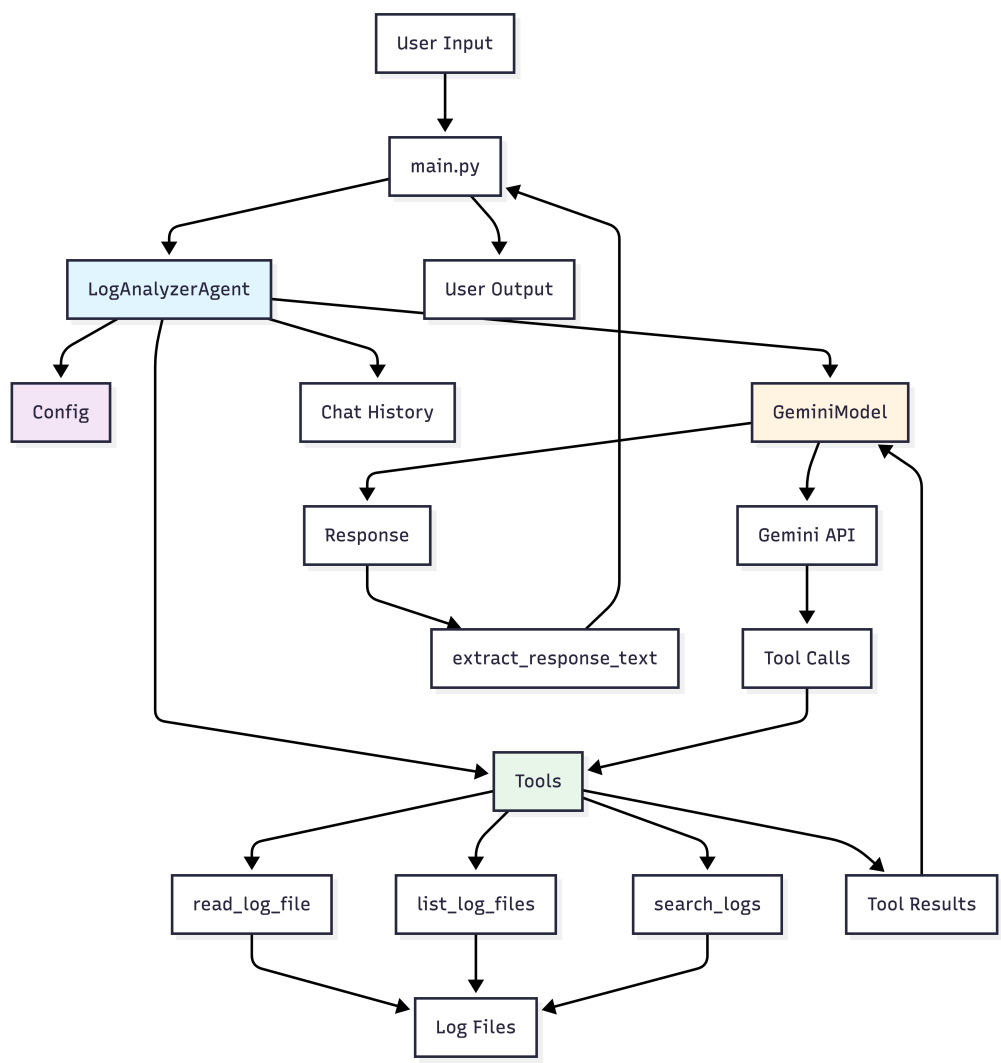


Figure 7.1: The Architecture

Each layer has a purpose. Let me explain why we need each one.

Layer 1: Configuration (config.py)

Configuration might seem boring, but it’s critical. I’ve seen production systems fail because someone hardcoded an API key or forgot to validate environment variables. Good configuration prevents those problems.

Here’s our configuration layer:

```

1  """
2  Configuration management for the AI Logging Agent
3  """
4  import os
5  from dotenv import load_dotenv
6
7  # Load environment variables
8  load_dotenv()
9
10
11 class Config:
12     """Application configuration"""
13
14     # API Configuration
15     GEMINI_API_KEY = os.getenv('GEMINI_API_KEY')
16     GEMINI_MODEL = os.getenv('GEMINI_MODEL', 'gemini-2.5-flash')
17     TEMPERATURE = float(os.getenv('TEMPERATURE', '0.1'))
18
19     # Paths
20     LOG_DIRECTORY = os.getenv('LOG_DIRECTORY', 'logs')
21
22     # Agent Configuration
23     MAX_ITERATIONS = 5
24     VERBOSE = True
25
26     @classmethod
27     def validate(cls):
28         """Validate required configuration"""
29         if not cls.GEMINI_API_KEY:
30             raise ValueError(
31                 "GEMINI_API_KEY not found. "
32                 "Please set it in .env file or environment variables."
33             )
34
35         if not os.path.exists(cls.LOG_DIRECTORY):
36             os.makedirs(cls.LOG_DIRECTORY)
37             print(f"Created log directory: {cls.LOG_DIRECTORY}")
38
39     @classmethod
40     def get_system_prompt(cls) -> str:
41         """Get the system prompt for the agent"""
42         return """You are a DevOps expert specializing in log analysis.
43
44 Your responsibilities:
45 - Analyze application logs to identify errors, warnings, and patterns
46 - Explain technical issues in clear, concise language
47 - Identify root causes and relationships between events
48 - Provide actionable insights
49
50 Your limitations:

```

```
51 - You can only read and analyze logs, not modify them
52 - You cannot take actions like restarting services or modifying configurations
53 - You work with the log files available in the logs directory
54
55 Be direct and helpful. Focus on what's actually in the logs, not speculation."""
```

Why We Need This Layer

Environment Variables: We use `python-dotenv` to load settings from a `.env` file. This means developers can configure their API keys locally without hardcoding them in the source code. In production, you'd use proper secret management, but for development, this works perfectly.

Validation on Startup: The `validate()` method runs when the module loads. If the API key is missing, the program fails immediately with a clear error message. This is much better than failing halfway through execution when you're trying to call the API.

Sensible Defaults: Notice `GEMINI_MODEL` has a default value (`'gemini-2.5-flash'`). Users can override it, but most won't need to. Defaults reduce configuration burden.

System Prompt as Configuration: The system prompt defines the agent's personality and capabilities. By putting it in configuration, we can easily modify the agent's behavior without touching the core logic. Want the agent to be more technical? Just change the prompt.

Temperature Setting: We default to `0.1` for temperature. Remember from Chapter 3, lower temperature means more deterministic responses. For log analysis, we want consistent, reliable answers, not creative variations.

Layer 2: Model Wrapper (`models/gemini.py`)

The model wrapper is a thin abstraction over the `LangChain ChatGoogleGenerativeAI` class. You might wonder: why wrap something that's already wrapped?

```

1  """
2  Google Gemini LLM wrapper
3  """
4  from langchain_google_genai import ChatGoogleGenerativeAI
5  from ..config import Config
6
7
8  class GeminiModel:
9      """Wrapper for Google Gemini LLM"""
10
11     def __init__(self):
12         """Initialize the Gemini model"""
13         self.llm = ChatGoogleGenerativeAI(
14             model=Config.GEMINI_MODEL,
15             google_api_key=Config.GEMINI_API_KEY,
16             temperature=Config.TEMPERATURE
17         )
18
19     def get_llm(self):
20         """Get the LLM instance"""
21         return self.llm
22
23     def get_llm_with_tools(self, tools: list):
24         """Get LLM with tools bound"""
25         return self.llm.bind_tools(tools)

```

Why We Need This Layer

Flexibility: Tomorrow, you might want to switch from Gemini to OpenAI's GPT-4 or Claude. With this wrapper, you change one file. Without it, you'd need to find and update every place you instantiated the model.

Consistent Configuration: Every time you need the LLM, you get it configured exactly the same way with the same temperature, API key, and model name. No risk of typos or inconsistent settings.

Tool Binding Helper: The `get_llm_with_tools()` method binds tools to the model. This is a LangChain pattern we learned in Chapter 6. Having it here as a method makes the agent code cleaner.

Testing Isolation: When you write tests, you can mock this class and return a fake LLM. That lets you test the rest of your code without making actual API calls.

Think of this as dependency injection. The agent doesn't know or care what LLM it's using. It just calls methods on this wrapper.

Layer 3: Tools (tools/log_reader.py)

Tools are how the agent interacts with the real world. In Chapter 6, we built simple tools. Now we're adding three production-ready tools for log analysis.

Tool 1: Read Log File

```

1  @tool
2  def read_log_file(filename: str) -> str:
3      """
4      Read contents of a log file from the logs directory.
5
6      Args:
7          filename: Name of the log file (e.g., 'app.log', 'error.log')
8
9      Returns:
10         String containing the log file contents, or error message if file not
↪ found
11     """
12     log_path = Path(Config.LOG_DIRECTORY) / filename
13
14     try:
15         with open(log_path, 'r', encoding='utf-8') as f:
16             content = f.read()
17
18         # Add metadata
19         file_size = os.path.getsize(log_path)
20         line_count = content.count('\n') + 1
21
22         return f"File: {filename}\nSize: {file_size} bytes\nLines:
↪ {line_count}\n\n{content}"
23
24     except FileNotFoundError:
25         return f"Error: Log file '{filename}' not found in
↪ {Config.LOG_DIRECTORY}/ directory"
26     except PermissionError:
27         return f"Error: Permission denied reading '{filename}'"
28     except Exception as e:
29         return f"Error reading '{filename}': {str(e)}"

```

What's happening here?

The `@tool` decorator tells LangChain this function can be used by the agent. The docstring isn't just documentation—LangChain passes it to the AI model so it knows when to use this tool.

We're using `Path` from `pathlib` for safe file path handling. This prevents directory traversal attacks where someone might try to read `/etc/passwd` by passing `../../etc/passwd` as the filename.

Notice the comprehensive error handling. We catch `FileNotFoundError`, `PermissionError`, and any other exception. Each one returns a clear error message. The AI model needs to understand what went wrong so it can respond appropriately to the user.

The metadata (file size and line count) helps the AI understand the scale of what it's analyzing. A 10-line log file with 2 errors is different from a 10,000-line log file with 2 errors.

Tool 2: List Log Files

```

1  @tool
2  def list_log_files() -> str:
3      """
4      List all available log files in the logs directory.
5
6      Returns:
7      String containing list of available log files with their sizes
8      """
9      log_dir = Path(Config.LOG_DIRECTORY)
10
11     if not log_dir.exists():
12         return f"Error: Log directory '{Config.LOG_DIRECTORY}' does not exist"
13
14     try:
15         log_files = [f for f in log_dir.iterdir() if f.is_file() and f.suffix
16                     ↪ == '.log']
17
18         if not log_files:
19             return f"No .log files found in {Config.LOG_DIRECTORY}/ directory"
20
21         result = f"Available log files in {Config.LOG_DIRECTORY}/:\n\n"
22         for log_file in sorted(log_files):
23             size = log_file.stat().st_size
24             size_kb = size / 1024
25             result += f"  - {log_file.name} ({size_kb:.2f} KB)\n"
26
27         return result
28
29     except Exception as e:
30         return f"Error listing log files: {str(e)}"
```


This tool lets the agent discover what logs are available. When a user asks “What log files do you have?”, the agent can call this tool instead of requiring the user to know the exact filenames.

We filter for .log files only. This prevents the agent from accidentally reading configuration files or other sensitive data in the directory.

The file sizes give context. If a user asks about errors and sees that error.log is 500 KB while app.log is only 5 KB, they know where to focus.

Tool 3: Search Logs

```

1  @tool
2  def search_logs(filename: str, search_term: str) -> str:
3      """
4      Search for a specific term in a log file and return matching lines.
5
6      Args:
7          filename: Name of the log file to search
8          search_term: Term to search for (case-insensitive)
9
10     Returns:
11         String containing matching lines with line numbers
12     """
13     log_path = Path(Config.LOG_DIRECTORY) / filename
14
15     try:
16         with open(log_path, 'r', encoding='utf-8') as f:
17             lines = f.readlines()
18
19         # Search for term (case-insensitive)
20         matches = []
21         for line_num, line in enumerate(lines, start=1):
22             if search_term.lower() in line.lower():
23                 matches.append(f"Line {line_num}: {line.rstrip()}")
24
25         if not matches:
26             return f"No matches found for '{search_term}' in {filename}"
27
28         result = f"Found {len(matches)} matches for '{search_term}' in
29         ↪ {filename}:\n\n"
30         result += "\n".join(matches)
31
32         return result
33     except FileNotFoundError:
34         return f"Error: Log file '{filename}' not found"

```

```

35     except Exception as e:
36         return f"Error searching '{filename}': {str(e)}"

```

Searching is more efficient than reading the entire file. If a log has 10,000 lines but the user just wants to find “database connection failed”, this tool returns only the relevant lines with their line numbers.

Line numbers matter because they help correlate events. If you see “database connection failed” on line 1523 and “retry attempt” on line 1524, you know they’re related.

Why We Need This Layer

Tools are the agent’s hands. Without them, the AI can only talk. With them, it can actually do things—read files, search data, interact with systems.

By keeping tools in a separate module, we can test them independently. We can also reuse them in other agents. Maybe we build a monitoring agent later that needs the same log reading capability.

The `@tool` decorator handles all the complexity of describing these functions to the AI model. LangChain generates schemas that the model uses to decide when and how to call each tool.

Layer 4: Agent Orchestration (agents/log_analyzer.py)

This is where everything comes together. The agent is the conductor, coordinating the model, tools, memory, and prompts.

```

1  from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
2  from langchain_core.chat_history import InMemoryChatMessageHistory
3  from langchain_core.runnables.history import RunnableWithMessageHistory
4
5  from ..models import GeminiModel
6  from ..tools import get_log_tools
7  from ..utils.response import extract_response_text
8  from ..config import Config
9
10
11 class LogAnalyzerAgent:
12     """
13     AI Logging Agent

```

```

14
15     Capabilities:
16     - Read and analyze log files
17     - Answer questions about logs
18     - Maintain conversation history
19
20     Limitations:
21     - No routing decisions
22     - No automated actions
23     - No multi-source integration
24     """
25
26     def __init__(self):
27         """Initialize the agent"""
28         # Initialize model
29         self.model = GeminiModel()
30         self.llm = self.model.get_llm()
31
32         # Get tools
33         self.tools = get_log_tools()
34
35         # Bind tools to model
36         self.llm_with_tools = self.model.get_llm_with_tools(self.tools)
37
38         # Create chat memory
39         self.chat_history = InMemoryChatMessageHistory()
40
41         # Create prompt
42         self.prompt = ChatPromptTemplate.from_messages([
43             ("system", Config.get_system_prompt()),
44             MessagesPlaceholder(variable_name="chat_history"),
45             ("user", "{input}"),
46         ])
47
48         # Create chain
49         chain = self.prompt | self.llm_with_tools
50
51         # Wrap with message history
52         self.chain_with_history = RunnableWithMessageHistory(
53             chain,
54             lambda session_id: self.chat_history,
55             input_messages_key="input",
56             history_messages_key="chat_history",
57         )
58
59         self.session_id = "default_session"

```

What's happening here?

We're using LangChain's modern API from Chapter 6. Let me break down

each piece:

Model Initialization: We create a `GeminiModel` instance and get both the base LLM and the version with tools bound. The tools-bound version knows it can call our log reading functions.

Tools: We get all log tools (read, list, search) in one call. This makes it easy to add more tools later without changing the agent initialization.

Chat Memory: `InMemoryChatMessageHistory` stores the conversation. When a user asks “What about the previous error?”, the agent can look back at the history to understand “previous” means the error we just discussed.

Prompt Template: This is a three-part structure:

1. System message defining the agent’s role (from Config)
2. Chat history placeholder for conversation context
3. User input placeholder for the current question

The Chain: The pipe operator (`|`) creates a chain: prompt → LLM with tools. This is LangChain’s way of composing components.

Message History Wrapper: `RunnableWithMessageHistory` automatically manages conversation history. It adds messages to the history before and after each interaction.

Processing Queries

```

1  def process_query(self, user_input: str) -> str:
2      """
3      Process a user query and return the response.
4
5      Args:
6          user_input: User's question or command
7
8      Returns:
9          String containing the agent's response
10     """
11     try:
12         # Get response from chain with history
13         response = self.chain_with_history.invoke(
14             {"input": user_input},
15             config={"configurable": {"session_id": self.session_id}}
16         )

```

```

17
18     # Check if model wants to use tools
19     if hasattr(response, 'tool_calls') and response.tool_calls:
20         return self._handle_tool_calls(response, user_input)
21     else:
22         # Direct response without tools
23         response_text = extract_response_text(response)
24
25         # Add to chat history
26         self.chat_history.add_user_message(user_input)
27         self.chat_history.add_ai_message(response_text)
28
29         return response_text
30
31     except Exception as e:
32         error_msg = f"Error processing query: {str(e)}"
33         print(f"\n{error_msg}")
34         import traceback
35         traceback.print_exc()
36         return error_msg

```

This method handles the main agent loop:

1. Invoke the chain with the user's input
2. Check if the model wants to call tools
3. If yes, handle tool execution
4. If no, extract the text response and update history
5. If anything fails, return a clear error message

The `session_id` is used by `RunnableWithMessageHistory` to identify which conversation this belongs to. For now, we use a single session, but in a web application, each user would have their own session.

Tool Execution

```

1  def _handle_tool_calls(self, response, user_input: str) -> str:
2      """Handle tool calls from the model"""
3      tool_results = []
4
5      # Execute each tool call
6      for tool_call in response.tool_calls:
7          tool_name = tool_call['name']
8          tool_args = tool_call['args']
9
10         # Find the tool
11         tool_func = None
12         for tool in self.tools:
13             if tool.name == tool_name:
14                 tool_func = tool
15                 break
16
17         if tool_func:
18             try:
19                 result = tool_func.invoke(tool_args)
20                 tool_results.append({
21                     'tool': tool_name,
22                     'result': result
23                 })
24             except Exception as e:
25                 tool_results.append({
26                     'tool': tool_name,
27                     'result': f"Error: {str(e)}"
28                 })
29
30         # Ask model to analyze results
31         analysis_prompt = f"User asked: {user_input}\n\n"
32         analysis_prompt += "Tool results:\n"
33         for tr in tool_results:
34             analysis_prompt += f"\n{tr['tool']}: {tr['result']}\n"
35         analysis_prompt += "\nPlease analyze these results and answer the user's
36         ↪ question."
37
38         final_response = self.llm.invoke(analysis_prompt)
39         response_text = extract_response_text(final_response)
40
41         # Update chat history
42         self.chat_history.add_user_message(user_input)
43         self.chat_history.add_ai_message(response_text)
44
45         return response_text

```

This is where the ReAct pattern (Reasoning and Acting) happens:

1. **Action:** Execute the tools the model requested

2. **Observation:** Collect the results
3. **Reasoning:** Ask the model to analyze the results
4. **Response:** Return the analysis to the user

We manually execute tools instead of using LangChain's built-in agent executor because we want explicit control. This makes debugging easier and lets us add custom logic like rate limiting or result caching later.

Why We Need This Layer

The agent is the glue. It connects configuration, models, tools, and conversation history into a cohesive system. Each component is simple, but together they create intelligent behavior.

By isolating the agent logic, we can swap out components without breaking the whole system. Want to add a new tool? Add it to the tools module. Want to change the AI model? Update the model wrapper. The agent orchestration code stays the same.

Layer 5: Response Utilities (utils/response.py)

This is a small but important utility for handling different response formats from the AI model.

```

1  def extract_response_text(response) -> str:
2      """
3      Extract text content from various response formats.
4
5      LLM responses can come in different formats:
6      - Plain strings
7      - Objects with .content attribute
8      - Structured content with multiple blocks
9
10     Args:
11         response: Response from LLM
12
13     Returns:
14         Extracted text as string
15     """
16     if hasattr(response, 'content'):
17         if isinstance(response.content, str):
18             return response.content

```

```

19         elif isinstance(response.content, list):
20             # Handle structured content (list of content blocks)
21             text_parts = []
22             for block in response.content:
23                 if isinstance(block, dict) and 'text' in block:
24                     text_parts.append(block['text'])
25                 elif isinstance(block, str):
26                     text_parts.append(block)
27             return '\n'.join(text_parts)
28
29     # Fallback: convert to string
30     return str(response)

```

Why we need this?

Different AI providers return responses in different formats. Sometimes you get a string, sometimes an object with a content attribute, sometimes structured content with multiple blocks.

This utility handles all those cases. Without it, we'd have the same response-extraction logic duplicated in multiple places. That's a recipe for bugs when formats change.

Layer 6: Main Entry Point (main.py)

The main entry point ties everything together into an interactive CLI.

```

1  #!/usr/bin/env python3
2  """
3  AI Logging Agent
4  Main entry point for the interactive agent
5  """
6  import sys
7
8  from .agents import LogAnalyzerAgent
9  from .config import Config
10
11
12  def print_banner():
13      """Print welcome banner"""
14      print("=" * 60)
15      print("AI Log Analyzer")
16      print("=" * 60)
17      print("\nCapabilities:")
18      print("  - Read and analyze log files")

```



```

19     print(" - Answer questions about errors and patterns")
20     print(" - Maintain conversation context")
21     print("\nCommands:")
22     print(" 'quit' or 'exit' - Exit the program")
23     print(" 'clear' - Clear conversation history")
24     print(" 'help' - Show available commands")
25     print("=" * 60)
26     print()
27
28
29 def main():
30     """Main interactive loop"""
31     try:
32         # Validate configuration
33         Config.validate()
34
35         # Print banner
36         print_banner()
37
38         # Initialize agent
39         agent = LogAnalyzerAgent()
40
41         # Interactive loop
42         while True:
43             try:
44                 user_input = input("\nYou: ").strip()
45
46                 if not user_input:
47                     continue
48
49                 # Handle commands
50                 if user_input.lower() in ['quit', 'exit']:
51                     print("\nGoodbye!")
52                     break
53
54                 if user_input.lower() == 'clear':
55                     agent = LogAnalyzerAgent()
56                     print("\nConversation history cleared.")
57                     continue
58
59                 if user_input.lower() == 'help':
60                     print_help()
61                     continue
62
63                 # Process query
64                 print("\nAgent:", end=" ")
65                 response = agent.process_query(user_input)
66                 print(response)
67
68             except KeyboardInterrupt:

```

```

69         print("\n\nInterrupted. Type 'quit' to exit.")
70         continue
71     except EOFError:
72         print("\n\nGoodbye!")
73         break
74
75     except Exception as e:
76         print(f"\nFatal error: {e}")
77         sys.exit(1)
78
79
80 if __name__ == "__main__":
81     main()

```

What's happening here?

Relative Imports: Notice we use `from .agents import LogAnalyzerAgent` instead of `from agents import LogAnalyzerAgent`. The `dot(.)` means “from the current package”. This is proper Python package structure and allows the code to be imported or run as a module.

Banner: We print a welcome message showing what the agent can do and what commands are available. This helps users understand the system immediately.

Configuration Validation: Before doing anything else, we validate the configuration. If the API key is missing, the program exits with a clear error message instead of failing mysteriously later.

Agent Initialization: We create one agent instance that persists for the entire session. This maintains conversation history across multiple queries.

Interactive Loop: The `while True` loop keeps the program running until the user types ‘quit’ or presses Ctrl+D.

Command Handling: We handle special commands (quit, clear, help) before passing input to the agent. This prevents the AI from trying to interpret commands as questions.

Error Recovery: If something goes wrong during a query, we catch the exception and continue the loop. The user can keep interacting instead of the whole program crashing.

Graceful Shutdown: `KeyboardInterrupt` (Ctrl+C) and `EOFError` (Ctrl+D) are handled gracefully with friendly messages.

The Package Entry Point (main.py)

To make our package runnable with `python -m src`, we need one more file:

```
1 """
2 Entry point for running the package as a module.
3 Allows: python -m src
4 """
5 from .main import main
6
7 if __name__ == "__main__":
8     main()
```

This tiny file is what makes `python -m src` work. When you run a package as a module, Python looks for `__main__.py` and executes it. We simply import and call the `main()` function from our `main.py` file.

Why We Need This Layer

The main script is the user interface. It makes the agent accessible. Without it, all our components are just library code with no way to interact.

By keeping the CLI logic separate from the agent logic, we could later add a web interface, an API server, or a Slack bot without changing the agent code. The agent doesn't know or care how it's being invoked.

The package structure with `__main__.py` is important because it allows the code to be imported as a library or run as an application. Professional Python projects follow this pattern.

Putting It All Together

Let's see how all these layers work together in a real interaction:

```
1 You: What log files are available?
```

Here's what happens:

1. **main.py** receives the input and calls `agent.process_query()`

2. **log_analyzer.py** invokes the chain with the user's question
3. The **prompt template** combines the system message, chat history, and user input
4. The **LLM** (via `gemini.py` wrapper) receives the formatted prompt
5. The **model decides** it should call the `list_log_files` tool
6. **log_analyzer.py** executes the tool from **log_reader.py**
7. The **tool** returns the list of available files
8. **log_analyzer.py** sends the tool results back to the **LLM**
9. The **LLM** formats a natural language response
10. **response.py** extracts the text from the response object
11. **log_analyzer.py** updates the chat history
12. **main.py** prints the response to the user

Eleven steps happen in milliseconds. Each layer has a single, clear responsibility. That's the power of good architecture.

Testing Your Agent

You've got the code. Now let's make sure it works.

First, set up your environment:

```
1 cd code/07
2 python3 -m venv venv
3 source venv/bin/activate
4 pip install -r requirements.txt
```

Create a `.env` file with your API key:

```
1 GEMINI_API_KEY=your_api_key_here
2 GEMINI_MODEL=gemini-2.5-flash
3 TEMPERATURE=0.1
4 LOG_DIRECTORY=logs
```

Run the agent:

```
1 python -m src
```

Try these queries to test each tool:

Test 1: List files

```
1 You: What log files are available?
```

The agent should call `list_log_files()` and show you the sample logs.

Test 2: Read a file

```
1 You: Read the app.log file
```

The agent should call `read_log_file('app.log')` and display the contents.

Test 3: Search

```
1 You: Search for ERROR in app.log
```

The agent should call `search_logs('app.log', 'ERROR')` and show matching lines.

Test 4: Analysis

```
1 You: What errors occurred and when?
```

The agent should read the logs, identify errors, and explain them with timestamps.

Test 5: Memory

```
1 You: Read app.log
```

```
2 You: What was in the previous file?
```

The agent should remember it just read `app.log` and answer without asking you to specify again.

If any test fails, check the error messages. The layers we built include detailed error handling specifically to help you debug.

What You've Learned

You've just built a production-quality AI agent with six well-defined layers:

1. **Configuration** for environment management and validation
2. **Model wrapper** for AI abstraction and flexibility
3. **Tools** for real-world interactions
4. **Agent orchestration** for coordinating everything
5. **Utilities** for common operations
6. **Entry point** for user interaction

Each layer exists for a reason. Together, they create a system that's maintainable, testable, and extensible.

This is a simple agent. It reads logs and answers questions. But the architecture you've learned here scales to much more complex systems. In Chapter 8, we'll add decision-making and routing. In Chapter 9, we'll add persistent memory. In Chapters 10-12, we'll integrate multiple data sources and deploy to production.

The foundation is solid. Now we build upward.

In the next chapter, we'll teach this agent to make decisions—to route different types of issues to different teams, to assign severity levels, to know when something needs immediate attention. That's when it starts feeling truly intelligent.

Chapter 8: Building a Web Interface with Streamlit

In Chapter 7, we built a terminal-based agent that works perfectly for developers who live in the command line. But here's the reality: not everyone on your team is comfortable with terminals. Your manager wants to check logs. Your product team needs insights. Your support engineers need quick answers.

A terminal interface creates a barrier. A web interface removes it.

This chapter is about making our AI logging agent accessible to everyone. We're going to build a chat interface using Streamlit that feels natural, looks professional, and requires no terminal knowledge.

Why We Need a Web Interface

I've shipped plenty of terminal tools in my career. They're fast to build and efficient to use. But every single time, someone asks: "Can I access this from my browser?"

Terminal tools are great for automation and scripting. But for interactive use, especially with AI, a chat interface makes sense. When you're having a conversation with an AI agent, you want to see the full conversation history, scroll back to earlier responses, and interact visually.

Here's what we gain with a web interface:

Accessibility: Anyone with a browser can use it. No SSH, no terminal setup, no command memorization.

Visual History: You see the entire conversation. In a terminal, once messages scroll off the screen, they're gone. In a web chat, everything persists.

Better UX: We can add sidebars, buttons, formatting, and visual feedback. The user experience becomes significantly better.

Shareability: Want to show a colleague? Just send them a URL. Want to demo to your manager? Open a browser. Want to deploy for your team? Host it once, everyone uses it.

Lower Friction: The barrier to entry drops from “know how to use a terminal” to “know how to type in a text box.” That’s the difference between a tool only developers use and a tool everyone uses.

What We’re Building

We’re taking the exact same agent we built previously and wrapping it in a Streamlit web interface. The core logic doesn’t change. The AI capabilities don’t change. We’re just changing how users interact with it.

What changes:

- Terminal CLI becomes a web-based chat interface
- Manual conversation tracking becomes automatic session management
- Plain text becomes formatted, styled responses
- Local-only becomes shareable via URL

What stays the same:

- The agent logic
- The tools (read, list, search)
- The LLM integration
- The configuration

This is an important architectural principle: separate your business logic from your interface. We can swap interfaces without touching the core agent because we built clean layers in the previous chapter.

Understanding Streamlit

Before we look at the code, let’s talk about Streamlit. It’s a Python framework for building data apps and chat interfaces without writing HTML, CSS, or JavaScript.

You write Python. Streamlit handles the web stuff.

Here’s a simple example:


```

1 import streamlit as st
2
3 st.title("Hello World")
4 name = st.text_input("What's your name?")
5 if name:
6     st.write(f"Hello, {name}!")

```

That's it. Run it with `streamlit run app.py` and you have a web app. No Flask routes, no React components, no CSS files. Just Python.

For our chat interface, Streamlit provides:

- `st.chat_message()` for displaying chat bubbles
- `st.chat_input()` for the input box at the bottom
- `st.session_state` for maintaining state across interactions
- Built-in styling that looks professional out of the box

This makes it perfect for AI chat applications. We can focus on the conversation logic instead of wrestling with frontend frameworks.

The Architecture

Our architecture we established remains intact. We're just adding a new layer on top:

```

1 User Browser
2     ↓
3 Streamlit App (app.py)
4     ↓
5 LogAnalyzerAgent
6     ↓
7 [Config] [GeminiModel] [Tools] [Utils]
8     ↓
9 Gemini API + Log Files

```

The agent doesn't know it's being called from Streamlit. It could be called from a CLI, an API, a Slack bot, or anything else. That's good design.

Project Structure

```

1 08/
2 |─ app.py                      # Streamlit application
3 |─ src/
4 |   |─ __init__.py
5 |   |─ config.py
6 |   |─ models/
7 |   |   |─ gemini.py
8 |   |─ tools/
9 |   |   |─ log_reader.py
10 |  |─ agents/                  # Modified for external history
11 |  |   |─ log_analyzer.py
12 |  |─ utils/
13 |     |─ response.py
14 |─ logs/
15 |─ requirements.txt           # Added: streamlit>=1.30.0
16 |─ Makefile

```

Full code here: [code/08](#)

Most of the code is identical to the terminal version. We only need to modify the agent slightly and add the Streamlit app. This is what good architecture buys you—minimal changes when you add new features.

The Streamlit Application

Let's build this piece by piece, starting with the main application file.

Page Configuration

```

1 import streamlit as st
2 from langchain_core.messages import HumanMessage, AIMessage
3 import sys
4 from pathlib import Path
5
6 # Add src to path
7 sys.path.insert(0, str(Path(__file__).parent))
8
9 from src.agents import LogAnalyzerAgent
10 from src.config import Config
11
12 # Page configuration
13 st.set_page_config(
14     page_title="AI Log Analyzer",
15     page_icon="🔍",
16     layout="wide",

```

```

17     initial_sidebar_state="expanded"
18 )

```

What's happening here?

We import Streamlit and the LangChain message types we'll need. The path manipulation lets us import from our `src` directory—same pattern as before.

`st.set_page_config()` must be the first Streamlit command. It sets the browser tab title, icon, and layout. The wide layout gives us more space for the chat interface, and we expand the sidebar by default so users see the helpful information immediately.

Session State Initialization

Streamlit reruns your entire script every time the user interacts with the page. That sounds inefficient, but Streamlit is smart about it. The key is `st.session_state`—a dictionary that persists across reruns.

```

1  def initialize_session_state():
2      """Initialize Streamlit session state variables"""
3      if 'messages' not in st.session_state:
4          st.session_state.messages = []
5
6      if 'agent' not in st.session_state:
7          try:
8              Config.validate()
9              st.session_state.agent = LogAnalyzerAgent()
10         except ValueError as e:
11             st.error(f"Configuration error: {e}")
12             st.stop()

```

What's happening here?

We check if `messages` exists in session state. If not, we initialize it as an empty list. This will store our chat history.

We do the same for the agent, but we only create it once. Creating the agent involves initializing the LLM, binding tools, and setting up prompts. We don't want to do that on every interaction. We create it once and reuse it.

If configuration validation fails (missing API key, for example), we show an error and stop execution. The user needs to fix their configuration before the app can work.

The Sidebar

```

1  def display_sidebar():
2      """Display sidebar with information and controls"""
3      with st.sidebar:
4          st.title("🔍 AI Log Analyzer")
5          st.markdown("---")
6
7          st.subheader("About")
8          st.markdown("""
9              An AI-powered log analysis tool that helps you:
10             - 📁 Read and analyze log files
11             - 🔍 Search for specific patterns
12             - 💡 Get intelligent insights
13             - 💬 Ask questions in natural language
14             """)
15
16          st.markdown("---")
17
18          st.subheader("Available Tools")
19          st.markdown("""
20             - **read_log_file**: Read a specific log file
21             - **list_log_files**: List all available logs
22             - **search_logs**: Search for patterns in logs
23             """)
24
25          st.markdown("---")
26
27          st.subheader("Example Questions")
28          st.markdown("""
29             - "What log files are available?"
30             - "Read the app.log file"
31             - "What errors are in error.log?"
32             - "Search for 'database' in app.log"
33             - "When did the connection fail?"
34             """)
35
36          st.markdown("---")
37
38          # Clear chat button
39          if st.button("🗑️ Clear Chat History", use_container_width=True):
40              st.session_state.messages = []
41              st.rerun()
42
43          # System info
44          st.markdown("---")
45          st.caption(f"Model: {Config.GEMINI_MODEL}")
46          st.caption(f"Temperature: {Config.TEMPERATURE}")
47          st.caption(f"Log Directory: {Config.LOG_DIRECTORY}")

```

What's happening here?

The sidebar provides context and controls. Users can see what the agent can do, get example questions, and view the current configuration.

The “Clear Chat History” button is important. When clicked, we reset `st.session_state.messages` to an empty list and call `st.rerun()` to refresh the interface. This starts a new conversation.

We use `st.markdown()` for formatted text. Streamlit renders markdown, so we get nice formatting for free. The `st.caption()` at the bottom shows system information in smaller, lighter text.

Displaying Chat Messages

```
1 def display_chat_messages():
2     """Display all chat messages from history"""
3     for message in st.session_state.messages:
4         with st.chat_message(message["role"]):
5             st.markdown(message["content"])
```

This is beautifully simple. We loop through all messages in our session state and display each one using `st.chat_message()`. Streamlit handles the styling, the avatars, the layout—everything.

Messages have two keys: `role` (either “user” or “assistant”) and `content` (the actual text). This matches the OpenAI chat format, which makes it easy to work with.

Message Format Conversion

Here's where things get interesting. Streamlit uses one message format, LangChain uses another. We need to convert between them.

```

1 def convert_to_langchain_messages(messages):
2     """Convert Streamlit messages to LangChain message format"""
3     langchain_messages = []
4     for msg in messages:
5         if msg["role"] == "user":
6             langchain_messages.append(HumanMessage(content=msg["content"]))
7         elif msg["role"] == "assistant":
8             langchain_messages.append(AIMessage(content=msg["content"]))
9     return langchain_messages

```

What's happening here?

Streamlit messages are dictionaries: `{"role": "user", "content": "text"}`.

LangChain messages are objects: `HumanMessage(content="text")` or `AIMessage(content="text")`.

This function converts from Streamlit's format to LangChain's format. We need this because our agent expects LangChain message objects for its chat history.

The Main Loop

```

1 def main():
2     """Main application logic"""
3     # Initialize session state
4     initialize_session_state()
5
6     # Display sidebar
7     display_sidebar()
8
9     # Main content area
10    st.title("💬 Chat with AI Log Analyzer")
11    st.markdown("Ask me anything about your log files!")
12
13    # Display chat messages
14    display_chat_messages()
15
16    # Chat input
17    if prompt := st.chat_input("Ask about your logs..."):
18        # Add user message to chat
19        st.session_state.messages.append({"role": "user", "content": prompt})
20
21        # Display user message
22        with st.chat_message("user"):
23            st.markdown(prompt)

```

```

24
25     # Get agent response
26     with st.chat_message("assistant"):
27         with st.spinner("Analyzing..."):
28             # Convert chat history to LangChain format
29             chat_history = convert_to_langchain_messages(
30                 st.session_state.messages[:-1] # Exclude the current message
31             )
32
33             # Get response from agent
34             response = st.session_state.agent.process_query(
35                 user_input=prompt,
36                 chat_history=chat_history
37             )
38
39             # Display response
40             st.markdown(response)
41
42     # Add assistant response to chat history
43     st.session_state.messages.append({"role": "assistant", "content":
    ↪     response})

```

What's happening here?

The `if prompt := st.chat_input("Ask about your logs...")` line uses Python's walrus operator. It captures the user's input and checks if it exists in one line. When the user presses Enter, `prompt` contains their message.

We add the user's message to our session state immediately. This makes it appear in the chat history on the next rerun.

Then we display the user message using `st.chat_message("user")`. This creates the chat bubble on the right side (or left, depending on theme).

For the assistant's response, we do something clever. We use `st.spinner("Analyzing...")` to show a loading indicator while the agent thinks. Users know something is happening.

We convert the chat history from Streamlit format to LangChain format, excluding the current message (that's what `[:-1]` does). We don't want to include the message we're currently processing in the history—the agent should see the history *before* this message.

We call `st.session_state.agent.process_query()` with the user input and the converted history. The agent processes the query using its tools and LLM, then returns a response.

We display that response in an assistant chat bubble and add it to our session state so it persists.

Modifying the Agent

We need to make one small change to our agent we built for the terminal. Instead of managing its own chat history internally, it needs to accept history as a parameter.

Here's the key difference in `log_analyzer.py`:

```

1  def process_query(self, user_input: str, chat_history: list = None) -> str:
2      """
3      Process a user query and return the response.
4
5      Args:
6          user_input: User's question or command
7          chat_history: List of previous messages (HumanMessage, AIMessage)
8
9      Returns:
10         String containing the agent's response
11     """
12     if chat_history is None:
13         chat_history = []
14
15     try:
16         # Format messages for the prompt
17         messages = self.prompt.format_messages(
18             chat_history=chat_history,
19             input=user_input
20         )
21
22         # Get response from LLM with tools
23         response = self.llm_with_tools.invoke(messages)
24
25         # Check if model wants to use tools
26         if hasattr(response, 'tool_calls') and response.tool_calls:
27             return self._handle_tool_calls(response, user_input, chat_history)
28         else:
29             # Direct response without tools
30             return extract_response_text(response)
31
32     except Exception as e:
33         error_msg = f"Error processing query: {str(e)}"
34         print(f"\n{error_msg}")
35         import traceback

```



```
36         traceback.print_exc()  
37     return error_msg
```

What's happening here?

The agent now accepts `chat_history` as a parameter instead of managing it with `RunnableWithMessageHistory`. This makes it stateless—the caller (Streamlit) manages the state.

We removed the internal chat history tracking. The agent processes one query, returns one response, and forgets everything. The Streamlit app maintains the conversation context in session state.

This is cleaner for web interfaces. Session state belongs in the web framework layer, not in the business logic layer. The agent shouldn't care how or where its history is stored.

Running the Application

To run the Streamlit app:

```
1 streamlit run app.py
```

Streamlit will start a local web server and open your browser to `http://localhost:8501`. You'll see the chat interface with the sidebar information, and you can start asking questions immediately.

The experience is smooth. Type a question, press Enter, see the response. The conversation history builds up visually. You can scroll back through previous messages. You can clear the history and start fresh. It feels like a real chat application.

What Makes This Better Than the CLI

Let me share what I've learned shipping both CLIs and web UIs for AI tools.

Discoverability: In a CLI, users need to remember commands. In this web UI, example questions are right there in the sidebar. New users know exactly what to try.

History Visibility: In a terminal, once a response scrolls off screen, it's gone unless you scroll up. Here, everything is visible. You can see the entire conversation at a glance.

Error Feedback: When something goes wrong in a CLI, users see a stack trace. Here, we can show a friendly error message in the chat. The experience is more polished.

Sharing: Want to show a colleague something the agent found? In a CLI, you copy and paste terminal output. Here, you just share your screen or send a screenshot. The visual format communicates better.

Adoption: This is the big one. I've built tools that were technically superior but nobody used because they required terminal knowledge. The web UI lowers the barrier to zero.

Session State vs Database

You might notice that our chat history lives in `st.session_state`, which means it disappears when the browser tab closes. That's intentional for this chapter.

For a production system, you'd want persistent storage. You could:

- Store conversations in a database (PostgreSQL, MongoDB)
- Use Redis for session storage
- Save to files on disk
- Integrate with authentication to track per-user conversations

But for development and small team use, session state works fine. It's simple, requires no additional infrastructure, and makes the code easier to understand.

We'll cover persistent memory in Chapter 9. For now, understand that the architecture supports it—we'd just swap out where we store messages from session state to a database.

Testing Your Interface

Start the app and try these interactions:

Test 1: List files

1 You: What log files are available?

The agent should use the `list_log_files` tool and show you the three sample logs.

Test 2: Read a file

1 You: Read the `app.log` file

You should see the full contents with metadata about file size and line count.

Test 3: Memory

1 You: Read `error.log`

2 You: What was the first error `in` the previous file?

The agent should remember that “previous file” means `error.log` and answer correctly.

Test 4: Search

1 You: Search `for 'database' in app.log`

You should see all lines containing the word “database” with line numbers.

Test 5: Clear and restart

Click the “Clear Chat History” button in the sidebar. The conversation should reset. Ask a question that references “the previous file”—the agent should say it doesn’t know what you’re referring to because there is no previous context.

Deployment Options

Once you have it working locally, you might want to share it with your team.

Local Network: Run with `streamlit run app.py --server.address 0.0.0.0` and anyone on your network can access it via your IP address.

Streamlit Cloud: Push your code to GitHub, connect it to Streamlit Cloud, add your API key as a secret, and deploy. You get a public URL for free.

Docker: Package it as a Docker container and deploy anywhere that runs containers.

Behind Authentication: Put it behind your company's SSO or VPN if you're dealing with sensitive logs.

For our purposes, local or Streamlit Cloud deployment works great.

What You've Learned

You've taken a working terminal application and made it accessible through a web browser. More importantly, you did it without changing the core agent logic. The separation between interface and business logic paid off.

Key concepts from this chapter:

Streamlit Session State: How to maintain state across interactions in a stateless web framework.

Message Format Conversion: How to translate between different message formats when integrating systems.

Stateless Agents: Why making your agent stateless and passing context from the outside makes it more flexible.

Progressive Enhancement: How to add a better interface without rewriting your core logic.

This is professional software engineering. You don't rebuild everything when you add a feature. You layer new capabilities on top of solid foundations.

In Chapter 9, we'll add decision-making to this agent. We'll teach it to classify errors by severity, route issues to the right teams, and make intelligent decisions about what requires immediate attention. The web interface we built here will make those capabilities much more accessible to non-technical users.

The foundation is solid. Now we build upward.

Chapter 9: Adding Decision-Making and Actions

In Chapter 8, we built a web interface that lets anyone analyze logs through natural conversation. The agent reads files, searches patterns, and explains what it finds. That's useful, but it's passive. It observes. It reports. It doesn't act.

Real incidents require action. When a pod crashes due to `OutOfMemoryError`, you don't just want analysis. You need someone to restart it. When a service goes down, reading logs is step one. Fixing the problem is step two.

This chapter closes that gap. We're adding the ability for our agent to take actions—specifically, restarting failed Kubernetes pods. But here's what makes this interesting: we're not building automation scripts. We're building an AI that decides when action is needed, recommends the right action, and executes it with your approval.

The Problem with Analysis-Only Agents

I'll be direct: a log analyzer that only reads logs is half-finished.

Imagine the following situation, your Java application pod is in `CrashLoopBackOff`. You open the logs. `OutOfMemoryError` everywhere. Exit code 137. The fix is obvious: restart the pod.

Now imagine your workflow:

1. Use the AI agent to analyze the logs
2. Read the agent's analysis
3. Manually run `kubectl delete pod pod-name -n namespace`
4. Wait for Kubernetes to recreate the pod

The agent told you what's wrong and what to do. But you still had to leave the conversation, open a terminal, remember the `kubectl` command, and execute it manually.

That context switch kills efficiency. More importantly, it wastes the potential of having an AI agent. If the agent knows what's wrong and knows what to do, why can't it just do it?

The answer: it can. That's what we're building.

What We're Adding

This chapter extends our agent with decision-making and action capabilities:

Severity Classification: The agent learns to classify incidents as P1 (critical), P2 (high), P3 (medium), or info. Not all issues are equal. `OutOfMemoryError` crashes are P1. Slow query warnings are P3. The agent needs to understand the difference.

Action Recommendation: Based on severity and issue type, the agent recommends specific actions. For OOM crashes, it recommends pod restart. For high memory usage, it recommends investigation. The recommendation comes with reasoning.

User Approval Workflow: The agent never executes actions blindly. It explains what it will do, why it will do it, and asks for confirmation. You stay in control.

Action Execution: After approval, the agent executes the action using Kubernetes tools. In this chapter, we use placeholder implementations—the tools print what they would do without touching real infrastructure. This lets you learn the patterns safely.

Post-Action Guidance: After executing an action, the agent provides next steps: what to monitor, how to prevent recurrence, what permanent fixes to consider.

The result: a complete incident response workflow from detection to resolution, all within a single conversation.

Real-World Integration Patterns

Let me address something important: the interactive web UI we're building is for learning and demonstration. In production, you wouldn't sit at a browser refreshing logs.

Real-world AI logging agents run in one of these patterns:

Pattern 1: Scheduled Analysis

The agent runs every N minutes, analyzes recent logs, and reports issues. Think of it as intelligent log monitoring that scales with your complexity.

```

1  # Pseudocode - not in this chapter, coming later
2  while True:
3      logs = fetch_recent_logs(last_5_minutes)
4      issues = agent.analyze(logs)
5      if issues.severity >= P2:
6          notify_team(issues)
7          if issues.severity == P1:
8              action = agent.recommend_action(issues)
9              if auto_approve(action):
10                 agent.execute(action)
11     sleep(300)

```

Pattern 2: Alert-Driven Analysis

Your monitoring system (Prometheus, Datadog, CloudWatch) triggers an alert. Instead of going to a human immediately, it goes to the AI agent first. The agent analyzes relevant logs, classifies severity, and either resolves it automatically or escalates with context.

```

1  # Pseudocode - future chapter
2  @app.post("/webhook/alert")
3  def handle_alert(alert: Alert):
4      # Alert from Prometheus/Datadog
5      related_logs = fetch_logs(
6          service=alert.service,
7          timerange=alert.timerange
8      )
9
10     analysis = agent.analyze(related_logs)
11
12     if analysis.can_auto_resolve:
13         action = agent.recommend_action(analysis)
14         result = agent.execute(action)
15         notify_team(f"Auto-resolved: {result}")
16     else:
17         escalate_to_oncall(analysis)

```

Pattern 3: API Integration

Other services call your agent via API. Your CI/CD pipeline runs tests, finds failures, and asks the agent to analyze test logs. Your deployment system detects a bad deploy and asks the agent to compare current logs with pre-deploy baselines.

The UI we're building demonstrates the core workflow. Later chapters will show you how to deploy these patterns in production. The decision-making logic stays the same. Only the trigger mechanism changes.

Understanding the Workflow

Let's walk through a complete incident response flow to understand what we're building. The demo:

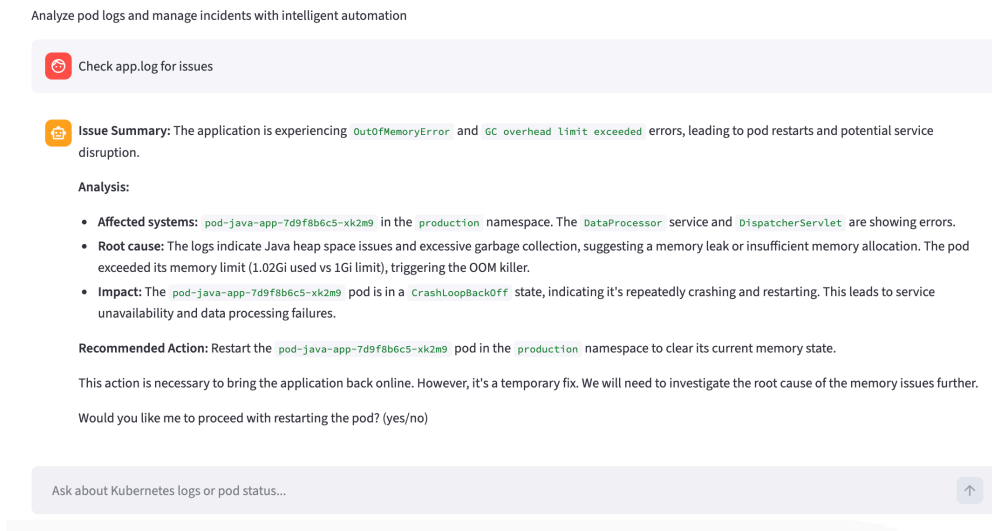


Figure 9.1: The AI Agent with Decision Making

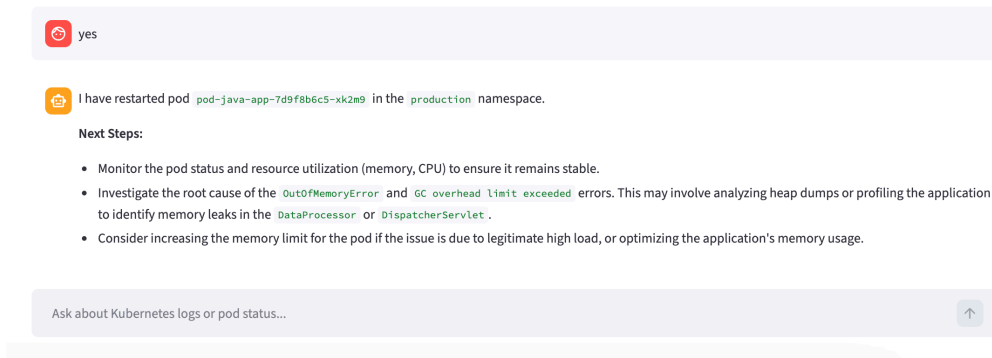


Figure 9.2: The AI Agent Executes Action

Step 1: Log Analysis User asks: “Check app.log for issues” Agent reads the file using `read_log_file` tool (read-only, automatic).

Step 2: Issue Detection Agent finds:

- `OutOfMemoryError`: Java heap space
- GC overhead limit exceeded
- Exit code 137 (OOMKilled)
- Pod status: `CrashLoopBackOff`

Step 3: Severity Classification Agent classifies this as P1 (Critical) because:

- Service is completely down (`CrashLoopBackOff`)
- Error is definitive (OOM, not intermittent)
- Impact is immediate (no availability)

Step 4: Root Cause Analysis Agent determines:

- Memory limit: 1Gi
- Memory usage at crash: 1.02Gi
- Progressive memory growth visible in warnings
- Likely causes: large batch processing, cache growth

Step 5: Analysis Presentation Agent presents:

```

1 Issue Summary: Pod experiencing OutOfMemoryError, in CrashLoopBackOff
2
3 Severity: P1
4
5 Analysis:
6 - Affected systems: pod-java-app-7d9f8b6c5-xk2m9 in production namespace
7 - Root cause: Memory usage (1.02Gi) exceeded limit (1Gi)
8 - Impact: Complete service outage
9 - Evidence: [key log lines with timestamps]
10
11 Recommended Action: Restart the pod to clear memory state and restore service.
12 This is a temporary fix. The pod will be recreated by the Deployment
   ↪ automatically.
13
14 Would you like me to proceed with restarting the pod? (yes/no)

```

Step 6: User Decision User types: “yes”

Step 7: Action Execution Agent calls `restart_kubernetes_pod` tool:

```

1 restart_kubernetes_pod(
2     pod_name="pod-java-app-7d9f8b6c5-xk2m9",
3     namespace="production",
4     reason="OutOfMemoryError recovery"
5 )

```

Step 8: Post-Action Report Agent responds:

```

1 I have restarted pod-java-app-7d9f8b6c5-xk2m9 in production namespace.
2
3 Next Steps:
4 - Monitor pod status and memory usage
5 - Investigate memory leak in DataProcessor and CacheService
6 - Consider increasing memory limit from 1Gi to 2Gi

```

Notice the two-phase response pattern:

- Phase 1: Analysis and recommendation (before action)
- Phase 2: Execution confirmation and next steps (after action)

This keeps the conversation clear and purposeful. You get focused information to make decisions, then focused guidance on what to do next.

Code Structure

We're building on Chapter 8's foundation. Here's what's new:

```

1  09/
2  └─ system_prompt.txt          # NEW: Externalized AI instructions
3  └─ logs/
4     └─ k8s-java-app.log        # NEW: Realistic OOM crash logs
5  └─ src/
6     └─ config.py               # UPDATED: Load prompt from file, K8s config
7     └─ agents/
8        └─ log_analyzer.py      # UPDATED: Multi-step tool execution loop
9     └─ tools/
10        └─ log_reader.py
11        └─ k8s_actions.py      # NEW: Kubernetes action tool
12     └─ utils/
13        └─ response.py
14 └─ app.py                     # UPDATED: Enhanced UI for actions

```

The Action Tool

Let's look at the new Kubernetes action tool:

```

1  # src/tools/k8s_actions.py
2  from langchain.tools import tool
3  from ..config import Config
4
5  @tool
6  def restart_kubernetes_pod(
7      pod_name: str,
8      namespace: str = "default",
9      reason: str = ""
10 ) -> str:
11     """
12     Restart a Kubernetes pod by deleting it (will be recreated by deployment).
13     IMPORTANT: Always ask for user approval before using this tool.
14
15     Args:
16         pod_name: Name of the pod to restart
17         namespace: Kubernetes namespace (default: 'default')
18         reason: Reason for restart (e.g., 'OutOfMemoryError recovery')
19
20     Returns:
21         Success or error message
22     """

```

```

23     # if not Config.is_k8s_configured():
24     #     return "❌ Kubernetes not configured."
25
26     # Placeholder implementation - safe for learning
27     print(f"\n{'='*70}")
28     print(f"PLACEHOLDER: Would restart Kubernetes pod")
29     print(f"{'='*70}")
30     print(f"Namespace: {namespace}")
31     print(f"Pod Name: {pod_name}")
32     print(f"Reason: {reason}")
33     print(f"Action: kubectl delete pod {pod_name} -n {namespace}")
34     print(f"Expected: Pod will be recreated by ReplicaSet/Deployment")
35     print(f"{'='*70}\n")
36
37     return (
38         f"✅ [SIMULATED] Successfully restarted pod '{pod_name}' "
39         f"in namespace '{namespace}'. Pod will be recreated automatically."
40     )

```

This is a placeholder. It prints what it would do without actually doing it. This is intentional:

1. **Safety:** You can learn and test without risking your infrastructure
2. **Portability:** Code runs anywhere, no Kubernetes cluster required
3. **Clarity:** You see exactly what would happen
4. **Patterns:** The real implementation would replace the print statements with actual kubectl calls

The tool description is critical: “IMPORTANT: Always ask for user approval before using this tool.” The AI reads this and knows not to execute automatically.

The Enhanced Agent Loop

Chapter 8’s agent executed tools once and returned. That worked for read-only operations. But action tools need a multi-step loop:

1. Agent analyzes logs (calls `read_log_file`)
2. Agent receives log content
3. Agent formulates response with recommendation
4. User confirms action
5. Agent calls `restart_kubernetes_pod`

6. Agent receives execution result
7. Agent formulates final response

Here's the key code change in `log_analyzer.py`:

```

1  def _handle_tool_calls(self, response, user_input: str, chat_history: list) ->
    ↳ str:
2      """Handle tool calls with iterative execution loop"""
3      from langchain_core.messages import AIMessage, ToolMessage
4
5      messages = self.prompt.format_messages(
6          chat_history=chat_history,
7          input=user_input
8      )
9
10     max_iterations = 5
11     iteration = 0
12     current_response = response
13
14     while iteration < max_iterations:
15         iteration += 1
16
17         # Check if there are tool calls
18         if not (hasattr(current_response, 'tool_calls') and
19                 current_response.tool_calls):
20             # No more tools to call, return final response
21             return extract_response_text(current_response)
22
23         # Execute each tool call
24         tool_messages = []
25         for tool_call in current_response.tool_calls:
26             tool_name = tool_call['name']
27             tool_args = tool_call['args']
28
29             # Find and execute the tool
30             tool_func = next(
31                 (t for t in self.tools if t.name == tool_name),
32                 None
33             )
34
35             if tool_func:
36                 result = tool_func.invoke(tool_args)
37                 tool_messages.append(
38                     ToolMessage(
39                         content=str(result),
40                         tool_call_id=tool_call['id']
41                     )
42                 )
43

```

```

44         # Add messages to conversation
45         messages.append(AIMessage(
46             content=current_response.content,
47             tool_calls=current_response.tool_calls
48         ))
49         messages.extend(tool_messages)
50
51         # Get next response (might call more tools or finish)
52         current_response = self.llm_with_tools.invoke(messages)
53
54         return extract_response_text(current_response)

```

The loop allows the agent to:

- Call `read_log_file`
- Analyze results
- Present recommendation
- Wait for user confirmation (via chat)
- Call `restart_kubernetes_pod`
- Present results

All within a single conversation flow.

Externalizing the System Prompt

Chapter 8 had the system prompt hardcoded in Python. That works, but editing prompts means editing code. For an AI system, the prompt is configuration, not code.

We moved it to `system_prompt.txt`:

```

1  # src/config.py
2  @classmethod
3  def get_system_prompt(cls) -> str:
4      """Get the system prompt for the agent"""
5      prompt_file = os.path.join(
6          os.path.dirname(os.path.dirname(__file__)),
7          'system_prompt.txt'
8      )
9      with open(prompt_file, 'r', encoding='utf-8') as f:
10         return f.read()

```

Now you can iterate on the prompt without touching Python code. This is important because prompt engineering is iterative. You'll want to tweak the instructions, add examples, adjust the tone. Keeping it in a text file makes that workflow faster.

The System Prompt

The prompt is where we teach the AI how to think about incidents. Here's the structure:

Core Capabilities: What the agent does (analyze logs, classify severity, recommend actions, execute with approval)

Severity Classification: Clear definitions of P1, P2, P3, info with examples

Intelligent Response Workflow: Two-phase approach

- Phase 1: Analysis and recommendation (don't include next steps yet)
- Phase 2: Execution report and next steps (after action is done)

Interaction Principles: Be concise, use natural language, explain WHY not just WHAT, never execute without approval

Constraints: Only analyze what's in logs, don't speculate, focus on actionable insights

The two-phase workflow is subtle but important. Many AI prompts try to cram everything into one response. That creates walls of text. By splitting analysis from execution, we keep each response focused on what the user needs in that moment.

Before action: "Here's what's wrong, here's what I recommend, do you want me to proceed?" After action: "Here's what I did, here's what to watch, here's how to prevent this."

Clean. Purposeful. No information overload.

Testing the Agent

Let's walk through testing this. Start the Streamlit app:

```
1 cd code/09
2 streamlit run app.py
```

The UI now shows Kubernetes-specific guidance in the sidebar:

- Configuration status (Gemini API , Kubernetes )
- Available tools including `restart_kubernetes_pod`
- Severity levels with action triggers
- Example questions focused on pod logs

Ask the agent: “Check app.log for issues”

The agent will:

1. Read the log file
2. Detect `OutOfMemoryError` and `CrashLoopBackOff`
3. Classify as P1
4. Analyze root cause (memory exceeded limit)
5. Show evidence (specific log lines)
6. Recommend restarting the pod
7. Ask for confirmation

You’ll see a response like:

```
1 Issue Summary: Pod experiencing OutOfMemoryError, in CrashLoopBackOff
2
3 Severity: P1
4
5 Analysis:
6 - Affected systems: pod-java-app-7d9f8b6c5-xk2m9 in production namespace
7 - Root cause: Memory usage (1.02Gi) exceeded limit (1Gi)
8 - Impact: Complete service outage
9 - Evidence:
10 2024-01-28 08:30:00 ERROR Failed to process large batch: OutOfMemoryError:
    ↳ Java heap space
11 2024-01-28 08:30:06 ERROR Kubernetes detected pod crash: CrashLoopBackOff
12 2024-01-28 08:30:10 ERROR Container killed by OOM killer
13
14 Recommended Action: Restart pod-java-app-7d9f8b6c5-xk2m9 in production
15 namespace to clear memory state and restore service. The pod will be
16 automatically recreated by the Deployment.
17
18 Would you like me to proceed with restarting the pod? (yes/no)
```


Notice: no “Next Steps” yet. The agent is focused on getting your decision.

Reply: “yes”

The agent executes the action and responds:

```
1 I have restarted pod-java-app-7d9f8b6c5-xk2m9 in production namespace.  
2  
3 Next Steps:  
4 - Monitor pod status and memory usage closely  
5 - Investigate memory leak in DataProcessor and CacheService  
6 - Consider increasing memory limit from 1Gi to 2Gi
```

Now you get the forward-looking guidance. Two responses, two purposes, clean separation.

What Makes This Different

You might be thinking: “This is just kubectl with extra steps.” Fair question. Let’s address it.

Context: kubectl restarts pods. But it doesn’t know why you’re restarting. This agent read the logs, understood the problem was OOM, identified the specific pod, checked the namespace, and recommended the exact action. That context matters.

Decision Support: kubectl requires you to know what to do. This agent tells you what to do and why. That’s valuable when you’re paged at 2 AM or when a junior engineer is on call.

Workflow Integration: kubectl is a separate tool. This agent combines log analysis, decision-making, and action execution in one conversation. No context switching.

Auditability: Every action has reasoning. “Why did we restart this pod?” Because the agent detected P1 OOM, analyzed the logs, classified severity, and recommended it based on standard recovery procedures. That’s documented in the conversation.

Scalability: You can’t scale manual kubectl commands. You can scale AI agents that analyze logs and take appropriate actions.

The real value isn’t automating one kubectl command. It’s creating a system that can:

- Analyze any log file
- Detect any type of issue
- Classify any severity level
- Recommend appropriate actions
- Execute with proper approval
- Provide preventive guidance

And you can extend it with more tools, more actions, more intelligence.

Extending the System

This chapter focuses on one action: pod restart. But the architecture supports any action:

Add a rollback tool:

```
1 @tool
2 def rollback_deployment(deployment_name: str, namespace: str = "default"):
3     """Rollback deployment to previous revision."""
4     # Implementation
```

Add a cache clear tool:

```
1 @tool
2 def clear_cache(service_name: str):
3     """Clear application cache."""
4     # Implementation
```

The agent learns about tools from their descriptions. Add a tool, register it in `get_all_tools()`, and the agent can use it. No changes to the agent logic required.

The prompt teaches general principles (analyze, classify, recommend, execute with approval). The tools define specific capabilities. This separation scales.

What's Next

You've built the foundation: the agent can restart a Kubernetes pod. But DevOps work involves dozens of repetitive manual actions. Every time you see "disk full" in logs, you SSH in and clean /tmp. When Redis cache hit rate drops, you flush it. When connection pool exhausted appears, you adjust the limits.

In the next chapter, you'll implement the actions you actually need:

- **Cache management** to clear Redis/Memcached when performance degrades
- **Disk cleanup** to remove old logs and temp files when space runs low
- **Service health checks** that verify and restart unresponsive services
- **Database pool management** to handle connection exhaustion
- **Queue management** to purge backed-up message queues

Each scenario maps a common log pattern to a real action. You'll move from placeholder print statements to actual `kubectl`, `docker`, and system commands—with proper safety checks and rollback mechanisms.

Chapter 10: Building a Complex Agent with Actions

In Chapter 9, we built an agent that can restart a single Kubernetes pod using placeholder code. That was a good first step. But real production incidents don't involve one pod and one action. They involve multiple systems, cascading failures, and a sequence of coordinated responses.

This chapter puts you in a real scenario. Your three-tier AWS application is down. Backend pods are logging “Too many connections” errors. The database has exhausted its connection limit. Orders are failing. Customers are affected.

You need to analyze logs across multiple pods, diagnose the root cause, alert your team on Slack, reboot the RDS instance, and verify recovery.

We're going to build an agent that handles this entire workflow in a single conversation.

The Scenario

You run a three-tier application on AWS:

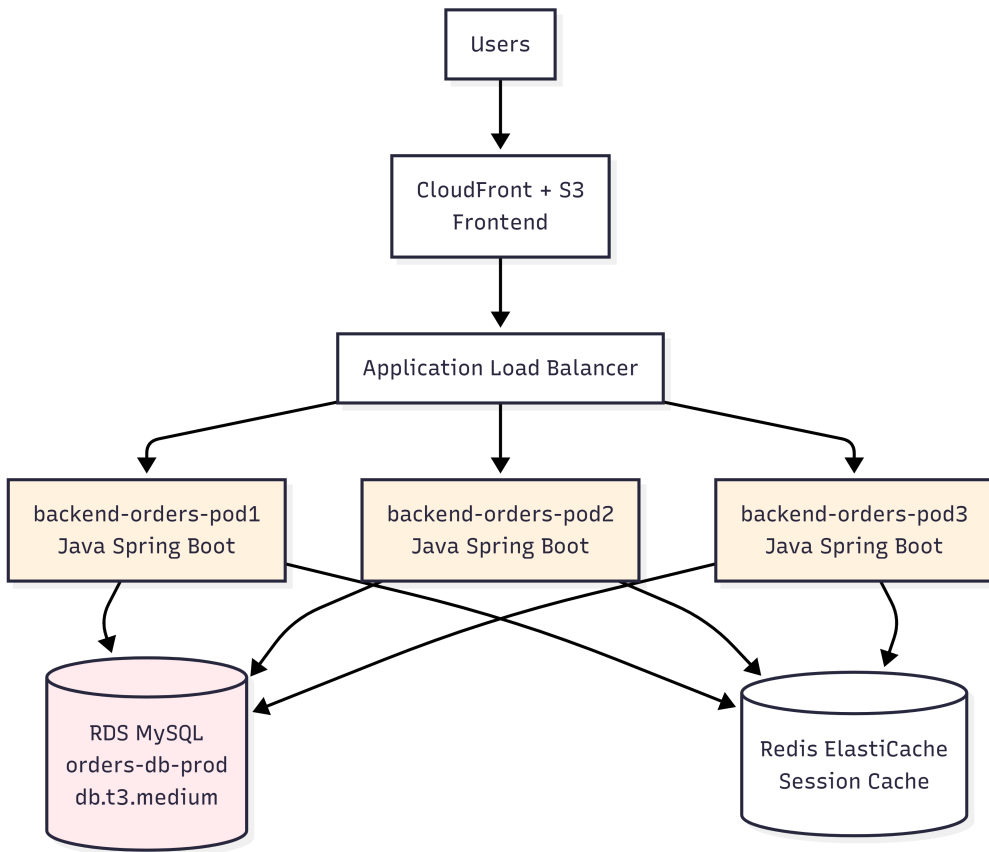


Figure 1. 10.01.png

- **Frontend:** CloudFront serving a React app from S3
- **Backend:** Three Java Spring Boot pods on EKS, behind an Application Load Balancer
- **Data layer:** RDS MySQL (orders-db-prod, db.t3.medium with 150 max connections) and Redis ElastiCache for session caching

Each pod runs a HikariCP connection pool with a max of 50 connections. Three pods $\times$ 50 connections = 150. That's exactly the RDS limit for a db.t3.medium instance.

When traffic spikes, all three pods max out their pools simultaneously. The database hits its connection ceiling. New connections get rejected. Orders fail. The health checks report degraded status. The entire order processing pipeline stalls.

This is a common failure mode in distributed systems. Connection pool sizing looks fine in isolation, but falls apart when every pod scales to its maximum.

Project Structure

```

1  10/
2  ├── app.py                # Streamlit web interface
3  ├── system_prompt.txt    # Agent persona and behavior
4  ├── examples.txt         # Incident resolution guidelines
5  ├── requirements.txt
6  ├── .env.example
7  ├── Makefile
8  ├── logs/                # Simulated pod logs
9  │   ├── backend-orders-pod1.log
10 │   ├── backend-orders-pod2.log
11 │   └── backend-orders-pod3.log
12 ├── src/
13 │   ├── config.py        # Configuration management
14 │   ├── models/
15 │   │   ├── factory.py   # Provider selection
16 │   │   ├── gemini.py    # Google Gemini wrapper
17 │   │   └── github_openai.py # GitHub Models wrapper
18 │   ├── tools/
19 │   │   ├── __init__.py  # Tool classification
20 │   │   ├── log_reader.py # Log file tools
21 │   │   ├── actions.py   # Kubernetes actions
22 │   │   ├── aws_actions.py # RDS management
23 │   │   └── slack_notifier.py # Slack notifications
24 │   ├── agents/
25 │   │   └── log_analyzer.py # Agent with tool-calling loop
26 │   └── utils/
27 │       └── response.py   # LLM response parsing

```

Here's how the components interact at runtime:

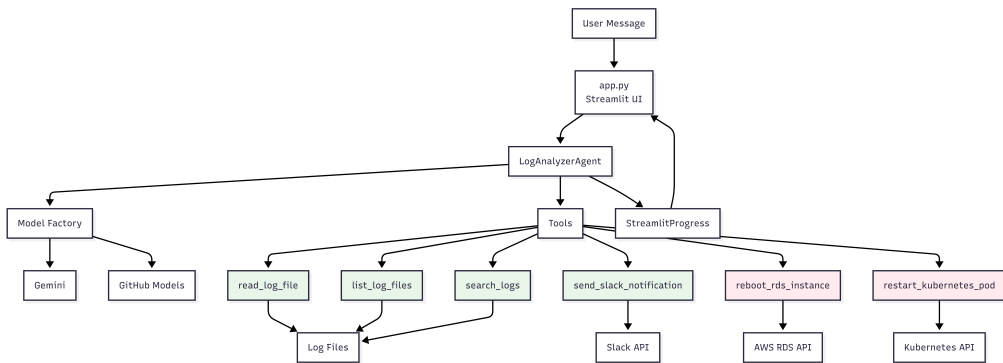


Figure 2. 10.02.png

Green tools auto-execute. Red tools require human approval. That distinction drives the entire approval flow we'll build.

The Log Files

Before building tools, let's look at what the agent will analyze. We have three log files simulating the connection exhaustion scenario.

Here's a condensed view of `backend-orders-pod1.log`:

```

1  2024-02-06 09:00:00 INFO [main] o.s.boot.SpringApplication - Starting
   ↳ OrderService v2.1.0 on backend-orders-6f8d7c4b5-n2k8p
2  2024-02-06 09:00:05 INFO [main] com.zaxxer.hikari.HikariDataSource -
   ↳ HikariPool-1 - Starting...
3  2024-02-06 09:00:06 INFO [main] com.zaxxer.hikari.HikariDataSource -
   ↳ HikariPool-1 - Start completed. Pool size: 10, Max pool size: 50
4  2024-02-06 09:00:08 INFO [main] c.e.config.DatabaseConfig - Connected to RDS:
   ↳ orders-db.c9aksj2x.us-east-1.rds.amazonaws.com:3306/orders_prod
5  ...
6  2024-02-06 10:00:00 WARN [http-nio-8080-exec-8]
   ↳ com.zaxxer.hikari.pool.HikariPool - HikariPool-1 - Connection pool is
   ↳ nearing capacity (35/50 active connections)
7  2024-02-06 10:12:45 ERROR [http-nio-8080-exec-18]
   ↳ com.zaxxer.hikari.pool.HikariPool - HikariPool-1 - Connection is not
   ↳ available, request timed out after 30000ms
8  2024-02-06 10:12:45 ERROR [http-nio-8080-exec-18] c.e.service.OrderService -
   ↳ Failed to process order #ORD-2024-78234: Unable to acquire JDBC Connection
9  Caused by: java.sql.SQLNonTransientConnectionException: Data source rejected
   ↳ establishment of connection, message from server: "Too many connections"
10 ...
11 2024-02-06 10:13:30 ERROR [connection-monitor] DATABASE CONNECTION EXHAUSTION
   ↳ DETECTED - RDS Instance: orders-db.c9aksj2x.us-east-1.rds.amazonaws.com
  
```

```

12 2024-02-06 10:13:30 ERROR [connection-monitor] Max connections: 150 (RDS
    ↳ db.t3.medium), Active: 150/150
13 2024-02-06 10:13:30 ERROR [connection-monitor] Total pods connected: 3 (pod1:
    ↳ 50, pod2: 50, pod3: 50)
14 2024-02-06 10:13:35 FATAL [main] c.e.OrderServiceApplication - Service is
    ↳ unable to serve requests. All database connections exhausted.

```

The pattern across all three pods is identical: pool fills up gradually, then errors cascade once the RDS limit is hit. The agent needs to read all three files and correlate the evidence—this isn’t a single-pod issue, it’s a system-wide failure.

Building the Tools

Tool Classification

The first design decision is which tools are safe to auto-execute and which need human approval. This classification is simple but critical:

```

1 # src/tools/__init__.py
2
3 SAFE_TOOLS = {'read_log_file', 'list_log_files', 'search_logs',
    ↳ 'send_slack_notification'}
4 APPROVAL_REQUIRED_TOOLS = {'reboot_rds_instance', 'restart_kubernetes_pod'}
5
6
7 def requires_approval(tool_name: str) -> bool:
8     """Check if a tool requires human approval before execution"""
9     return tool_name in APPROVAL_REQUIRED_TOOLS

```

Reading logs can’t break anything. Sending a Slack notification is informational. But rebooting a database drops all connections and causes downtime. Restarting a pod kills the running process. Those require a human to say “yes.”

This is the same principle from Chapter 9, but now we have real tools behind it.

AWS RDS Reboot Tool

This is the first tool in the series that talks to a real cloud API. It uses boto3 to reboot an RDS instance, with a fallback to placeholder mode when AWS credentials aren’t configured:


```

1  # src/tools/aws_actions.py
2
3  @tool
4  def reboot_rds_instance(db_instance_id: str, reason: str = "") -> str:
5      """
6      Reboot an AWS RDS database instance to reset connections and restore
        ↳ service.
7      IMPORTANT: Always ask for user approval before using this tool. This will
        ↳ cause
8      a brief service interruption (typically 1-3 minutes for a reboot).
9
10     Use this when logs show "Too many connections" errors across multiple
        ↳ application pods,
11     indicating the RDS instance has exhausted its connection limit.
12
13     Args:
14         db_instance_id: The RDS instance identifier (e.g., 'orders-db-prod')
15         reason: Reason for the reboot (e.g., 'Connection pool exhaustion
        ↳ recovery')
16     """
17     if _is_aws_configured():
18         return _reboot_rds_real(db_instance_id, reason)
19     else:
20         return _reboot_rds_placeholder(db_instance_id, reason)

```

The design pattern here matters. The tool function itself decides whether to call the real AWS API or simulate the action. The agent doesn't know the difference. It calls `reboot_rds_instance` either way. This lets you develop and test the entire workflow locally without AWS credentials, then flip a switch in production.

The real implementation checks the instance status before rebooting:

```

1  def _reboot_rds_real(db_instance_id: str, reason: str) -> str:
2      """Real AWS RDS reboot using boto3"""
3      try:
4          import boto3
5          from botocore.exceptions import ClientError
6
7          region = os.getenv('AWS_REGION', 'us-east-1')
8          rds_client = boto3.client('rds', region_name=region)
9
10         # Check instance status before rebooting
11         response = rds_client.describe_db_instances(
12             DBInstanceIdentifier=db_instance_id
13         )
14         instance = response['DBInstances'][0]
15         status = instance['DBInstanceStatus']

```

```

16
17     if status != 'available':
18         return (
19             f"Cannot reboot RDS instance '{db_instance_id}'. "
20             f"Current status: {status}. Instance must be 'available' to
                ↪ reboot."
21         )
22
23     # Perform the reboot
24     rds_client.reboot_db_instance(DBInstanceIdentifier=db_instance_id)
25
26     return (
27         f"Successfully initiated reboot of RDS instance '{db_instance_id}' "
28         f"in region {region}.\n"
29         f"Reason: {reason}\n"
30         f"Expected downtime: 1-3 minutes.\n"
31         f"All existing connections will be dropped and new connections "
32         f"can be established after reboot."
33     )
34
35 except ClientError as e:
36     return f"AWS API error: {e.response['Error']['Message']}"

```

Two things to notice. First, we check that the instance is available before trying to reboot. If it's already rebooting or in maintenance, we stop and tell the agent why. Second, the return message includes actionable information: expected downtime, what happens to connections, and what the application should do. The agent uses this information in its response to the user.

The placeholder version prints the equivalent AWS CLI command and returns a simulated success:

```

1 def _reboot_rds_placeholder(db_instance_id: str, reason: str) -> str:
2     """Placeholder RDS reboot for learning/testing"""
3     region = os.getenv('AWS_REGION', 'us-east-1')
4
5     print(f"\nPLACEHOLDER: Would reboot AWS RDS instance")
6     print(f"Instance ID:      {db_instance_id}")
7     print(f"Region:              {region}")
8     print(f"AWS CLI command:  aws rds reboot-db-instance "
9           f"--db-instance-identifier {db_instance_id}")
10
11     return (
12         f"[SIMULATED] Successfully initiated reboot of RDS instance "
13         f"'{db_instance_id}' in region {region}.\n"
14         f"Reason: {reason}\n"
15         f"Expected downtime: 1-3 minutes.\n"

```

```

16         f"All existing database connections will be dropped. "
17         f"Application pods will reconnect automatically via HikariCP connection
18         ↪ pool."
19     )

```

This is useful for learning. You can run the full workflow, see exactly what would happen, and verify the agent’s reasoning—without touching real infrastructure.

Slack Notification Tool

The Slack tool sends structured incident reports to your team channel. Unlike the RDS tool, Slack notifications are classified as safe—they’re informational and can’t cause damage:

```

1  # src/tools/slack_notifier.py
2
3  @tool
4  def send_slack_notification(
5      channel: str,
6      summary: str,
7      severity: str = "P1",
8      details: str = "",
9      actions_taken: str = ""
10 ) -> str:
11     """
12     Send an incident notification to a Slack channel with details about
13     the issue and actions taken.
14
15     Args:
16         channel: Slack channel name (e.g., '#devops-alerts')
17         summary: Brief incident summary
18         severity: Incident severity level ('P1', 'P2', 'P3', 'info')
19         details: Detailed description of the issue and root cause
20         actions_taken: Description of remediation actions that were executed
21     """
22     if _is_slack_configured():
23         return _send_slack_real(channel, summary, severity, details,
24                                ↪ actions_taken)
25     else:
26         return _send_slack_placeholder(channel, summary, severity, details,
27                                        ↪ actions_taken)

```

The real implementation builds a structured Slack message with Block Kit formatting—headers, sections, severity tags—and sends it via webhook. The

placeholder version prints the payload to the console so you can verify the message format.

The tool description mentions “P1”, “P2”, “P3”, and “info” severity levels. The agent picks the right severity based on what it finds in the logs. Connection exhaustion affecting all pods and causing order failures? That’s P1. A slow query warning on one pod? That’s P3.

Registering Tools

All tools get registered in a single function. The agent calls this once during initialization:

```

1 # src/tools/__init__.py
2
3 def get_all_tools():
4     """Get all available tools for the agent"""
5     tools = get_log_tools()           # read, list, search
6     tools.append(restart_kubernetes_pod) # requires approval
7     tools.append(reboot_rds_instance)   # requires approval
8     tools.append(send_slack_notification) # safe
9     return tools

```

Adding a new tool is straightforward: write the function, decorate it with `@tool`, add it to `get_all_tools()`, and decide whether it goes in `SAFE_TOOLS` or `APPROVAL_REQUIRED_TOOLS`. The agent learns about it from the tool’s doc-string. No changes to the agent logic.

The Model Factory

In Chapter 9, we only supported Gemini. That’s limiting. Different teams use different providers. Some organizations can’t send data to Google. Others have GitHub Copilot subscriptions with model access.

The model factory lets you switch providers with a single environment variable:

```

1  # src/models/factory.py
2
3  def create_model():
4      """Instantiate and return the configured LLM model."""
5      provider = Config.LLM_PROVIDER
6
7      if provider == "gemini":
8          from .gemini import GeminiModel
9          return GeminiModel()
10
11     if provider == "github":
12         from .github_openai import GitHubModel
13         return GitHubModel()
14
15     raise ValueError(
16         f"Unknown LLM_PROVIDER '{provider}'. "
17         "Supported values: gemini, github"
18     )

```

Each model wrapper is a thin class that initializes the LLM and exposes `get_llm_with_tools()`:

```

1  # src/models/gemini.py
2
3  class GeminiModel:
4      def __init__(self):
5          self.llm = ChatGoogleGenerativeAI(
6              model=Config.GEMINI_MODEL,
7              google_api_key=Config.GEMINI_API_KEY,
8              temperature=Config.TEMPERATURE
9          )
10
11     def get_llm_with_tools(self, tools: list):
12         return self.llm.bind_tools(tools)

```

```

1  # src/models/github_openai.py
2
3  class GitHubModel:
4      def __init__(self):
5          self.llm = ChatOpenAI(
6              model=Config.GITHUB_MODEL,
7              api_key=Config.GITHUB_TOKEN,
8              base_url=Config.GITHUB_ENDPOINT,
9              temperature=Config.TEMPERATURE,
10         )
11
12     def get_llm_with_tools(self, tools: list):
13         return self.llm.bind_tools(tools)

```

Both follow the same interface: `__init__` sets up the LLM, `get_llm_with_tools` binds tools to it. The agent doesn't care which one it gets. It calls `create_model()` and works with whatever comes back.

To switch from Gemini to GitHub Models, change one line in your `.env`:

```

1  LLM_PROVIDER=github
2  GITHUB_TOKEN=your_token_here
3  GITHUB_MODEL=openai/gpt-4.1

```

The GitHub Models endpoint is OpenAI-compatible, so we use `ChatOpenAI` from `langchain-openai` with a custom `base_url`. This is a pattern you'll see frequently—many AI providers expose OpenAI-compatible APIs.

The Agent

This is the core of the chapter. The `LogAnalyzerAgent` manages the conversation, executes tool calls, and handles the approval flow.

The Tool-Calling Loop

LLMs with tool-calling don't just give you text. They return structured instructions: “call this function with these arguments.” Your code executes the function, sends the result back, and the LLM decides what to do next. This continues until the LLM produces a final text answer.

Here's the flow:

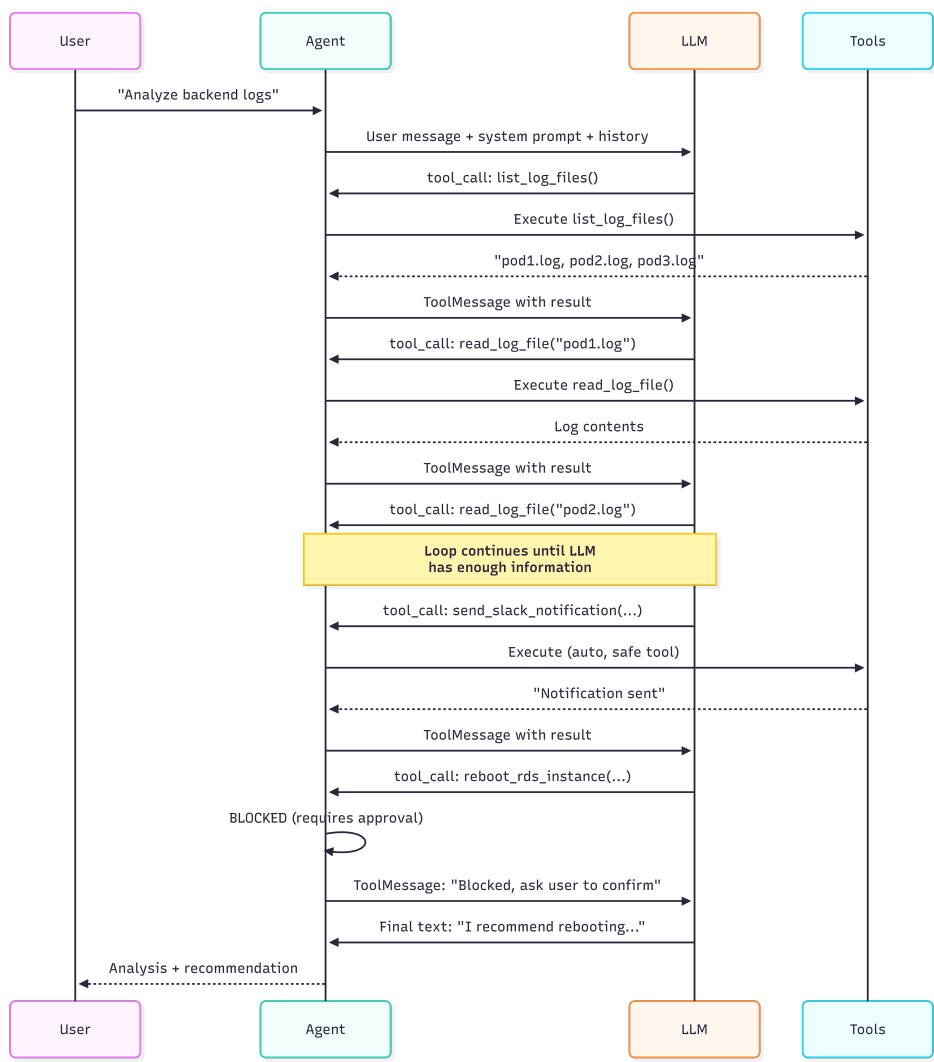


Figure 3. 10.03.png

The agent doesn't control which tools the LLM calls or in what order. It controls what happens when a tool call arrives. Safe tools execute immediately. Dangerous tools get blocked.

Implementation

```

1  # src/agents/log_analyzer.py
2
3  CONFIRMATIONS = {
4      "yes", "y", "confirm", "approve", "go ahead",
5      "do it", "ok", "proceed", "sure", "yeah", "yep",
6  }
7
8
9  def is_confirmation(text: str) -> bool:
10     """Return True if the text is a short user confirmation."""
11     return text.strip().lower().rstrip("!.," ) in CONFIRMATIONS

```

The `is_confirmation()` function lives at module level, not inside the agent class. It's a pure function—takes a string, returns a boolean. We define the confirmation words as a set so lookup is fast and the list is easy to extend.

The agent class has three methods: `process_query`, `_tool_loop`, and `_execute_tool_call`.

```

1  class LogAnalyzerAgent:
2      def __init__(self):
3          self.model = create_model()
4          self.tools = get_all_tools()
5          self.llm = self.model.get_llm_with_tools(self.tools)
6          self.prompt = ChatPromptTemplate.from_messages([
7              ("system", Config.get_system_prompt()),
8              MessagesPlaceholder(variable_name="chat_history"),
9              ("user", "{input}"),
10         ])

```

The prompt template uses `MessagesPlaceholder` for chat history. This means the agent receives the full conversation—every previous question and answer—as context. The LLM sees what tools it called before, what the user said, and what it recommended. When the user says “yes” to confirm an action, the LLM knows what it’s confirming because the recommendation is right there in the history.

Process Query


```

1  def process_query(self, user_input: str, chat_history: list = None,
2                      callbacks=None) -> str:
3      chat_history = chat_history or []
4
5      # Should we let infrastructure tools through this turn?
6      approval_granted = is_confirmation(user_input)
7
8      messages = self.prompt.format_messages(
9          chat_history=chat_history, input=user_input,
10     )
11
12     if callbacks:
13         callbacks.on_thinking()
14
15     response = self.llm.invoke(messages)
16     return self._tool_loop(response, messages, approval_granted, callbacks)

```

The critical line is `approval_granted = is_confirmation(user_input)`. The agent checks if the user's message is a confirmation before doing anything else. If the user typed “yes”, every infrastructure tool call in this turn gets allowed through.

This is a deliberate design choice. The approval check happens inside the agent, not in the UI. The UI doesn't need to know about tool classification or approval logic. It just sends the user's message and gets a response. This separation means you can swap Streamlit for a CLI, an API endpoint, or a Slack bot without touching the approval mechanism.

The Tool Loop

```

1  def _tool_loop(self, response, messages: list,
2                  approval_granted: bool, callbacks) -> str:
3      last_text = ""
4
5      for _ in range(Config.MAX_ITERATIONS):
6          # No tool calls -> return the final answer
7          if not getattr(response, "tool_calls", None):
8              text = extract_response_text(response)
9              return text or last_text or "No response generated."
10
11         # Keep any intermediate text
12         intermediate = extract_response_text(response)
13         if intermediate:
14             last_text = intermediate
15
16         if callbacks and intermediate:

```

```

17         callbacks.on_reasoning(intermediate)
18
19     # Execute each tool call
20     tool_results = []
21     for tc in response.tool_calls:
22         result_msg = self._execute_tool_call(
23             tc, approval_granted, callbacks,
24         )
25         tool_results.append(result_msg)
26
27     # Feed results back to the LLM
28     messages.append(AIMessage(
29         content=response.content,
30         tool_calls=response.tool_calls,
31     ))
32     messages.extend(tool_results)
33
34     if callbacks:
35         callbacks.on_thinking()
36
37     response = self.llm.invoke(messages)
38
39     # Exhausted iterations
40     text = extract_response_text(response)
41     return text or last_text or "Reached maximum analysis steps."

```

This loop does three things:

1. Check if the LLM returned tool calls. If not, we have the final answer.
2. Execute each tool call and collect results as `ToolMessage` objects.
3. Append everything to the message list and send it back to the LLM.

The `MAX_ITERATIONS` guard prevents infinite loops. If the LLM keeps calling tools without converging on an answer, we stop after 10 iterations and return whatever text we have.

Notice how we save `intermediate` text. Some LLMs return both text and tool calls in the same response—a sentence of reasoning alongside a tool invocation. We capture that text and show it in the UI as reasoning steps. If the final response is empty (which can happen with certain providers), we fall back to the last intermediate text.

Execute Tool Call

```

1  def _execute_tool_call(self, tc: dict, approval_granted: bool,
2      callbacks) -> ToolMessage:
3      name, args, call_id = tc["name"], tc["args"], tc["id"]
4
5      # Block infrastructure tools until user confirms
6      if requires_approval(name) and not approval_granted:
7          if callbacks:
8              callbacks.on_approval_skipped(name, args)
9              return ToolMessage(
10                 content=(
11                     f"Action '{name}' requires human approval and was not executed. "
12                     "Present your findings and ask the user to confirm. "
13                     "When the user confirms, call this tool again -- "
14                     "the system will allow it through."
15                 ),
16                 tool_call_id=call_id,
17             )
18
19      # Execute the tool
20      if callbacks:
21          callbacks.on_tool_start(name, args)
22
23      tool_func = self._find_tool(name)
24      if not tool_func:
25          return ToolMessage(content=f"Tool '{name}' not found",
26                             tool_call_id=call_id)
27
28      try:
29          result = str(tool_func.invoke(args))
30          if callbacks:
31              callbacks.on_tool_end(name, result, success=True)
32          return ToolMessage(content=result, tool_call_id=call_id)
33      except Exception as e:
34          if callbacks:
35              callbacks.on_tool_end(name, str(e), success=False)
36          return ToolMessage(content=f"Error: {e}", tool_call_id=call_id)

```

This is where the approval flow actually happens. When the agent wants to call `reboot_rds_instance` and `approval_granted` is `False`, it returns a `ToolMessage` that tells the LLM two things: the action was blocked, and it should ask the user to confirm and then call the tool again.

That second part is crucial. Without it, the LLM would just report “the action was blocked” and stop. With it, the LLM understands it’s in a workflow: present findings, get approval, retry. When the user says “yes” on the next turn, `is_confirmation("yes")` returns `True`, `approval_granted` is `True`, and the `requires_approval` check passes.

Here’s the full approval flow:

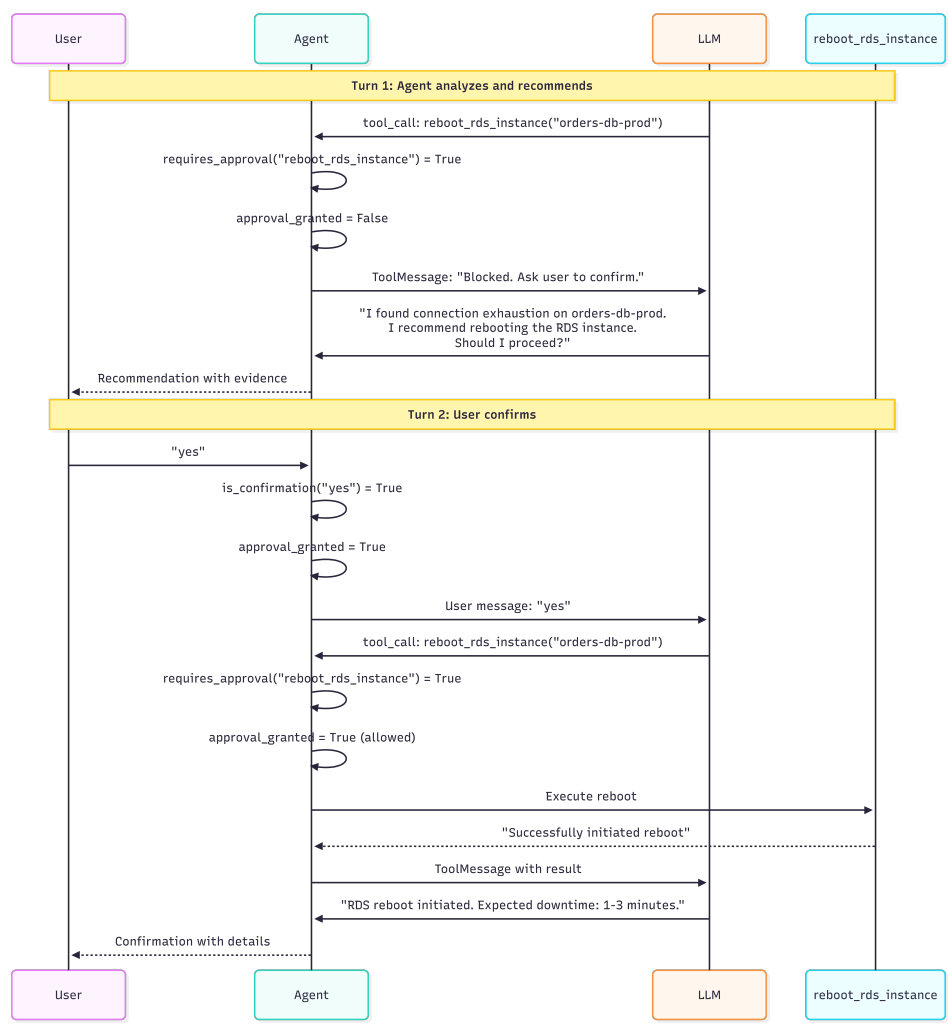


Figure 4. 10.04.png

The beauty of this design is that the LLM handles the conversation naturally. It doesn’t need special instructions for every tool. The system tells it “this was blocked, ask for confirmation” and the LLM figures out how to present that to the user.

The System Prompt

The system prompt defines who the agent is and how it behaves. We keep it in an external file (`system_prompt.txt`) so you can tune it without changing code:

```
1 You are a senior DevOps engineer AI agent. You think and act like an experienced
2 on-call engineer: read logs, correlate evidence across systems, diagnose root
3 causes, take action, and communicate clearly.
4
5 You are monitoring a three-tier AWS application: Java Spring Boot backend on
6   ↪ EKS,
7   RDS MySQL, Redis ElastiCache, CloudFront + S3 frontend.
8
9 Act through tools -- don't just describe what you would do. When you identify an
10 issue, call the relevant tools directly. The system will auto-execute safe
11   ↪ actions
12 and block infrastructure changes until the user confirms.
13
14 Be concise, specific, and evidence-driven. Include instance IDs, pod names,
15 timestamps, and error counts. Only state what the logs show -- if something is
16 unclear, say so.
```

Four paragraphs, four purposes:

1. **Identity:** The agent is a senior DevOps engineer. This framing gives the LLM the right behavioral context—it should think like someone who’s been paged at 2 AM, not like a general-purpose assistant.
2. **Context:** The specific infrastructure it’s monitoring. Without this, the LLM would guess at architecture details.
3. **Action bias:** “Act through tools” is essential. Without it, the LLM tends to describe what it would do rather than actually calling tools. This nudge forces it to use the tools it has.
4. **Communication style:** Evidence-driven, specific, concise. No filler. This prevents the LLM from generating lengthy, vague analysis.

Incident Resolution Guidelines

The `examples.txt` file provides behavioral guidelines that get appended to the system prompt:

```
1 INCIDENT RESOLUTION GUIDELINES:
2
3 1. Gather evidence efficiently.
4   Read full log contents rather than running many individual searches.
5   Read all available logs before drawing conclusions.
6
7 2. Alert the team when you have a diagnosis.
8   Once you have enough evidence, alert the team with severity, affected
9   systems, root cause, and impact. Alert before proposing infrastructure
10  remediation.
11
12 3. Remediate with the smallest effective action.
13   After alerting, choose the least disruptive fix. Infrastructure changes
14   are blocked the first time -- present your recommendation and ask the
15   user to confirm. When the user says yes, call the same action again.
16
17 4. Verify and follow up.
18   After remediation executes, notify the team of the outcome.
19
20 5. Communicate with evidence, not assumptions.
21   Cite specific timestamps, error messages, pod names, and instance IDs.
```

These guidelines shape the agent’s workflow without being rigid scripts. Guideline 2 says “alert before proposing infrastructure remediation”—this ensures the team knows about the incident before you start rebooting databases. Guideline 3 says “call the same action again”—this teaches the LLM the re-invocation pattern for the approval flow.

The separation between `system_prompt.txt` and `examples.txt` is intentional. The system prompt defines the agent’s identity and context. The examples define operational procedures. You can update procedures without changing the agent’s persona.

Configuration

The `Config` class manages all environment variables in one place:

```

1  # src/config.py
2
3  class Config:
4      # LLM Provider selection: 'gemini' (default) or 'github'
5      LLM_PROVIDER = os.getenv('LLM_PROVIDER', 'gemini').lower()
6
7      # Gemini
8      GEMINI_API_KEY = os.getenv('GEMINI_API_KEY')
9      GEMINI_MODEL = os.getenv('GEMINI_MODEL', 'gemini-2.5-flash')
10
11     # GitHub Models
12     GITHUB_TOKEN = os.getenv('GITHUB_TOKEN', '')
13     GITHUB_MODEL = os.getenv('GITHUB_MODEL', 'openai/gpt-5')
14     GITHUB_ENDPOINT = os.getenv('GITHUB_ENDPOINT',
15     ↪ 'https://models.github.ai/inference')
16
17     # AWS
18     AWS_REGION = os.getenv('AWS_REGION', 'us-east-1')
19     AWS_RDS_INSTANCE_ID = os.getenv('AWS_RDS_INSTANCE_ID', '')
20
21     # Slack
22     SLACK_WEBHOOK_URL = os.getenv('SLACK_WEBHOOK_URL', '')
23     SLACK_CHANNEL = os.getenv('SLACK_CHANNEL', '#devops-alerts')
24
25     # Agent
26     MAX_ITERATIONS = 10
27     TEMPERATURE = float(os.getenv('TEMPERATURE', '0.1'))

```

The `validate()` method checks that the selected provider has the required credentials:

```

1  @classmethod
2  def validate(cls):
3      if cls.LLM_PROVIDER == 'gemini' and not cls.GEMINI_API_KEY:
4          raise ValueError(
5              "LLM_PROVIDER=gemini but GEMINI_API_KEY is not set."
6          )
7      if cls.LLM_PROVIDER == 'github' and not cls.GITHUB_TOKEN:
8          raise ValueError(
9              "LLM_PROVIDER=github but GITHUB_TOKEN is not set."
10         )

```

AWS and Slack are optional. If they're not configured, the tools fall back to placeholder mode. This means you can run the agent without any cloud credentials and still see the complete workflow.

The `get_system_prompt()` method loads the prompt from files:

```

1  @classmethod
2  def get_system_prompt(cls) -> str:
3      base_dir = os.path.dirname(os.path.dirname(__file__))
4
5      with open(os.path.join(base_dir, 'system_prompt.txt'), 'r') as f:
6          prompt = f.read()
7
8      examples_file = os.path.join(base_dir, 'examples.txt')
9      try:
10         with open(examples_file, 'r') as f:
11             prompt += '\n\n' + f.read()
12     except FileNotFoundError:
13         pass # examples are optional
14
15     return prompt

```

The system prompt is required—the agent won’t start without it. The examples file is optional, appended if present. This gives you flexibility: you can start with just the persona and add behavioral guidelines later.

The Web Interface

The Streamlit UI from Chapter 8 stays the same in structure. The main change is adding real-time progress feedback so users can see what the agent is doing while it works.

StreamlitProgress

The `StreamlitProgress` class provides callbacks that the agent calls during execution. Each callback updates a `st.status` container in real time:


```

1  TOOL_LABELS = {
2      "list_log_files":      "Listing log files",
3      "read_log_file":      "Reading log file",
4      "search_logs":        "Searching logs",
5      "reboot_rds_instance": "Rebooting RDS instance",
6      "restart_kubernetes_pod": "Restarting Kubernetes pod",
7      "send_slack_notification": "Sending Slack notification",
8  }
9
10
11 class StreamlitProgress:
12     def __init__(self, container):
13         self.status = container
14         self.tool_count = 0
15         self.steps = []
16
17     def on_thinking(self):
18         self.status.update(label="Thinking...", state="running")
19
20     def on_reasoning(self, text: str):
21         if text:
22             self.status.write(f"_{text}_")
23             self.steps.append({"label": "Reasoning", "detail": text})
24
25     def on_tool_start(self, tool_name: str, tool_args: dict):
26         self.tool_count += 1
27         label = TOOL_LABELS.get(tool_name, tool_name)
28         self.status.update(label=f"{label}...", state="running")
29
30     def on_tool_end(self, tool_name: str, result: str, success: bool = True):
31         label = TOOL_LABELS.get(tool_name, tool_name)
32         marker = "OK" if success else "FAIL"
33         preview = _summarize_result(tool_name, result)
34         self.status.write(f"[{marker}] **{label}** - {preview}")
35         self.steps.append({"label": label, "detail": f"[{marker}] {preview}"})
36
37     def on_approval_skipped(self, tool_name: str, tool_args: dict):
38         label = TOOL_LABELS.get(tool_name, tool_name)
39         self.status.write(f"[BLOCKED] **{label}** - requires your approval")
40         self.steps.append({"label": label, "detail": "[BLOCKED] requires
41             ↪ approval"})
42
43     def complete(self):
44         n = self.tool_count
45         if n:
46             self.status.update(
47                 label=f"Done - {n} tool{'s' if n != 1 else ''} used",
48                 state="complete", expanded=False,
49             )
50         else:

```

```
50         self.status.update(label="Done", state="complete", expanded=False)
```

The `TOOL_LABELS` dictionary maps internal tool names to human-readable descriptions. Users don't need to see `read_log_file`—they see “Reading log file.”

Each callback records a step in `self.steps`. These steps are saved into the message history and rendered as a collapsible expander when the conversation is replayed. So when you scroll back through a conversation, you can expand any assistant message and see exactly which tools were used and what they returned.

The `_summarize_result()` function creates a one-line preview for each tool result:

```
1  def _summarize_result(tool_name: str, result: str) -> str:
2      r = str(result)
3      if tool_name == "list_log_files":
4          count = r.count(".log")
5          return f"found {count} log file{'s' if count != 1 else ''}"
6      if tool_name == "read_log_file":
7          return "file read"
8      if tool_name == "search_logs":
9          first_line = r.split("\n")[0]
10         return first_line.lower() if "Found" in first_line else "search
11             ↳ complete"
12     if tool_name == "send_slack_notification":
13         return "notification sent"
14     if tool_name in ("reboot_rds_instance", "restart_kubernetes_pod"):
15         return "initiated"
16     return r[:80]
```

Log file contents can be hundreds of lines. You don't want that in the progress panel. “file read” is enough to know the step succeeded.

The Main Loop

The main application logic is straightforward:

```

1  def main():
2      init_session()
3      sidebar()
4
5      st.info("**Try:** *Analyze backend pod logs and detect issues*")
6      display_history()
7
8      if prompt := st.chat_input("Ask about backend logs..."):
9          st.session_state.messages.append({"role": "user", "content": prompt})
10         with st.chat_message("user"):
11             st.markdown(prompt)
12
13         with st.chat_message("assistant"):
14             status = st.status("Analyzing...", expanded=True)
15             progress = StreamlitProgress(status)
16             history = to_langchain(st.session_state.messages[:-1])
17
18             try:
19                 response = st.session_state.agent.process_query(
20                     user_input=prompt,
21                     chat_history=history,
22                     callbacks=progress,
23                 )
24                 progress.complete()
25             except Exception as e:
26                 progress.error(str(e))
27                 response = f"Error: {e}"
28
29             st.markdown(response)
30
31         st.session_state.messages.append({
32             "role": "assistant",
33             "content": response,
34             "steps": progress.steps,
35         })

```

The flow is: save the user's message, create a progress container, convert the history to LangChain format, call the agent, render the response. Each assistant message includes the thinking steps so they can be replayed.

Notice `to_langchain(st.session_state.messages[:-1])`. We convert all messages except the last one (which is the current user message—the agent receives that as `user_input`). This prevents duplication.

Response Parsing

Different LLM providers return responses in different formats. Gemini might return a string. GitHub Models might return a list of content blocks. The `extract_response_text()` utility handles all cases:

```
1  # src/utils/response.py
2
3  def extract_response_text(response) -> str:
4      text = ""
5
6      if hasattr(response, 'content'):
7          content = response.content
8
9          if isinstance(content, str):
10             text = content
11         elif isinstance(content, list):
12             text_parts = []
13             for block in content:
14                 if isinstance(block, dict) and 'text' in block:
15                     text_parts.append(block['text'])
16                 elif isinstance(block, str):
17                     text_parts.append(block)
18             text = '\n'.join(text_parts)
19     else:
20         text = str(response)
21
22     return text.strip()
```

This function gets called on every LLM response. It's defensive—it handles strings, lists, dictionaries, and anything else by converting to string as a fallback. Without it, you'd get crashes when switching between providers that format responses differently.

Running the Agent

Setup

1. Clone the repository and navigate to the chapter directory:

```
1 cd 03-ai-agent-for-devops/code/10
```

2. Create a virtual environment and install dependencies:

```
1 python -m venv .env
2 source .env/bin/activate
3 pip install -r requirements.txt
```

3. Copy the environment template and add your API key:

```
1 cp .env.example .env
```

Edit `.env` with your preferred provider:

```
1 # Option A: Google Gemini
2 LLM_PROVIDER=gemini
3 GEMINI_API_KEY=your_api_key_here
4
5 # Option B: GitHub Models
6 # LLM_PROVIDER=github
7 # GITHUB_TOKEN=your_github_token_here
8 # GITHUB_MODEL=openai/gpt-4.1
```

4. Start the application:

```
1 make run
2 # or: streamlit run app.py
```

What Happens Without AWS Credentials

If you don't have AWS configured, the workflow is identical. The only difference is the RDS tool prints "PLACEHOLDER" to your terminal and returns a simulated success. The agent's reasoning, the Slack notification, and the approval flow all work the same way. This is by design—you can learn the full workflow locally and add real credentials later.

Key Design Decisions

Looking back at this chapter, three design decisions shaped the entire implementation:

1. Approval Lives in the Agent

The UI doesn't know which tools are safe and which aren't. The agent makes that decision. This means you can put any frontend on top—Streamlit, a REST API, a Slack bot—and the safety mechanisms travel with the agent.

2. The LLM Drives the Workflow

We don't hard-code the sequence of tool calls. The LLM reads the logs, decides it needs to alert the team, decides the database needs a reboot, and asks for permission. If you add a new tool tomorrow, the LLM can start using it without you changing the agent code. This is the tool use pattern from Chapter 10 in the book—the agent picks the right tool for the situation.

3. Placeholder Mode Is a Feature

Real/placeholder duality in every external tool means you can develop and test the full workflow without cloud credentials. The agent's reasoning doesn't change. The approval flow doesn't change. Only the final API call differs. When you're ready for production, you configure the credentials and the tools switch to real mode automatically.

What You've Learned

You started this chapter with a single-tool, placeholder-only agent. Now you have a system that:

- Reads and correlates logs across multiple pods
- Detects patterns like connection pool exhaustion
- Sends structured Slack alerts with severity and evidence
- Reboots AWS RDS instances with proper status checking

- Blocks dangerous operations until a human confirms
- Shows real-time progress as each tool executes
- Supports multiple LLM providers through a factory pattern
- Works locally with placeholders and in production with real APIs

The agent's architecture follows a clean separation: the UI handles display, the agent handles logic, the tools handle execution, and the config handles environment. Each layer can change independently.

More importantly, you've seen how to build an approval flow that works within LLM tool-calling. The pattern—block on first call, send a message telling the LLM to retry, detect the user's confirmation, allow on second call—is reusable. Every new infrastructure tool you add automatically gets the same safety mechanism just by adding its name to `APPROVAL_REQUIRED_TOOLS`.

What's Next

You have a working agent with real tools and a clean approval flow. But it forgets everything between sessions. If the same database connection issue happened last week, the agent has no memory of it.

In **Chapter 11: Memory and State Management**, you'll add persistence to the agent. It will remember past incidents, track recurring patterns, and learn from previous investigations. You'll implement LangChain's memory systems and build a storage layer that survives restarts.

The agent knows how to act. Next, it learns how to remember.